

UNIVERSIDAD POLITÉCNICA DE MADRID

ESCUELA TÉCNICA SUPERIOR DE INGENIEROS DE TELECOMUNICACIÓN



MÁSTER UNIVERSITARIO EN INGENIERÍA DE
TELECOMUNICACIÓN

TRABAJO FIN DE MÁSTER

DESARROLLO DE UNA PLATAFORMA DE
COMUNICACIÓN CON PERIFÉRICOS SOBRE
MPSOC DE ALTAS PRESTACIONES PARA
ORDENADORES DE A BORDO EN
SISTEMAS ESPACIALES

JORGE ALEJANDRO ESTEFANÍA HIDALGO

MÁSTER UNIVERSITARIO EN INGENIERÍA DE TELECOMUNICACIÓN

TRABAJO FIN DE MÁSTER

Título: Desarrollo de una plataforma de comunicación con periféricos sobre MPSoC de altas prestaciones para Ordenadores de a Bordo en sistemas espaciales.

Autor: D. Jorge Alejandro Estefanía Hidalgo

Tutor: D. Daniel Sánchez García

Ponente: D. Álvaro Araujo Pinto

Departamento: Departamento de Ingeniería Electrónica

MIEMBROS DEL TRIBUNAL

Presidente: D.

Vocal: D.

Secretario: D.

Suplente: D.

Los miembros del tribunal arriba nombrados acuerdan otorgar la calificación de:
.....

Madrid, a de de 20...

UNIVERSIDAD POLITÉCNICA DE MADRID

ESCUELA TÉCNICA SUPERIOR DE INGENIEROS DE TELECOMUNICACIÓN



MÁSTER UNIVERSITARIO EN INGENIERÍA DE
TELECOMUNICACIÓN

TRABAJO FIN DE MÁSTER

**Desarrollo de una plataforma de
comunicación con periféricos sobre
MPSoC de altas prestaciones para
Ordenadores de a Bordo en sistemas
espaciales**

Jorge Alejandro Estefanía Hidalgo

RESUMEN

Este Trabajo de Fin de Máster presenta el desarrollo de una plataforma de comunicación con periféricos sobre el MPSoC Zynq UltraScale+ ZCU102, en el marco del proyecto LINCE, una iniciativa del PERTE Aeroespacial liderada por Indra para el desarrollo de microsátélites de órbita baja.

El trabajo abarca cinco áreas principales.

En primer lugar, el diseño e implementación en VHDL de un transceptor serie altamente configurable para la lógica programable del MPSoC.

En segundo lugar, el desarrollo de un driver serie en C sobre el sistema operativo en tiempo real RTEMS en el sistema de procesamiento del MPSoC, que utiliza el transceptor serie desarrollado y ofrece una API pública que abstrae completamente del hardware. En tercer lugar, el diseño y fabricación de tres placas de circuito impreso de ampliación de la ZCU102 que se conectan a ella para expandir sus capacidades y permiten ampliar la ventana de desarrollo. La primera placa ofrece soporte hardware para 7 líneas RS422 y 7 líneas RS485, las cuales se pueden conectar en buses compartidos configurables por hardware. La segunda placa ofrece soporte para 3 líneas RS422 o RS485 seleccionable, dos líneas CAN, cuatro líneas PWM y cuatro líneas analógicas. La tercera placa ofrece soporte para cinco líneas RS422 o RS485 seleccionable, tres pares de líneas de potencia para controlar motores por PWM y dos líneas SpaceWire.

En cuarto lugar, el estudio del transporte de datos entre el sistema de procesamiento y la lógica programable, que resultó ser el factor determinante del rendimiento del conjunto. Se implementaron y midieron tres arquitecturas completas sobre el mismo transceptor: una basada en AXI GPIO, otra con catorce controladores de acceso directo a memoria (DMA) y una tercera con un único controlador multicanal (MCDMA) gobernado por un puente diseñado en VHDL. La comparativa demuestra que la primera interrumpe al procesador una vez por byte y se queda en el 30 % del techo de la línea, mientras que la tercera alcanza el 99 % con cuatrocientas veces menos interrupciones.

En quinto lugar, la validación de la plataforma mediante el desarrollo de herramientas de software y hardware adicionales, que permitieron verificar el correcto funcionamiento del driver serie y de las distintas funcionalidades de las placas de ampliación.

El conjunto del trabajo constituye una plataforma funcional y validada que el equipo de Sener, Indra y el laboratorio B105 pueden utilizar como base para el subsistema de comunicaciones del ordenador de a bordo del satélite LINCE en el futuro.

SUMMARY

[Pendiente de redacción — máximo 500 palabras.]

PALABRAS CLAVE

MPSoC, FPGA, VHDL, RTEMS, RS422, RS485, CAN, SpaceWire, driver serie, PCB, AXI, DMA, MCDMA, AXI-Stream, sistemas empotrados, tiempo real, satélite, ordenador de a bordo.

KEYWORDS

[Pendiente de redacción.]

Agradecimientos

...

Índice

Resumen y Palabras Clave	II
Agradecimientos	IV
Lista de acrónimos	XI
1. Introducción y objetivos	1
1.1. Introducción y motivación	1
1.2. Objetivos	2
1.3. Metodología	2
1.4. Estructura de la memoria	4
2. Marco teórico	5
2.1. Proyecto LINCE	5
2.2. MPSoC Zynq UltraScale+ ZCU102	5
2.3. Vivado y Vitis (Xilinx): PS y PL	6
2.3.1. Vivado Design Suite	6
2.3.2. Vitis Unified Software Platform	6
2.4. RTEMS	7
2.5. Comunicaciones serie	7
2.5.1. RS485	7
2.5.2. RS422	8
2.5.3. SpaceWire	8
2.5.4. CAN	8
2.6. Transporte de datos entre el PS y la PL	9
2.6.1. Bus AXI	9
2.6.2. Acceso directo a memoria (DMA)	10
3. Desarrollo	11
3.1. Preparación del entorno de desarrollo	11
3.1.1. Instalación de Vivado y Vitis	11
3.1.2. Instalación de RTEMS	11
3.1.3. Establecimiento de un flujo de desarrollo completo	12
3.2. Diseño del transceptor serie configurable (PL)	17
3.2.1. Oscilador controlado numéricamente (NCO)	18
3.2.2. Transmisor	19
3.2.3. Receptor	21
3.2.4. Registro de desplazamiento	24
3.2.5. FIFO	24
3.2.6. Bloque de canal	24
3.2.7. Bloque <i>top</i> del transceptor	25
3.2.8. Bloque Zynq UltraScale+ MPSoC	27

3.3.	Herramientas para facilitar el desarrollo del transceptor (PL)	27
3.3.1.	Generación de transceivers con TCL	27
3.3.2.	Generación de proyecto con TCL	27
3.4.	Diseño del driver serie en RTEMS (PS)	27
3.4.1.	Interfaz hardware: mapa de registros	28
3.4.2.	Arquitectura software	28
3.4.3.	Descubrimiento dinámico del hardware	29
3.4.4.	Modelo de interrupciones: ISR maestra y tareas worker	29
3.4.5.	Inicialización y configuración del hardware	29
3.4.6.	API pública	30
3.4.7.	Decisiones de diseño destacadas	30
3.5.	Diseño hardware	30
3.5.1.	Selección de componentes: la restricción de 1,8 V	31
3.5.2.	Reglas de diseño y proceso de fabricación	32
3.5.3.	Diseño de la placa de comunicación serie	32
3.5.4.	Diseño de la placa CDHS	34
3.5.5.	Diseño de la placa AOCS	37
3.5.6.	Fabricación	38
3.6.	Desarrollo de herramientas de testing	39
3.6.1.	Aplicación de testing del driver serie en RTEMS	39
3.6.2.	Aplicación de testing de las placas CDHS y AOCS	41
3.6.3.	Validación de hardware: arneses y equipamiento	41
3.7.	Arquitecturas de transporte PS-PL	43
3.7.1.	Variante A — AXI GPIO (referencia)	44
3.7.2.	Variante B — catorce AXI DMA	44
3.7.3.	Variante C — AXI MCDMA con puente VHDL	46
3.7.4.	Verificación y depuración iterativa del driver	48
3.7.5.	Banco de pruebas: loopback en la PL	50
4.	Resultados	52
4.1.	Validación CDHS	52
4.1.1.	Montaje	52
4.1.2.	Comunicaciones serie RS422/RS485	53
4.1.3.	Bus CAN	54
4.1.4.	ADC SPI (termistores)	55
4.1.5.	PWM (calentadores)	56
4.2.	Validación AOCS	57
4.2.1.	Montaje	57
4.2.2.	Comunicaciones serie RS422/RS485	57
4.2.3.	PWM (puentes en H)	58
4.3.	Validación de la placa de comunicación serie	58
4.3.1.	Prueba previa: 14 transceptores simultáneos	58
4.3.2.	Comparativa de las arquitecturas de transporte PS-PL	59
4.3.3.	Caracterización eléctrica de la placa de comunicación serie	66
5.	Conclusiones y líneas futuras	72
5.1.	Conclusiones	72
5.2.	Líneas futuras	73

Bibliografía	76
Anexo 1: Mapas de señales y tabla NCO	79
Anexo A: Aspectos éticos, económicos, sociales y ambientales	85
Anexo B: Presupuesto económico	87

Índice de figuras

3.1. <i>Block Design</i> del ejemplo de la puerta AND: bloque Zynq UltraScale+ MPSoC, los dos AXI GPIO y el módulo VHDL AND_GATE	13
3.2. Generador de imágenes de arranque de Vitis, con los componentes que conforman el BOOT.bin	14
3.3. Circuito del oscilador controlado numéricamente (NCO).	19
3.4. Diagrama FSM del transmisor serie.	20
3.5. Diagrama FSM del receptor serie.	23
3.6. Arquitectura general de la placa de comunicación serie: los siete drivers RS422 (Driver7–Driver13, arriba), el conector FMC (P2, izquierda) y los siete drivers RS485 (Driver0–Driver6, abajo).	33
3.7. Diagrama de bloques de la placa CDHS.	34
3.8. Render 3D de la placa CDHS.	34
3.9. Subsistema CAN de la placa CDHS.	35
3.10. Canal RS de la placa CDHS con THVD1424RGTR.	36
3.11. Diagrama de bloques de la placa AOCS.	37
3.12. Render 3D de la placa AOCS.	37
3.13. Subsistema SpaceWire de la placa AOCS.	37
3.14. Aplicación de pasta de soldadura con stencil.	38
3.15. Placa CDHS tras el proceso de soldadura.	38
3.16. Placa AOCS tras el proceso de soldadura.	39
3.17. Arnés de <i>loopback</i> RS422 a 4 hilos entre dos conectores D-Sub-9.	42
3.18. Arnés de <i>loopback</i> RS485 a 2 hilos entre dos conectores D-Sub-9.	42
3.19. Arnés de interconexión CAN nominal–redundante sobre el conector J1.	43
3.20. Arquitectura asimétrica de la variante MCDMA: un canal de transmisión y catorce de recepción.	46
4.1. Sistema completo montado: ZCU102 con la placa CDHS conectada al FMC.	52
4.2. Placa CDHS con componentes montados (ambas caras).	53
4.3. Captura completa del terminal de pruebas RS en la placa CDHS.	54
4.4. Señal diferencial CAN sin resistencias de terminación.	55
4.5. Señal diferencial CAN con resistencias de terminación ($60\ \Omega + 60\ \Omega$).	55
4.6. Captura completa del terminal de lectura ADC (placa CDHS).	56
4.7. Señal PWM medida en el conector J5 de la placa CDHS: canal de 1 kHz con <i>duty cycle</i> del 50%.	56
4.8. Placas AOCS con componentes montados (vista superior de las dos unidades fabricadas).	57
4.9. Captura completa del terminal de pruebas RS en la placa AOCS.	58
4.10. Señales PWM de control de motor en la placa AOCS (par complementario para puente en H).	58
4.11. Ocupación de la FPGA por variante, en porcentaje de los recursos del dispositivo. Menos es mejor. El banco de catorce DMA triplica largamente el consumo de la solución multicanal para el mismo número de canales.	60

4.12. Interrupciones por kilobyte transferido. Menos es mejor: cada interrupción es una expropiación de la CPU. Las barras de DMA14 y MCDMA son invisibles a esta escala, y eso es precisamente el resultado. La cifra de DMA14 recoge solo la transmisión.	61
4.13. Latencia media de entrega de un frame de 11 bytes. Menos es mejor. Se excluye DMA14 por carecer de lazo de recepción.	62
4.14. <i>Throughput</i> de un canal frente al techo físico de la línea a 115 200 8N1. Más es mejor. El MCDMA lo alcanza al 99 %; la variante GPIO se queda en el 30 %.	63
4.15. <i>Throughput</i> frente a la velocidad de línea, en escala logarítmica en ambos ejes. El MCDMA crece de forma proporcional a la velocidad del enlace; la variante GPIO es una recta plana, porque su límite no está en la línea sino en el coste de una interrupción por byte. En el punto de 4 Mbaudios la diferencia es de 70 veces.	64
4.16. PCB de comunicación serie configurada para la campaña de caracterización, con los <i>jumpers</i> fijando la topología de la tabla 4.9.	67
4.17. Velocidad máxima con tasa de error nula en cada bus de la PCB, según el valor del bit SLO. El pin SLO del LTC2865 es un habilitador de modo lento activo a nivel bajo: con el valor por defecto (0) el limitador de <i>slew rate</i> está activo y limita el driver a 250 kbps nominales, con lo que el bus multipunto alcanza 460 kbps. Desactivándolo (1), los tres buses operan de forma limpia a 4 Mbps.	69
4.18. Tasa de error del bus A (RS-485 multipunto, 7 nodos) frente a la velocidad de línea, con y sin el limitador de <i>slew</i> . Con el limitador activo —el valor por defecto— el enlace se degrada a partir de 460 kbps y colapsa por completo a 2 Mbps, coherente con el máximo nominal de 250 kbps del chip en ese modo; desactivándolo la tasa de error es nula en todo el barrido. Los puntos de 3 y 4 Mbps no existen en la curva de SLO = 0 porque el barrido se interrumpe en cuanto un punto no deja pasar un solo byte correcto. . . .	70

Índice de tablas

3.1. Posiciones del selector de modo de arranque SW6 de la ZCU102. Adaptada de [1].	15
3.2. Puertos del transmisor TX_CONFIGURABLE_SERIAL.	19
3.3. Puertos del receptor RX_CONFIGURABLE_SERIAL.	22
3.4. Reparto de bits de los vectores de interfaz del bloque CONFIGURABLE_SERIAL.TOP.	26
3.5. API pública del driver serie.	30
3.6. Control complementario DE/RE en el THVD1424.	36
3.7. Las tres arquitecturas de transporte implementadas y comparadas.	44
3.8. Mapa de registros del bloque TX_ISR_EOF_HANDLER (base 0xA0001000).	47
3.9. Mapa de memoria de la variante MCDMA.	48
3.10. Estructura del descriptor de buffer (BD) del MCDMA.	48
3.11. Topología de buses reproducida por el módulo de <i>loopback</i> de la PL.	50
4.1. Ocupación de la lógica programable de las tres variantes sobre el XCZU9EG.	59

4.2.	Huella de software del driver de cada variante (tamaño de .text y recursos RTEMS reservados en ejecución).	60
4.3.	Interrupciones por kilobyte transferido.	61
4.4.	Latencia de entrega de un frame de 11 bytes, del envío en el maestro a la recepción completa en el esclavo (115 200 8N1).	62
4.5.	<i>Throughput</i> en B/s. El porcentaje es sobre el techo físico de la línea (11 520 B/s).	62
4.6.	<i>Throughput</i> (B/s) frente a la velocidad de línea (baudios). La variante DMA14 se omite por carecer de lazo de recepción.	63
4.7.	Rechazo de frames corruptos (T7) y comportamiento en desbordamiento del <i>ring</i> de software (T5).	64
4.8.	Síntesis de la comparativa. En negrita, el mejor valor de cada fila.	65
4.9.	Topología de buses de la PCB de comunicación serie.	66
4.10.	Resultado de los nueve tests de caracterización de la PCB.	68
4.11.	Integridad del bus multipunto (T3) frente a la velocidad, con SLO = 0. El escalón entre 460 kbps y 1 Mbps es nítido.	69
4.12.	Tasa de error en régimen permanente (T9), 10 s de tráfico continuo por bus.	71
5.1.	Mapa de señales entre la placa LINCE Comunicación Serial y el conector FMC HPC0 (P2) de la ZCU102.	80
5.2.	Mapa de señales entre la placa CDHS y el conector FMC HPC0 de la ZCU102.	81
5.3.	Mapa de señales entre la placa AOCS y el conector FMC HPC0 de la ZCU102.	82
5.4.	Tabla NCO: frecuencias de baudrate soportadas y error medido (extracto, modo <i>full-bit</i>).	83
5.5.	Distribución de las líneas de código desarrolladas en el trabajo.	84
5.6.	Presupuesto económico	87

Lista de acrónimos

AOCS *Attitude and Orbit Control Subsystem* — Subsistema de control de actitud y órbita.

ADC *Analog-to-Digital Converter* — Convertidor analógico-digital.

AMBA *Advanced Microcontroller Bus Architecture* — Familia de estándares de interconexión de ARM.

AXI *Advanced eXtensible Interface* — Bus de interconexión de alto rendimiento del estándar ARM AMBA.

BD *Buffer Descriptor* — Descriptor de *buffer*: estructura en memoria que define una transferencia de DMA en modo *scatter-gather*.

BRAM *Block RAM* — Bloque de memoria RAM integrado en la FPGA.

BSP *Board Support Package* — Paquete de soporte de placa.

BOM *Bill of Materials* — Lista de materiales de una PCB.

CAN *Controller Area Network* — Bus de comunicaciones serie multipunto.

CDHS *Command and Data Handling System* — Subsistema de mando y gestión de datos.

CMRR *Common Mode Rejection Ratio* — Razón de rechazo al modo común.

COTS *Commercial Off-The-Shelf* — Componente comercial de catálogo, no cualificado específicamente para espacio.

DDR *Double Data Rate* — Memoria dinámica principal del sistema de procesamiento.

DMA *Direct Memory Access* — Acceso directo a memoria sin intervención de la CPU.

DSP *Digital Signal Processor* — Bloque de procesamiento digital de señal.

DTB *Device Tree Binary* — Árbol de dispositivos en formato binario para sistemas Linux/U-Boot.

EMI *Electromagnetic Interference* — Interferencia electromagnética.

ESD *Electrostatic Discharge* — Descarga electrostática.

ESA *European Space Agency* — Agencia Espacial Europea.

ETSIT Escuela Técnica Superior de Ingenieros de Telecomunicación.

FIFO *First In, First Out* — Estructura de datos de cola.

FMC *FPGA Mezzanine Card* — Tarjeta de expansión para FPGA según estándar VITA 57.1.

FPGA *Field Programmable Gate Array* — Matriz lógica programable en campo.

FSM *Finite State Machine* — Máquina de estados finitos.

FSBL *First Stage Boot Loader* — Cargador de arranque de primera etapa.

GIC *Generic Interrupt Controller* — Controlador de interrupciones del procesador ARM.

HDL *Hardware Description Language* — Lenguaje de descripción de hardware.

INTC *Interrupt Controller* — Controlador de interrupciones (IP de la lógica programable).

IRQ *Interrupt Request* — Petición de interrupción.

ISR *Interrupt Service Routine* — Rutina de servicio de interrupción.

LEO *Low Earth Orbit* — Órbita terrestre baja.

LINCE Línea de industrialización de cargas de pago y plataformas espaciales.

LVDS *Low-Voltage Differential Signaling* — Señalización diferencial de bajo voltaje.

LUT *Look-Up Table* — Tabla de búsqueda, bloque básico de una FPGA.

MCDMA *Multichannel Direct Memory Access* — Controlador de DMA multicanal de Xilinx (IP `axi_mcdma`, PG288).

MMIO *Memory-Mapped I/O* — Entrada/salida con mapeado en memoria.

MM2S *Memory-Mapped to Stream* — Canal de un DMA que lee de memoria y produce un flujo AXI-Stream (camino de transmisión).

MMU *Memory Management Unit* — Unidad de gestión de memoria; define, entre otras cosas, qué regiones son cacheables.

MPSoC *Multiprocessor System-on-Chip* — Sistema multiprocesador en chip.

NCO *Numerically Controlled Oscillator* — Oscilador controlado numéricamente.

OBC *On-Board Computer* — Ordenador de a bordo.

PCB *Printed Circuit Board* — Placa de circuito impreso.

PERTE Proyecto Estratégico para la Recuperación y Transformación Económica — Instrumento de financiación pública del Gobierno de España.

PL *Programmable Logic* — Lógica programable (FPGA del MPSoC).

PMUFW *Power Management Unit Firmware* — Firmware de la unidad de gestión de potencia.

PS *Processing System* — Sistema de procesamiento (ARM del MPSoC).

PWM *Pulse Width Modulation* — Modulación por ancho de pulso.

RTOS *Real-Time Operating System* — Sistema operativo de tiempo real.

RTEMS *Real-Time Executive for Multiprocessor Systems* — RTOS de código abierto para sistemas empuetrados.

S2MM *Stream to Memory-Mapped* — Canal de un DMA que consume un flujo AXI-Stream y lo escribe en memoria (camino de recepción).

SG *Scatter-Gather* — Modo de operación de un DMA en el que las transferencias se describen mediante una lista enlazada de descriptores en memoria.

SMD *Surface Mount Device* — Componente de montaje superficial.

SPI *Serial Peripheral Interface* — Interfaz serie para periféricos.

TCL *Tool Command Language* — Lenguaje de scripting utilizado en Vivado.

TFM Trabajo Fin de Máster.

TVS *Transient Voltage Suppressor* — Supresor de sobretensiones transitorias.

UART *Universal Asynchronous Receiver-Transmitter* — Transceptor serie asíncrono universal.

UPM Universidad Politécnica de Madrid.

VHDL *Very High Speed Integrated Circuit Hardware Description Language* — Lenguaje de descripción de hardware.

WNS *Worst Negative Slack* — Peor margen de temporización de un diseño implementado; positivo indica que se cumplen las restricciones.

XSA *Xilinx Support Archive* — Archivo de especificación hardware exportado por Vivado para Vitis.

1. Introducción y objetivos

1.1. Introducción y motivación

Este trabajo se enmarca en el proyecto **LINCE** (*Línea de INdustrialización de Cargas de pago y plataformas Espaciales*), una iniciativa del PERTE Aeroespacial liderada por Indra que impulsa la producción en serie de plataformas de microsátélites (100–200 kg) y de sus cargas de pago para órbita baja (LEO), con el objetivo de reducir costes y plazos y consolidar la autonomía estratégica española en este segmento; su constelación demostradora, STARTICAL, validará en vuelo tecnologías críticas de comunicaciones y vigilancia [2].

Dentro del consorcio, Sener participa como socio tecnológico: desarrolla y valida el firmware del OBC, y traslada a los demás participantes los requisitos técnicos que recibe de Indra, líder del consorcio. La Universidad Politécnica de Madrid participa a través del grupo B105 Electronic Systems Lab, responsable del subsistema de comunicaciones del ordenador de a bordo (OBC); este trabajo se desarrolla dentro de esa responsabilidad. El proyecto sigue el enfoque **NewSpace**: uso de componentes comerciales (COTS, *commercial off-the-shelf*) frente a los cualificados para espacio, trasladando la fiabilidad desde la pieza individual hacia la arquitectura del sistema.

El OBC concentra el control del satélite y se comunica con los periféricos de la plataforma (propulsores, cámaras, seguidores de estrellas, sensores térmicos) mediante enlaces serie. Los requisitos de LINCE fijan catorce enlaces RS422/RS485 operando de forma simultánea, cada uno con sus propios parámetros de línea. Esa cifra obliga a estudiar el transporte de datos desde cada transceptor hasta la aplicación de usuario. Hay que balancear los recursos del sistema de manera que se minimicen el área de la lógica programable (PL) y el uso de CPU del sistema de procesamiento (PS) sin penalizar el rendimiento del conjunto. La ZCU102, plataforma de desarrollo basada en el MPSoC Zynq UltraScale+ escogida para el prototipado, no dispone ni de tantos transceptores físicos serie ni de los conectores de los periféricos del satélite, de modo que el propio banco de ensayo forma parte del problema a resolver.

El autor se incorporó al laboratorio B105 Electronic Systems Lab como becario para trabajar en el proyecto LINCE debido a su interés en proyectos espaciales y satelitales. La aportación inicial se centró en el desarrollo de firmware en VHDL y C para las comunicaciones serie RS422 y RS485 sobre la ZCU102. La necesidad de ejercitar ese firmware en condiciones eléctricas reales amplió después el alcance del trabajo al diseño y fabricación del hardware que completa el banco de ensayo.

1.2. Objetivos

El objetivo principal de este trabajo es desarrollar una plataforma funcional y validada de comunicación con periféricos serie sobre el MPSoC Zynq UltraScale+ ZCU102, que sirva como base para el OBC del proyecto LINCE. Para alcanzarlo, se plantearon los siguientes objetivos específicos:

1. **Diseñar un módulo VHDL de transceptor serie configurable** para la lógica programable del MPSoC, capaz de operar sobre los estándares RS422 y RS485, con parámetros configurables en tiempo de ejecución: baudrate, paridad, número de bits de datos, bits de parada, orden de bit y limitación del *slew rate* del transceptor físico. El número total de transceptores serie que deben operar a la vez es de 14, por los requisitos del proyecto LINCE.
2. **Diseñar la arquitectura de transporte de datos entre la lógica programable y el sistema de procesamiento.** El transceptor resuelve la capa física de un enlace, pero llevar los flujos de los catorce enlaces instanciados hasta el procesador es un problema distinto, y su solución condiciona el rendimiento del conjunto. Se plantea investigar las alternativas disponibles, desarrollarlas, compararlas y seleccionar la que mejor escale en rendimiento, en ocupación de la lógica programable y en consumo de recursos de CPU.
3. **Desarrollar un driver de software completo en C para RTEMS 7** que abstraiga el acceso al hardware y permita a la aplicación gestionar los transceptores serie, incluyendo la configuración de todos los parámetros exponiendo una API pública sencilla de utilizar.
4. **Diseñar y fabricar una placa de comunicación serie** que permita ejercitar los catorce transceptores de forma simultánea y sobre topologías de bus realistas, como puede ser punto a punto o multipunto compartido.
5. **Diseñar y fabricar dos placas de circuito impreso de expansión de la ZCU102** que permitan la conexión con diversos periféricos, bajo las especificaciones de los subsistemas CDHS y AOCS del satélite, incluyendo interfaces RS422/RS485, SpaceWire, CAN, PWM y medida de señales analógicas.
6. **Integrar y validar el sistema completo**, verificando la comunicación entre la ZCU102 y las tres placas, y comprobando que cada placa cumple con su función de proporcionar una capa física con buses compartidos o de exponer las conexiones a los distintos periféricos sin errores.

1.3. Metodología

Durante el desarrollo del trabajo se ha tenido un contacto estrecho con los compañeros de Sener. Se han seguido **metodologías ágiles** mediante el uso del programa *Jira*, que permite organizar las tareas en función del tiempo que requieren. La metodología ágil utilizada consiste en establecer las tareas que se van a realizar durante un tiempo establecido, denominado como *Sprint*. Al final de cada *Sprint*, se tiene una reunión estructurada en las siguientes fases: revisión de *sprint*, preparación del siguiente *sprint*, análisis de posibles mejoras acerca de la metodología, y preguntas finales. Durante la reunión los participantes se turnan para hablar. En nuestro caso, los *sprints* han tenido una duración de dos

semanas, y se han realizado un total de 20 sprints.

Además de las reuniones de *Sprint*, se ha mantenido una comunicación constante con el equipo de Sener para resolver todo tipo de dudas acerca de los requerimientos del proyecto, y de cómo orientar muchos aspectos.

El trabajo se estructuró en tres fases principales:

Fase 1: Familiarización y establecimiento del entorno (octubre – enero): Los primeros meses se dedicaron a establecer las bases del entorno de desarrollo: instalación y configuración de Vivado y Vitis, compilación de RTEMS 7 desde código fuente para la arquitectura AArch64 mediante el RTEMS Source Builder, y validación del flujo completo (síntesis en Vivado → empaquetado en Vitis → ejecución en RTEMS sobre la ZCU102) con un ejemplo funcional de extremo a extremo. Esta fase fue fundamental para comprender la arquitectura PS-PL del MPSoC y las particularidades del entorno de compilación cruzada para sistemas empotrados.

Fase 2: Desarrollo e integración (febrero – junio): Con el entorno establecido, se abordó el desarrollo principal: el transceptor serie en VHDL, el driver para RTEMS, el diseño de las PCB de comunicación serie, CDHS y AOCS, y las aplicaciones de prueba. Se siguió una metodología iterativa: cada módulo se implementaba, se validaba de forma aislada (mediante simulación en Vivado o prueba directa sobre la placa) y se integraba en el sistema global antes de avanzar al siguiente.

Fase 3: Estudio del transporte y caracterización (junio – julio): Se caracterizó el firmware desarrollado (VHDL y C) y el hardware (PCBs) mediante un conjunto de pruebas, las cuales revelaron que la aproximación inicial del sistema de transporte de datos no era eficiente. Se desarrollaron dos alternativas nuevas: la primera consistió en un sistema de transporte basado en catorce controladores AXI DMA independientes, y la segunda en un sistema de transporte basado en un único controlador AXI MCDMA con un puente VHDL que multiplexa los catorce canales. Ambas alternativas fueron implementadas y comparadas, revelando que la segunda alternativa era la más eficiente y escalable.

En el diseño y fabricación del hardware se asumió desde el principio la detección de errores de diseño como parte del proceso: cada placa se caracterizaba en el laboratorio y, cuando el calendario lo permitía, se corregía y se volvía a fabricar. Quedó fuera del alcance la validación de la comunicación SpaceWire presente en la PCB AOCS, por su alta complejidad técnica.

Durante el desarrollo de los bloques VHDL, se comprobó en todo momento el correcto funcionamiento de los mismos mediante simulaciones de testbench antes de integrarlos en el sistema completo. Además, se realizaron simulaciones del sistema completo, las cuales permitieron encontrar y corregir algunos errores.

Por último, se desarrollaron múltiples automatizaciones en forma de scripts de shell, Python, TCL, etc, que agilizaron el flujo de trabajo permitiendo ahorrar tiempo en tareas repetitivas y eliminar errores manuales.

Todo el trabajo desarrollado (código VHDL, aplicaciones y drivers de RTEMS, bancos de pruebas, diseños de las placas y scripts de automatización) se ha versionado en un único repositorio Git, disponible públicamente en GitHub [3]. Bajo el directorio `tfm/` se organiza por bloques temáticos: `01_ip_serie` (transceptor serie en VHDL), `02_transporte` (las tres arquitecturas de transporte), `03_bench_loopback` y `04_pcb_caracterizacion`

(bancos de prueba y caracterización), `05_aplicaciones_cdhs_aocs` (aplicaciones sobre las placas CDHS y AOCS), `06_firmware` (componentes y scripts de arranque), `07_tools` (herramientas de desarrollo comunes) y `00_docs` (documentación, esquemáticos y BOM). A lo largo de la memoria se hace referencia a rutas concretas dentro de este repositorio, siempre relativas a su raíz.

1.4. Estructura de la memoria

El resto del documento se organiza en cuatro capítulos y tres anexos.

El **capítulo 2** recoge el marco teórico: el proyecto LINCE, el MPSoC Zynq UltraScale+ y la plataforma ZCU102, el entorno de herramientas Vivado y Vitis, el sistema operativo de tiempo real RTEMS, los estándares de comunicación empleados (RS422, RS485, SpaceWire y CAN) y los mecanismos de transporte de datos entre el sistema de procesamiento y la lógica programable, con el bus AXI y el acceso directo a memoria.

El **capítulo 3** describe el desarrollo: la preparación del entorno y del flujo de compilación, el diseño del transceptor serie configurable en VHDL, el driver para RTEMS 7, las tres arquitecturas de transporte implementadas, el diseño y fabricación de las tres placas de circuito impreso y las herramientas de prueba desarrolladas.

El **capítulo 4** presenta los resultados: la validación de la placa de comunicación serie con los catorce transceptores en funcionamiento simultáneo, la comparativa cuantitativa entre las tres arquitecturas de transporte, la caracterización eléctrica de la placa y la validación de las placas CDHS y AOCS sobre cada uno de sus interfaces.

El **capítulo 5** reúne las conclusiones y las líneas futuras de trabajo. Los anexos contienen los mapas de señales y la tabla del oscilador controlado numéricamente, el análisis de aspectos éticos, económicos, sociales y ambientales, y el presupuesto económico del trabajo.

2. Marco teórico

2.1. Proyecto LINCE

El Proyecto LINCE (Línea de industrialización de cargas de pago y plataformas espaciales) es una iniciativa estratégica enmarcada en el PERTE Aeroespacial, cuyo propósito es consolidar la capacidad industrial de España en el segmento de los microsátélites (100–200 kg) destinados a órbitas bajas (LEO). Liderado por Indra, el consorcio busca desarrollar plataformas satelitales modulares, fiables y de altas prestaciones, optimizando costes y plazos mediante procesos de producción en serie. Este ecosistema integra a grandes empresas, PYMES y centros de investigación, como la Universidad Politécnica de Madrid (UPM), para habilitar servicios avanzados de telecomunicaciones, vigilancia y observación, garantizando así la autonomía estratégica de España y Europa en el sector espacial [2].

2.2. MPSoC Zynq UltraScale+ ZCU102

La placa de desarrollo sobre la que se va a trabajar en este proyecto es la Tarjeta de Evaluación de propósito general ZCU102, desarrollada por Xilinx, orientada al prototipado rápido de sistemas empujados de alta complejidad. Esta placa ofrece soporte a todas las interfaces de alta velocidad y lógica programable del MPSoC (*Multiprocessor System-on-Chip*) Zynq UltraScale+, que integra en un único encapsulado de silicio un sistema de procesamiento con cuatro núcleos y una matriz lógica programable, conectados internamente mediante buses de alto rendimiento. Esta arquitectura divide el sistema en dos dominios complementarios, permitiendo separar la carga de trabajo entre el Sistema de Procesamiento (PS, *Processing System*) y la lógica programable (PL, *Programmable Logic*).

El **PS** representa el dominio del software. Su desarrollo es análogo a la programación de cualquier microcontrolador o microprocesador clásico; está basado en una arquitectura física fija (núcleos ARM en este dispositivo) diseñada para ejecutar instrucciones de código en de forma estrictamente secuencial. Este dominio es el encargado de ejecutar sistemas operativos (como RTEMS o Linux), gestionar la memoria y ejecutar las rutinas de control de alto nivel.

La **PL** representa el dominio del hardware a medida. Se trata de una matriz tipo FPGA (*Field Programmable Gate Array*), la cual no ejecuta un programa línea a línea, sino que se reconfigura físicamente. Mediante lenguajes de descripción de hardware (como VHDL), se define la interconexión de miles de recursos internos disponibles en el silicio, tales como tablas de búsqueda (LUTs), biestables, memorias BRAM y bloques de procesamiento digital de señales (DSP); para conformar circuitos digitales reales y específicos. Esta ca-

racterística otorga a la PL un determinismo temporal estricto y la capacidad de procesar señales con un paralelismo masivo inalcanzable para un procesador convencional.

Entre las características destacadas de la ZCU102, una que resulta relevante para este trabajo es la capacidad de expansión externa mediante dos conectores FMC HPC (*High Pin Count*). Estos puertos son el punto de anclaje para tarjetas de expansión o *Mezzanines*, permitiendo extraer pines digitales y pares diferenciales hacia el exterior del sistema. Siguen el estándar VITA 57.1 *FPGA Mezzanine Card (FMC) specification* [4].

2.3. Vivado y Vitis (Xilinx): PS y PL

Los programas que más se van a utilizar durante este trabajo son Vivado y Vitis, ambos desarrollados por Xilinx para el desarrollo sobre sus sistemas hardware.

2.3.1. Vivado Design Suite

Es el entorno oficial de Xilinx para el diseño, síntesis e implementación de circuitos digitales destinados a la PL. Permite programar en Lenguaje de Descripción Hardware (HDL) los circuitos que se desean implementar, ofrece una herramienta de diseño por bloques y permite realizar simulaciones temporales para verificar el comportamiento del hardware desarrollado.

En el contexto específico de este proyecto, Vivado se utiliza para encapsular los transceptores serie diseñados a medida en VHDL y enlazarlos al núcleo del procesador Zynq. Esta integración se realiza mediante el bus de interconexión de alto rendimiento AXI (*Advanced eXtensible Interface*) a través de la herramienta *Block Design*. Asimismo, Vivado gestiona el archivo de restricciones físicas (.xdc), necesario para mapear lógicamente las señales de los periféricos desde el silicio hacia los pines reales de los conectores FMC. El flujo de trabajo en Vivado culmina con la generación del *Bitstream* (el archivo binario que configura físicamente la FPGA) y la exportación de la arquitectura del hardware en un archivo unificado (.xsa).

2.3.2. Vitis Unified Software Platform

Vitis es el entorno de desarrollo integrado (IDE) proporcionado por Xilinx para la programación del dominio del software, es decir, el PS. Funciona en tándem con Vivado: importa el archivo de especificación hardware (.xsa) y genera de forma automatizada el Paquete de Soporte de la Placa o BSP (*Board Support Package*). Este BSP contiene el mapa de memoria estático y los controladores de bajo nivel (*drivers*) indispensables para que el código en C/C++ pueda acceder a los periféricos instanciados previamente en la PL.

Durante el desarrollo de este trabajo, Vitis se emplea para la compilación cruzada orientada a los núcleos ARM del sistema. Se utiliza para generar el firmware crítico de inicialización, concretamente el PMUFW (*Power Management Unit Firmware*) y el cargador de arranque de primera etapa FSBL (*First Stage Boot Loader*). Finalmente, Vitis proporciona las herramientas de empaquetado para fusionar el código ejecutable, el *Bitstream* de la PL y los archivos de arranque en un único fichero binario (BOOT.BIN), habilitando el arranque autónomo de la placa desde una memoria no volátil como una tarjeta SD.

2.4. RTEMS

RTEMS (*Real-Time Executive for Multiprocessor Systems*) es un sistema operativo de tiempo real (RTOS) de código abierto diseñado de forma específica para sistemas empujados de misión crítica. A diferencia de los sistemas operativos de propósito general, como distribuciones estándar de Linux, RTEMS no utiliza memoria virtual y opera en un único espacio de direcciones físico (arquitectura de un solo proceso y múltiples hilos). Esta arquitectura elimina la latencia de los cambios de contexto pesados, minimiza la sobrecarga del procesador y garantiza tiempos de respuesta estrictamente deterministas ante las interrupciones del hardware.

El uso de RTEMS es un estándar de facto en la industria aeroespacial europea y americana (siendo empleado activamente por agencias como la ESA y la NASA en misiones satelitales y sondas espaciales) debido a su robustez, su certificación para entornos críticos y su compatibilidad nativa con arquitecturas ARM como la del Zynq UltraScale+. En el marco del proyecto LINCE y de este TFM, RTEMS se ejecuta sobre el PS actuando como el cerebro temporal del sistema. Su función es orquestar las rutinas de control de alto nivel, gestionar la lectura/escritura de los registros de los transceptores de la PL y asegurar que las transacciones en los buses de comunicación (SpaceWire, RS422, RS485 o CAN) cumplan con los plazos de tiempo (*deadlines*) exigidos durante las pruebas *Hardware-in-the-Loop* de los subsistemas del satélite.

2.5. Comunicaciones serie

En el diseño de sistemas empujados de misión crítica y arquitecturas satelitales, las comunicaciones serie constituyen la columna vertebral para el intercambio de datos entre el Ordenador de a Bordo (OBC) y sus múltiples periféricos, tales como sensores de actitud, actuadores mecánicos y cargas de pago. Frente a las topologías de bus paralelo tradicionales, los enlaces serie proporcionan una reducción drástica de las líneas físicas necesarias, lo que se traduce de forma directa en una disminución crítica del volumen y peso del arnés del satélite. Además, al operar frecuentemente mediante señalización diferencial, mitigan en gran medida la susceptibilidad a las interferencias electromagnéticas (EMI) propias del entorno espacial.

En este apartado se detallan los fundamentos técnicos, topologías y características eléctricas de los protocolos de comunicación serie específicos que han sido seleccionados e integrados en la plataforma de hardware durante el desarrollo de este trabajo: RS485, RS422, SpaceWire y bus CAN.

2.5.1. RS485

El estándar TIA/EIA-485, comúnmente conocido como RS485, define las características eléctricas de receptores y transmisores para su uso en sistemas de comunicaciones serie balanceados (diferenciales). Su principal característica es la capacidad de operar en topologías multipunto (buses compartidos), permitiendo conectar hasta 32 transceptores estándar en el mismo par de cables (o más, si se emplean transceptores de carga fraccional).

La señalización diferencial, donde la información se transmite como la diferencia de potencial entre dos hilos trenzados (A y B), le otorga un alto Rechazo al Modo Común

(CMRR). Esto significa que cualquier ruido electromagnético inducido en el cable afecta por igual a ambas líneas, cancelándose en el receptor. Habitualmente se implementa en modo *Half-Duplex* (2 hilos), donde la transmisión y recepción comparten el medio físico, exigiendo un control estricto del flujo de datos mediante software (gestión de los pines de *Driver Enable*) para evitar colisiones. Es un estándar robusto capaz de alcanzar distancias de hasta 1.200 metros (a velocidades reducidas) o velocidades de hasta 50 Mbps en distancias cortas, requiriendo resistencias de terminación (típicamente de 120Ω) en los extremos del bus para evitar reflexiones destructivas de la señal [5, 6].

2.5.2. RS422

El estándar TIA/EIA-422 (RS422) es el predecesor técnico del RS485 y comparte con él la base física de la señalización diferencial para garantizar la inmunidad al ruido y cubrir largas distancias. Sin embargo, su principal diferencia radica en la topología de la red: el RS422 está diseñado estrictamente para comunicaciones punto a punto o punto a multipunto (conocido como *multi-drop*).

A diferencia del RS485, los transceptores RS422 no están diseñados para ceder el control del bus. En un bus RS422 solo puede existir un único transmisor (maestro) conectado a un máximo de 10 receptores (esclavos) en el mismo par de hilos. Para lograr una comunicación bidireccional continua (*Full-Duplex*), el RS422 requiere obligatoriamente el uso de cuatro hilos (dos pares trenzados: uno dedicado exclusivamente a la transmisión y otro a la recepción). Esta separación física de los canales de ida y vuelta elimina la necesidad de arbitrar el bus por software, reduciendo la sobrecarga de procesamiento y garantizando un determinismo temporal absoluto, lo que lo hace idóneo para el envío continuo de telemetría de sensores críticos [7, 8].

2.5.3. SpaceWire

SpaceWire es un estándar de red de comunicaciones espaciales de alta velocidad, coordinado por la Agencia Espacial Europea (ESA) y ampliamente adoptado a nivel mundial en misiones científicas y comerciales. Está diseñado específicamente para interconectar nodos de alto rendimiento a bordo de satélites, como memorias masivas, procesadores de instrumentación y enlaces de telemetría, ofreciendo un gran ancho de banda y una fiabilidad extrema.

A nivel físico, SpaceWire utiliza señalización diferencial de bajo voltaje (LVDS) e implementa una codificación única denominada *Data-Strobe* (DS). En lugar de transmitir una señal de reloj independiente que sufriría desvíos temporales (*skew*) respecto a los datos a altas frecuencias, la señalización DS envía los datos por un par diferencial y una señal *Strobe* por otro. El reloj se recupera en el receptor realizando una operación lógica XOR entre la señal de datos y el *Strobe*. Esto permite enlaces punto a punto *full-duplex* asíncronos con velocidades que abarcan desde los 2 Mbps hasta más de 400 Mbps. Además, su arquitectura en capas soporta el enrutamiento de paquetes, permitiendo construir topologías de red complejas mediante *routers* SpaceWire [9, 10].

2.5.4. CAN

El bus CAN es un estándar de comunicaciones serie robusto diseñado originalmente para el sector de la automoción, pero cuya alta tolerancia a fallos lo ha convertido en un estándar

de facto (*CAN for Aerospace*) para las redes internas de monitorización, telemetría y telecomandos (TM/TC) en nanosatélites y satélites comerciales.

Es un bus multipunto y multi-maestro que opera bajo el paradigma CSMA/CD+AMP (*Carrier Sense Multiple Access with Collision Detection and Arbitration on Message Priority*). En una red CAN, los nodos no tienen una dirección física; en su lugar, transmiten tramas de datos encabezadas por un identificador que define el contenido y la prioridad del mensaje. Si dos nodos intentan transmitir simultáneamente, la colisión se resuelve a nivel de bit y de forma no destructiva: el mensaje con el identificador de mayor prioridad sobrescribe al de menor prioridad sin corromper el bus, garantizando un determinismo estricto para las alarmas críticas. Además, la capa de enlace del protocolo CAN incorpora mecanismos de detección de errores por hardware (CRC, *bit stuffing*, comprobación de tramas) y confinamiento automático de nodos defectuosos, asegurando que un periférico averiado no bloquee el sistema de control de actitud u otros subsistemas vitales del Ordenador de a Bordo [11, 12].

2.6. Transporte de datos entre el PS y la PL

Implementar los transceptores serie en la lógica programable resuelve la mitad del problema: la serialización de los bits. Queda la otra mitad, que es la que realmente determina el rendimiento del sistema completo: **por dónde y a qué coste viajan los bytes entre la memoria del procesador y esos transceptores**. El MPSoC ofrece varios caminos para ello, y la elección entre ellos no es un detalle de implementación, sino una decisión de arquitectura con consecuencias directas sobre la carga de CPU, la latencia y la ocupación de la FPGA.

Esta sección describe los mecanismos disponibles. Su aplicación concreta se desarrolla en la sección 3.7.

2.6.1. Bus AXI

La comunicación entre el PS y la PL del Zynq UltraScale+ se realiza a través de puertos que implementan el estándar AMBA AXI4 de ARM [13]. El estándar define tres perfiles:

- **AXI4-Lite**. Perfil reducido, orientado a la lectura y escritura de *registros* individuales. Cada transacción mueve una palabra y requiere que el procesador ejecute una instrucción de carga o almacenamiento. Este bus es comúnmente utilizado para configurar periféricos.
- **AXI4 full (*memory-mapped*)**. Perfil completo, con soporte de ráfagas (*bursts*) de hasta 256 transferencias por transacción. Es el bus que utilizan los maestros de la PL para acceder a la DDR del PS a través de los puertos de alto rendimiento (HP), sin pasar por la CPU.
- **AXI4-Stream**. Perfil sin direcciones: un flujo continuo de datos con señales de *handshake* (TVALID/TREADY), marca de fin de paquete (TLAST) y, opcionalmente, un identificador de destino (TDEST). Es el perfil idóneo para conectar un periférico que produce o consume bytes en secuencia, como un UART.

2.6.2. Acceso directo a memoria (DMA)

Un controlador DMA es un maestro de bus que transfiere bloques de datos entre la memoria y un periférico **sin intervención de la CPU**. El procesador se limita a programar la transferencia (dirección de origen o destino y longitud) y a recibir una única interrupción cuando el bloque completo ha terminado. La tasa de interrupción deja de depender del número de bytes y pasa a depender del número de *paquetes*, lo que en la práctica supone una reducción de dos a tres órdenes de magnitud.

En el ecosistema de Xilinx, dos IP cubren este papel.

AXI DMA (PG021)

El IP `axi_dma` [14] traduce entre AXI4-Stream y AXI4 *memory-mapped* en dos canales independientes:

- **MM2S** (*Memory-Mapped to Stream*): lee de la memoria y emite un flujo AXI-Stream hacia el periférico.
- **S2MM** (*Stream to Memory-Mapped*): recibe un flujo del periférico y lo escribe en memoria. El fin de paquete lo marca la señal `TLAST` del *stream*.

Cada canal admite dos modos de funcionamiento. En modo **directo** (*simple DMA*) el procesador escribe la dirección del *buffer* y la longitud en sendos registros, y el motor ejecuta una sola transferencia. En modo **scatter-gather** (SG) el procesador construye en memoria una lista enlazada de *descriptores de buffer* (BD), cada uno con la dirección, la longitud y los bits de control del bloque, y el DMA la recorre autónomamente. El modo SG permite encadenar transferencias sin que la CPU intervenga entre ellas, a costa de que el propio DMA lea y escriba los descriptores en memoria.

La limitación de este IP es que **sirve a un único stream**. Para N periféricos hacen falta N instancias, con el consiguiente coste de área que esto supone, además de la complejidad en software de gestionar N controladores independientes y N interrupciones.

AXI MCDMA (PG288)

El IP `axi_mcdma` [15] es la respuesta a esa limitación: un controlador **multicanal** que multiplexa hasta 16 canales lógicos sobre un único motor y un único puerto hacia la memoria. Cada canal tiene su propio juego de registros, su propio anillo de descriptores y su propio *buffer* en la DDR, pero comparten el hardware de transferencia.

El encaminamiento entre canales lógicos y flujos físicos se realiza gracias a la señal `TDEST` del AXI-Stream. En el sentido S2MM (entrada), el MCDMA lee el `TDEST` que acompaña a cada paquete, lo usa como identificador de canal y escribe la carga útil en el *buffer* apuntado por el descriptor activo de ese canal; el demultiplexado, por tanto, lo resuelve el propio hardware sin intervención del procesador. En el sentido MM2S (salida), la operación es la inversa: el motor etiqueta cada paquete que emite con el `TDEST` del canal que lo originó, de modo que el periférico o el conmutador *stream* situado aguas abajo pueda encaminarlo a su destino. Basta así un único par de interfaces AXI-Stream para dar servicio a los N periféricos, frente a las N instancias independientes que exigiría el `axi_dma`.

3. Desarrollo

3.1. Preparación del entorno de desarrollo

Durante este trabajo se utilizó una máquina virtual con un sistema operativo Linux (Ubuntu/Debian) como entorno de desarrollo principal, por dos motivos prácticos: por un lado, la máquina virtual puede dejarse trabajando de forma indefinida, lo que resulta necesario para las compilaciones prolongadas de la herramienta de síntesis; por otro, se le pueden asignar muchos más recursos de memoria y almacenamiento de los que estarían disponibles en un equipo de trabajo convencional.

Se siguieron los siguientes pasos para dejar operativo el entorno de desarrollo.

3.1.1. Instalación de Vivado y Vitis

Para el desarrollo de la lógica programable (PL) y la generación de imágenes de arranque, se utilizaron las herramientas oficiales de AMD Xilinx: Vivado Design Suite y Vitis Unified Software Platform. Vivado se empleó para el diseño a nivel de hardware y la generación del *bitstream*, mientras que Vitis se utilizó para empaquetar los binarios y generar la imagen de arranque (BOOT.bin). Ambas estaban instaladas previamente en la máquina virtual.

3.1.2. Instalación de RTEMS

En lugar de utilizar binarios precompilados, se optó por compilar el sistema operativo RTEMS (versión 7) desde cero, garantizando el control total sobre la configuración del núcleo. Se siguió para ello la guía *Quick Start* del manual de usuario oficial del proyecto [16], que describe paso a paso todo el proceso. La compilación se realizó utilizando la herramienta RTEMS Source Builder (RSB) [17]. Se generó un *toolchain* cruzado específico para la arquitectura AArch64. Se configuró y compiló el Board Support Package (BSP) correspondiente a la plataforma ZynqMP (*zynqmp_apu*), habilitando explícitamente el soporte para multiprocesamiento simétrico (SMP).

Para validar el entorno recién compilado, se ejecutaron pruebas preliminares utilizando el emulador QEMU (*qemu-system-aarch64*) configurado con la máquina *xlnx-zcu102*. Se verificó la correcta ejecución de dos de los ejemplos que el propio RTEMS incluye en su BSP: *hello.exe*, que confirma que el arranque, la consola serie y la salida estándar funcionan, y *dhrystone.exe*, una implementación del *benchmark* sintético Dhrystone [18], que ejercita la aritmética entera, el manejo de cadenas y las llamadas a función, y que sirve aquí únicamente como comprobación de que el *toolchain* genera código funcional y el planificador ejecuta una carga sostenida.

3.1.3. Establecimiento de un flujo de desarrollo completo

Una vez preparadas todas las herramientas de desarrollo, se realizó un desarrollo completo de principio a fin implementando un ejemplo sencillo: una **puerta AND**.

En este ejemplo se repartieron las tareas entre las dos mitades del MPSoC: la puerta AND se implementó en la **PL**, describiéndola en VHDL, y en el **PS** se escribió un programa que excitaba todas las combinaciones de sus entradas y leía la salida en cada caso, escribiendo los resultados por la consola serie.

La complejidad de este ejercicio no reside en la puerta AND en sí, sino en recorrer correctamente el procedimiento que lleva un programa escrito en C/C++ y un *bitstream* hasta la ZCU102. En resumen, ese procedimiento consiste en generar dos ficheros y copiarlos en una tarjeta SD: el `BOOT.bin`, que contiene el *bitstream* de la PL y todo el software de arranque, y el `rtems.img`, que contiene la aplicación que se ejecutará en el PS. El primero se construye con Vivado y Vitis; el segundo, compilando la aplicación de RTEMS. Los apartados que siguen recorren cada uno de esos pasos.

Parte Vivado. Se creó un proyecto nuevo en Vivado llamado `and_gate_ps_p`, y se diseñó la puerta AND en VHDL:

Programación 3.1: AND_GATE.vhd — ejemplo de puerta AND en VHDL

```
1 library IEEE;
2 use IEEE.STD_LOGIC_1164.ALL;
3
4 entity AND_GATE is
5     Port ( A_B_IN : in  STD_LOGIC_VECTOR (1 downto 0);
6           Z_OUT   : out STD_LOGIC);
7 end AND_GATE;
8
9 architecture Behavioral of AND_GATE is
10 begin
11     Z_OUT <= A_B_IN(1) and A_B_IN(0);
12 end Behavioral;
```

Después se creó un *Block Design*, se añadió el bloque Zynq UltraScale+ MPSoC IP y se configuró automáticamente usando *Block Automation* de Vivado. Se añadió el fichero VHDL del paso anterior al *Block Design*. Se añadieron dos AXI GPIO IPs al *block design* para comunicar el PS con la PL, y se conectaron los bloques entre sí, utilizando *Automate Connection* de Vivado para todas las conexiones faltantes.

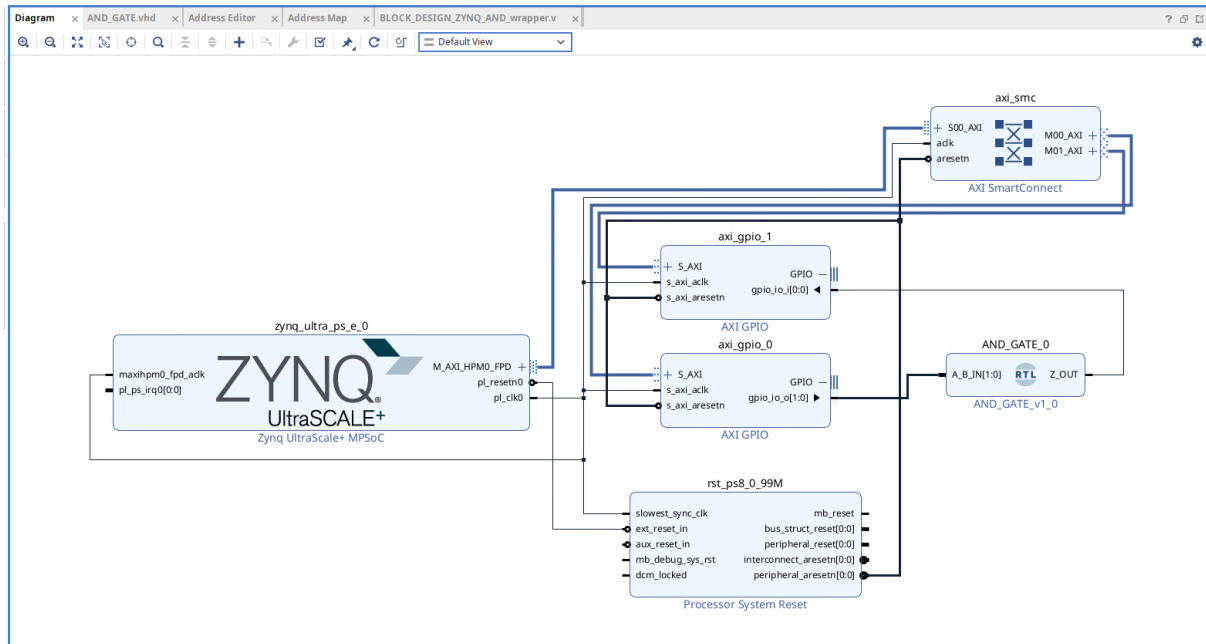


Figura 3.1: *Block Design* del ejemplo de la puerta AND: bloque Zynq UltraScale+ MPSoC, los dos AXI GPIO y el módulo VHDL AND_GATE.

Después se creó un *Wrapper* del *Block Design* para que Vivado pudiera sintetizarlo correctamente. En la pestaña *Address Editor* pueden verse las direcciones de los registros para acceder a la parte PL desde el PS. Por último, se generó el *BitStream* y se exportó el hardware como .xsa.

Generación de la imagen de arranque (Vitis). El objetivo de este paso es empaquetar el hardware generado en Vivado, junto con el software de arranque, en un único binario (BOOT.bin) que el MPSoC pueda ejecutar desde la tarjeta SD. El proceso fue el siguiente:

1. Se creó un componente de tipo *Platform* en Vitis, utilizando como base el .xsa exportado desde Vivado, y se compiló.
2. Sobre esa plataforma se creó una aplicación a partir del ejemplo *Zynq MP FSBL*, y se compiló para obtener el `zynqmp_fsbl_project.elf`.
3. Se lanzó el generador de imágenes de Vitis (*Create Zynq UltraScale+ Boot Image*) y se añadieron los componentes en el orden indicado en la figura 3.2, ya que el orden dentro del fichero BIF determina la secuencia de arranque.

Los componentes que forman la imagen, en orden, son:

- `zynqmp_fsbl_project.elf`, en modo *bootloader*, destino a53-0 y nivel de excepción EL3. Se encuentra en la carpeta `build` de la aplicación anterior.
- `pmufw.elf`, en modo *pmufw_image* y destino a53-0. Es el firmware de la unidad de gestión de potencia y se descargó del repositorio de firmware precompilado de Xilinx para la ZCU102¹.
- `BLOCK_DESIGN_ZYNQ_AND_wrapper.bit`, en modo *datafile* y con la PL como dispositivo de destino. Es el *bitstream* generado por Vivado.

¹https://github.com/Xilinx/soc-prebuilt-firmware/tree/xilinx_v2025.1/zcu102-zynqmp

- `bl31.elf`, en modo *datafile*, destino `a53-0` y nivel de excepción EL3. Es el *firmware* de ARM Trusted Firmware y se descarga del mismo sitio que el PMUFW.
- `u-boot.elf`, en modo *datafile*, destino `a53-0` y nivel de excepción EL2. U-Boot es un gestor de arranque de segunda etapa: se ejecuta después del FSBL, inicializa los periféricos necesarios (tarjeta SD, red, consola serie) y es el encargado de cargar en memoria la imagen del sistema operativo (en este caso `rtems.img`) y cederle el control. Ofrece además una consola interactiva desde la que se puede inspeccionar la memoria o elegir manualmente qué imagen arrancar, lo que resultó muy útil durante la depuración.
- `system.dtb`, en modo *datafile*, destino `a53-0` y dirección de carga `0x100000`. Este *device tree* es el que permite a BL31 saber dónde está enlazado el BL33 (U-Boot) y arrancarlo.

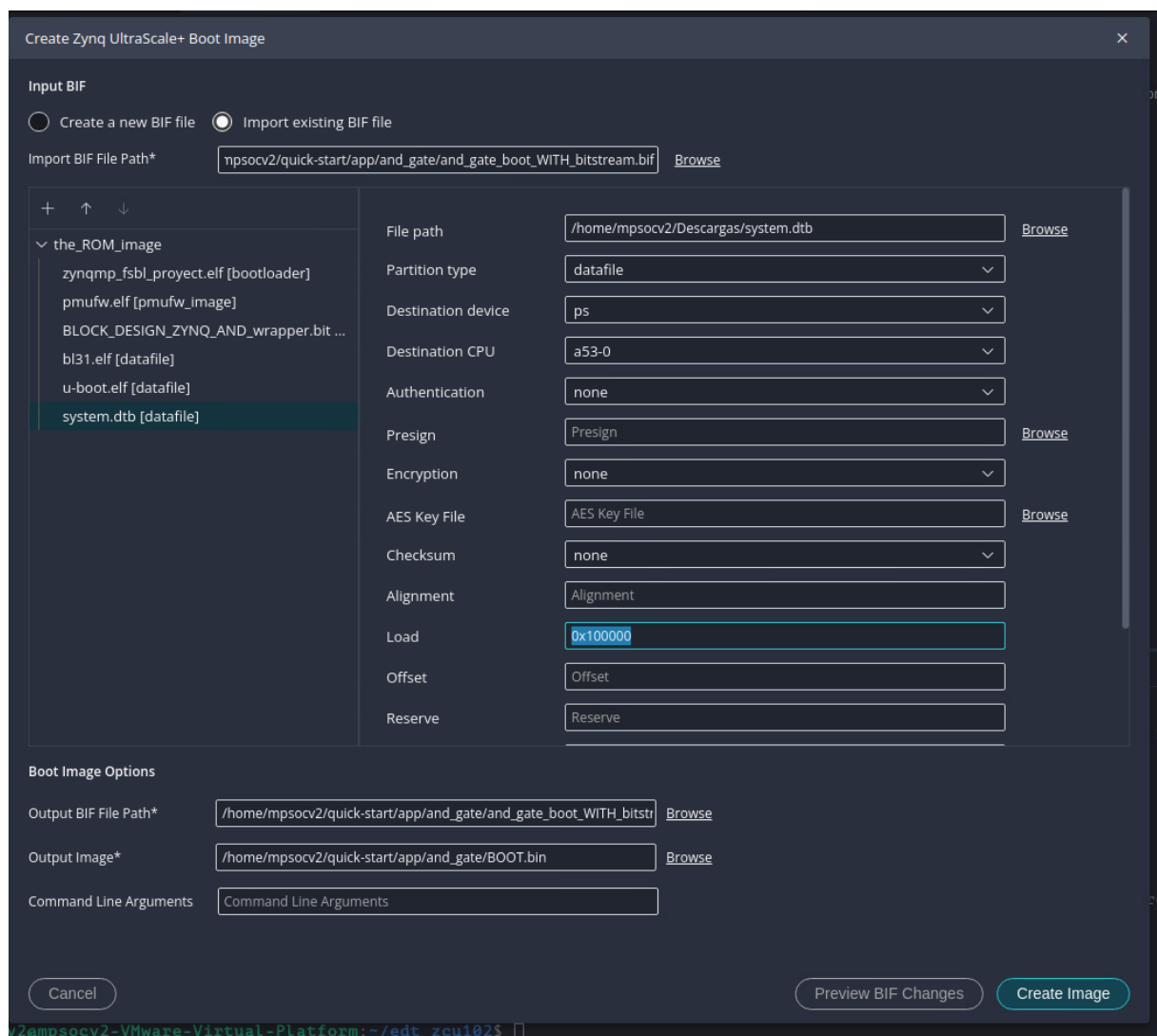


Figura 3.2: Generador de imágenes de arranque de Vitis, con los componentes que conforman el `BOOT.bin`.

Parte RTEMS. Una vez generado el archivo de arranque, se desarrolló un software de ejemplo que interactuaba con los puertos GPIO conectados por AXI, excitando todas las combinaciones posibles de la puerta AND en la PL y leyendo su salida. Fue necesario

añadir un archivo de mapeado de memoria para que el sistema operativo reconociera las direcciones correspondientes a la PL, y un `wscript` para la compilación. Ambos fuentes están disponibles en `tfm/00_docs/AND_example/` dentro del repositorio del proyecto [3].

El primero de ellos, `and_demo.c`, contiene la tarea `Init` de RTEMS: escribe la pareja de entradas (A,B) en el registro de datos del `axi_gpio_0` (mapeado en `0xA0000000`), espera un instante a que la lógica de la PL se establezca y lee la salida Z del `axi_gpio_1` (`0xA0010000`), imprimiendo cada resultado por la consola serie.

El segundo, `mmu_pl_map.c`, es el archivo de mapeado mencionado: declara como memoria de periférico el rango `0xA0000000–0xA01FFFFFF`, donde Vivado ha situado los AXI GPIO. Este mapeado se realiza nada más arrancar `Init`, ya que RTEMS desconoce a priori las direcciones de la PL y, sin él, el primer acceso a los GPIO aborta la ejecución.

Carga en la tarjeta SD y arranque de la placa. Los dos ficheros se copian en la partición FAT de una tarjeta SD, que se inserta en el zócalo J100 de la placa [1]. El `BOOT.bin` es el que el MPSoC ejecuta nada más alimentarse: inicializa el PS, carga el *bitstream* en la PL y encadena el resto de etapas hasta U-Boot, que a su vez carga el `rtems.img` en memoria y le cede el control. Si el ensayo solo ejercita la PL, el `BOOT.bin` basta por sí solo, ya que lleva dentro el *bitstream*.

Antes de encender la placa hay que colocar el selector de modo de arranque en la posición de tarjeta SD. Es el conmutador DIP **SW6**, la referencia 44 del plano de componentes de la ZCU102 [1], cuyas posiciones recoge la tabla 3.1. Conviene tenerlo presente porque la configuración de fábrica es **QSPI32**: sin cambiar el conmutador, la placa no arranca desde la tarjeta aunque los ficheros estén correctamente copiados.

Modo de arranque	PS_MODE[3:0]	SW6 [4:1]
JTAG	0000	ON, ON, ON, ON
QSPI32 (por defecto)	0010	ON, ON, OFF, ON
Tarjeta SD	1110	OFF, OFF, OFF, ON

Tabla 3.1: Posiciones del selector de modo de arranque SW6 de la ZCU102. Adaptada de [1].

Con la tarjeta insertada y el conmutador en su sitio, el barrido completo de la tabla de verdad se obtiene por la consola serie y confirma el comportamiento esperado de la puerta AND:

Programación 3.2: Salida de terminal — barrido de la puerta AND

```

1 ## Transferring control to RTEMS (at address 000100000) ...
2 Barrido AND via PL
3 A=0, B=0, Z=0 (raw=0x00000000)
4 A=0, B=1, Z=0 (raw=0x00000000)
5 A=1, B=0, Z=0 (raw=0x00000000)
6 A=1, B=1, Z=1 (raw=0x00000001)
7
8 [ RTEMS shutdown ]

```

La primera línea la emite U-Boot al ceder el control a la imagen de RTEMS; las cuatro siguientes son las escritas por `and_demo.c`, con la salida Z a uno únicamente cuando ambas entradas valen uno. Esta traza fue la primera confirmación de que el flujo completo (VHDL sintetizado en la PL, imagen de arranque, aplicación de RTEMS y comunicación PS $\leftrightarrow$ PL por AXI) funcionaba de extremo a extremo. La salida se conserva en `tfm/00_docs/logs_terminal/and_gate_output.txt`.

Para facilitar el desarrollo rápido en la parte de RTEMS se desarrollaron dos herramientas, disponibles en `tfm/07_tools/` dentro del repositorio del proyecto [3]. La primera, `make_img.sh`, automatiza la compilación: deduce el nombre de la aplicación del directorio desde el que se lanza, ejecuta `waf`² (configurándolo la primera vez), convierte el ejecutable a binario plano con `objcopy`³, lo comprime y lo empaqueta con `mkimage`⁴ en el formato de imagen que espera U-Boot, generando `rtems.img`.

La segunda, `automate_ymodem_update.py`, evita tener que quitar y poner la tarjeta SD en cada prueba. Escrito en Python sobre `pyserial`⁵, invoca al script anterior para regenerar la imagen y, a través de la conexión USB-serie con la placa, interrumpe el *autoboot*⁶ de U-Boot, transfiere `rtems.img` por YMODEM⁷ con el comando `loady` y la escribe en la partición FAT⁸ de la SD mediante `fatwrite`; ambos son comandos de la consola de U-Boot. Opcionalmente puede enviar también el `BOOT.bin`, pero en la práctica no se utilizó porque este archivo resultaba ser demasiado grande y la conexión por USB-serial muy lenta. De esta manera, cada vez que se quería probar un cambio en la aplicación de RTEMS bastaba con ejecutar este único script y reiniciar la ZCU pulsando su botón de ResetCPU, que reinicia el sistema de procesamiento sin cortar la alimentación de la placa.

Conviene señalar que poner en marcha todo este flujo costó bastante tiempo. Cada intento implicaba sintetizar el diseño en Vivado, crear y compilar la plataforma en Vitis, crear y compilar la aplicación, generar la imagen de arranque, compilar la imagen de RTEMS, copiar todo a la tarjeta SD y arrancar la placa. Una iteración completa llevaba bastantes minutos y, cuando la placa no arrancaba, no había ninguna indicación clara del motivo, así que había que volver atrás y probar otra combinación de componentes o de parámetros. Solo después de varios intentos se dio con la configuración que funciona.

Una vez conseguido el primer arranque correcto apareció un segundo problema: al modificar el diseño VHDL y repetir el proceso, el sistema seguía comportándose como antes del cambio. La causa era que Vitis reutiliza artefactos ya compilados de la plataforma y de la aplicación, de modo que un `.xsa` nuevo no siempre se propagaba a la imagen final. Limpiar y recompilar no resultó fiable, así que se optó por la solución directa: crear una plataforma

²`waf` es el sistema de compilación que emplean las aplicaciones de RTEMS; el fichero `wscript` de cada aplicación describe sus fuentes y opciones de compilación.

³Utilidad del *toolchain* cruzado que extrae del ejecutable compilado una copia del contenido tal cual debe quedar en memoria, sin la información auxiliar que el ejecutable lleva para el enlazado y la depuración.

⁴Herramienta de U-Boot que añade al binario una cabecera con los datos que el gestor de arranque necesita para cargarlo y ejecutarlo, entre ellos la dirección de memoria en la que debe copiarlo.

⁵Biblioteca de Python para manejar puertos serie, empleada aquí para dialogar con la consola de U-Boot.

⁶Arranque automático que U-Boot lanza tras un breve tiempo de espera; pulsando una tecla durante esa ventana se accede a su consola de comandos.

⁷Protocolo para enviar ficheros por un enlace serie: los trocea, los numera y comprueba que cada trozo llega sin errores.

⁸Partición de la tarjeta SD, con el mismo formato que usa un USB corriente, de la que la ZCU102 lee los ficheros de arranque.

y una aplicación nuevas para cada cambio en la PL. Es más lento, pero garantiza que la imagen generada corresponde realmente al hardware que se acaba de sintetizar.

Automatización de la generación de la imagen. Repetir a mano toda esta secuencia en cada iteración del diseño (exportar el XSA, crear y compilar la plataforma en Vitis, compilar el FSBL y ensamblar el BOOT.BIN) resultaba tedioso y, sobre todo, propenso al error humano: basta olvidar un componente, invertir su orden dentro del BIF o arrastrar un artefacto antiguo para que la placa no arranque sin dar ninguna pista del motivo. A eso se sumaba el tiempo puramente logístico de ir abriendo cada aplicación gráfica y navegar por sus asistentes. Para eliminar ambos problemas se desarrolló el script `generate_boot.sh` (disponible en `tfm/06_firmware/boot_scripts/` del repositorio del proyecto [3]), un *wizard* interactivo en Bash que encadena todos estos pasos de forma desatendida. El script ofrece tres puntos de entrada según el estado en que se encuentre el trabajo: partir del proyecto de Vivado y generar *bitstream* desde cero, partir de un proyecto con *bitstream* ya generado y solo exportar el XSA, o partir directamente de un XSA ya exportado. A partir de ahí invoca Vitis en modo script mediante su API Python para crear la plataforma y compilar el FSBL, y llama a `bootgen` para ensamblar el BOOT.BIN final con los componentes de arranque precompilados (PMUFW, BL31, U-Boot, DTB). Esto redujo el tiempo de iteración hardware → imagen de varios minutos de clics a una sola ejecución de terminal, y garantiza que la imagen generada corresponde siempre al hardware que se acaba de sintetizar.

Una vez verificado que esta metodología de desarrollo funcionaba correctamente, se estableció como el proceso estándar a seguir para los próximos desarrollos.

3.2. Diseño del transceptor serie configurable (PL)

El transceptor serie se diseñó para ser altamente configurable y adaptable a cualquier dispositivo, debido a que se desconocían los dispositivos finales que iban a estar conectados al OBC.

El transceptor completo consta de varios bloques conectados entre sí. Los bloques fundamentales son:

- **Oscilador controlado numéricamente** (`NCO.vhd`): proporciona la señal de temporización de bit para las 54 velocidades soportadas, de modo que transmisor y receptor puedan trabajar a cualquiera de ellas.
- **Transmisor** (`TX_CONFIGURABLE_SERIAL.vhd`): serializa y emite las tramas, y gobierna la señal de *Driver Enable* del transceptor físico.
- **Receptor** (`RX_CONFIGURABLE_SERIAL.vhd`): detecta el inicio de trama, muestrea la línea, comprueba paridad y bit de parada y valida la palabra recibida.
- **Registro de desplazamiento** (`ShiftRegister.vhd`): convierte a paralelo los bits recibidos. Incorpora la lógica necesaria para adaptarse a todas las combinaciones de número y orden de los bits.
- **Memoria FIFO**: almacena las palabras recibidas hasta que el PS las lee.
- **Bloque de canal** (`CONFIGURABLE_SERIAL.vhd`): integra los bloques anteriores y añade la lógica de acoplamiento entre ellos.

- **Bloque *top*** (`CONFIGURABLE_SERIAL_TOP.vhd`): empaqueta el canal completo en los dos vectores que lo conectan al PS.

Los parámetros configurables son:

- Baudrate (desde 50 hasta 4.000.000 baudios).
- Número de *stop bits* (1, 1,5 o 2).
- Paridad (par, impar, marca, espacio o deshabilitada).
- Orden de los bits (LSB o MSB).
- Número de bits de datos (entre 5 y 9).

3.2.1. Oscilador controlado numéricamente (NCO)

El transmisor y el receptor comparten una misma base de tiempos, que es el bloque que se describe en primer lugar porque su salida —el *tick*— es la referencia con la que ambos miden el tiempo de bit.

El transceptor debía poder variar su velocidad en tiempo de ejecución sobre un rango muy amplio de baudrates, y un divisor entero del reloj de la FPGA no puede generar con exactitud la mayoría de esas frecuencias. La solución fue un **oscilador controlado numéricamente** (NCO), un tipo de circuito que corrige periódicamente el desfase acumulado por esa imprecisión, y que se diseñó a medida para las necesidades del proyecto: genera la señal de *enable* al baudrate seleccionado de entre 54 valores de velocidad almacenados en una tabla de verdad.

El bloque expone una interfaz mínima: además del reloj, una entrada de reset asíncrono activo a nivel bajo (*rst*), una habilitación (*en*), el selector de velocidad (*baud_sel*, 6 bits), un selector de modo (*half_mode*) y la salida de temporización (*tick*). El número de bits del acumulador es un parámetro *generic*, fijado a 32.

El circuito consiste en un sumador acumulador: para cada frecuencia seleccionada, se tiene calculado un incremento concreto que depende del número de bits del sumador. Cuantos más bits tenga este sumador, mejor precisión se consigue. En esta implementación se han utilizado 32 bits. El incremento se suma y acumula en cada período de reloj; cuando el sumador se desborda, se utiliza el bit de *carry_out* como *tick* del NCO. Este *tick* se genera a la frecuencia deseada, y el mecanismo de acumulación garantiza que el error promedio sea inferior al de un divisor entero simple.

Los incrementos no se almacenan como literales: los calcula en tiempo de síntesis una función que evalúa $inc = \lfloor 2^{32} \cdot f_{baud} / f_{clk} \rfloor$ para cada entrada de la tabla, de modo que basta cambiar la constante de frecuencia de reloj para reajustar las 54 velocidades. La tabla tiene dos columnas por entrada: el incremento nominal y el de doble frecuencia, que selecciona *half_mode*. Este segundo valor es el que permite medir semiperíodos de bit, algo que necesitan tanto el transmisor, en el campo de parada, como el receptor, en el bit de inicio. Tabularlo en lugar de duplicar el primero evita arrastrar multiplicado por dos el error de truncamiento del incremento nominal.

Las dos entradas de control restantes son las que permiten a las máquinas de estados del transmisor y del receptor anclar la fase del temporizador: *rst* pone a cero acumulador y salida, con lo que el *tick* arranca alineado con el inicio de la trama, y *en* congela el

acumulador en los estados en los que no se necesita temporización, evitando que la fase derive entre tramas. La salida `tick` está registrada, por lo que aparece un ciclo de reloj después del desbordamiento.

Para validar su funcionamiento se desarrolló un *testbench* dedicado que recorre las velocidades soportadas y mide la frecuencia real del *tick* generado. De esa campaña procede la tabla del Anexo 1, y su lectura valida el diseño: el peor error de todo el rango es de 120 ppm (a 3,6864 Mbaudios), dos órdenes de magnitud por debajo de la tolerancia habitual del $\pm 2\%$ de un receptor UART, y en la mayoría de las velocidades estándar el error es nulo o inferior a 1 ppm. El NCO queda así acreditado como fuente de temporización fiable para todo el rango de operación del transceptor.

[Figura pendiente: Diagrama del circuito NCO de 32 bits]

Figura 3.3: Circuito del oscilador controlado numéricamente (NCO).

3.2.2. Transmisor

El transmisor recibe una palabra en paralelo y la emite por la línea serie con el formato de trama que fijan sus entradas de configuración. Sus puertos se recogen en la tabla 3.2.

Puerto	Dir.	Función
Clk	in	Reloj del sistema, 100 MHz, activo por flanco de subida.
Reset	in	Reset asíncrono del sistema, activo a nivel bajo.
Start	in	Pulso de petición de transmisión.
Data	in	Palabra a transmitir (9 bits, alineada en el LSB).
baud_sel	in	Índice de velocidad dentro de la tabla del NCO (6 bits).
stop_bit	in	Duración del campo de parada, en semiperiodos de bit (010 = 1, 011 = 1,5, 100 = 2).
parity	in	Modo de paridad: par, impar, <i>mark</i> , <i>space</i> o deshabilitada.
bit_order	in	Orden de emisión: 0 = LSB <i>first</i> , 1 = MSB <i>first</i> .
data_bits	in	Número de bits de datos (000–100 $\rightarrow$ 5–9 bits).
EOT	out	Transmisor libre: a nivel alto únicamente en reposo.
DE	out	<i>Driver Enable</i> del transceptor físico.
TX	out	Línea de transmisión serie.

Tabla 3.2: Puertos del transmisor TX_CONFIGURABLE_SERIAL.

Dispone además de un parámetro *generic*, DELAY_CYCLES, cuyo uso se detalla más adelante.

Máquina de estados

La temporización del tiempo de bit y del semiperiodo de bit la proporciona el NCO, instanciado dentro del propio transmisor: la FSM lo habilita, lo pone a cero y conmuta su entrada `half_mode` según el estado, y consume su salida `tick` como base de tiempos. El comportamiento se organiza en siete estados:

- **Idle**: estado de reposo. Mantiene la línea en alto, EOT a uno y el NCO detenido. Al llegar **Start** activa DE, pone a cero el contador de retardo y pasa a **PreDE**.
- **PreDE**: mantiene DE activo y la línea en reposo durante `DELAY_CYCLES` ciclos de reloj, para dar tiempo al transceptor físico a habilitar su etapa de salida antes de que empiece la trama. Al cumplirse la cuenta se borra el contador de bits y se resetea el acumulador del NCO —de modo que la fase del temporizador arranque desde cero justo en el bit de inicio, sin arrastrar el desfase acumulado de la trama anterior— y se pasa a **StartBit**.
- **StartBit**: fuerza la línea a nivel bajo durante un tiempo de bit completo. Con el primer *tick* del NCO pasa a **SendData**.
- **SendData**: saca a la línea el bit seleccionado por el multiplexor de serialización y avanza el contador de bits con cada *tick*. Cuando el contador alcanza el número de bits configurado, pasa a **ParityBit** presentando ya el bit de paridad o, si la paridad está deshabilitada (`parity = 100`), directamente a **StopBit** con la línea en alto.
- **ParityBit**: mantiene el bit de paridad durante un tiempo de bit y pasa a **StopBit**.
- **StopBit**: mantiene la línea en alto y conmuta el NCO a `half_mode`, que lo hace oscilar al doble de la velocidad seleccionada. Contando semiperiodos —dos, tres o cuatro según `stop_bit`— se obtienen los tres campos de parada admitidos, incluido el de 1,5 bits, que no es expresable con una cuenta de bits enteros. Alcanzada la cuenta pasa a **PostDE**.
- **PostDE**: simétrico de **PreDE**. Mantiene DE activo y la línea en reposo otros tantos ciclos antes de volver a **Idle**, donde DE se libera.

El siguiente diagrama FSM resume el comportamiento del transmisor. Debido a su tamaño, se ha dividido en tres partes para su legibilidad.

[Figura pendiente: Diagrama FSM del transmisor (tres partes)]

Figura 3.4: Diagrama FSM del transmisor serie.

Lógica de apoyo a la máquina de estados

La FSM se limita a secuenciar la trama y a generar señales de control; el trabajo de seleccionar qué bit sale a la línea en cada momento recae en un camino de datos puramente combinacional que la acompaña, y que evita tener que desplazar o rotar la palabra de entrada.

Banco de contadores. Tres contadores síncronos con habilitación y borrado independientes, gobernados por la FSM: el de bits de datos (4 bits), el de semiperiodos del campo

de parada (3 bits) y el de retardo de DE (14 bits, suficiente para el valor por defecto de DELAY_CYCLES). Todos comparten el mismo proceso síncrono y el mismo reset asíncrono.

Decodificador de anchura de palabra. Traduce el código de `data_bits` (000–100) al número real de bits (5–9), que es el valor terminal con el que se compara el contador de datos.

Multiplexor de serialización. Selecciona el bit a emitir a partir de la cuenta actual, en tres etapas. La rama LSB *first* es directa: el índice del contador es el índice del bit. La rama MSB *first* requiere cinco multiplexores distintos, uno por anchura de palabra, porque el primer bit emitido depende del tamaño de la trama: es el bit 4 en una palabra de 5 bits y el bit 8 en una de 9. Una segunda etapa elige entre esas cinco ramas según `data_bits`, y una tercera entre LSB y MSB *first* según `bit_order`. El resultado es que la palabra permanece inmóvil en su registro de entrada durante toda la trama.

Generador de paridad. Una red de puertas XOR calcula la paridad de los bits activos —una expresión por cada anchura de palabra— y un multiplexor final aplica el modo configurado: paridad par, su complemento para la impar, o las constantes uno y cero para los modos *mark* y *space*.

Reset del NCO. La entrada de reset del NCO se combina con el reset global del sistema, de modo que el temporizador se inicializa tanto cuando lo pide la FSM —antes de cada trama— como cuando se resetea el transceptor completo.

Retardo de establecimiento del *Driver Enable*

Los estados PreDE y PostDE se incorporaron después de la primera versión del transmisor, al aparecer caracteres corruptos de forma ocasional en las primeras recepciones sobre la placa CDHS. La hipótesis de partida era que DE conmutaba con demasiada rapidez en el paso de transmisión a recepción y no dejaba establecerse al transceptor físico. Los dos estados sitúan un margen configurable a ambos lados de la trama: DE se activa DELAY_CYCLES ciclos antes del bit de inicio y se libera otros tantos después del último bit de parada. El valor por defecto es de 10 000 ciclos, esto es, 100 μ s con el reloj de 100 MHz; en simulación se reduce a una decena de ciclos mediante el *generic* para no alargar los *testbench*, sin que ello afecte al valor de síntesis.

La medida no eliminó los caracteres corruptos —la causa estaba en el receptor, y se aborda en el apartado siguiente—, pero los dos estados se conservaron: el margen es coherente con los tiempos de habilitación que especifican los transceptores empleados y su coste, 200 μ s por trama en el peor caso, es despreciable frente al tiempo de trama salvo a las velocidades más altas.

3.2.3. Receptor

El receptor vigila la línea serie, recupera la temporización de bit a partir del flanco de inicio y entrega los bits muestreados al registro de desplazamiento, señalando al final si la trama es válida. Sus puertos se recogen en la tabla 3.3; las cinco entradas de configuración son las mismas del transmisor y con idéntica codificación, de modo que ambos comparten la palabra de configuración del canal.

Puerto	Dir.	Función
Clk, Reset	in	Reloj de 100 MHz y reset asíncrono activo a nivel bajo.
baud_sel, stop_bit, parity, bit_order, data_bits	in	Configuración del formato de trama, idéntica a la del transmisor.
LineRD_in	in	Línea de recepción, ya registrada por el bloque de canal.
shiftRegister_Q	in	Palabra normalizada que va acumulando el registro de desplazamiento, sobre la que se calcula la paridad esperada.
ERROR_OK	in	Borrado de las banderas de error.
PAR_ERROR	out	Error de paridad.
FRAME_ERROR	out	Error de trama (bit de parada ausente).
Valid_out	out	Pulso de bit válido; habilita el registro de desplazamiento.
Code_out	out	Valor muestreado de la línea, que entra al registro de desplazamiento.
Store_out	out	Pulso de escritura de la palabra completa en la FIFO.

Tabla 3.3: Puertos del receptor `RX_CONFIGURABLE_SERIAL`.

Máquina de estados

El receptor no dispone de un reloj común con el emisor, de modo que la referencia temporal ha de extraerla de la propia línea. Su FSM consta de cinco estados:

- **Idle**: NCO detenido y línea en observación. Un nivel bajo en la línea se interpreta como posible bit de inicio: se resetea el acumulador del NCO —con lo que la fase del temporizador queda anclada al flanco de bajada— se borra el contador de bits y se pasa a **StartBit**.
- **StartBit**: el NCO trabaja en **half_mode**, de forma que su primer *tick* llega a mitad del bit de inicio. En ese instante se vuelve a muestrear la línea: si sigue en bajo, el bit de inicio es legítimo y se pasa a **RcvData**; si ha vuelto a alto, se descarta y se regresa a **Idle**.
- **RcvData**: cada *tick* —ya a la velocidad nominal— genera un pulso de **Valid_out**, que hace que el registro de desplazamiento capture el valor presente en la línea, e incrementa el contador de bits. Como la fase arrancó a mitad del bit de inicio, todos los *ticks* posteriores caen en el centro de su bit, que es el punto de máxima inmunidad frente al error de velocidad entre los dos extremos del enlace. Alcanzado el número de bits configurado, se pasa a **ParityBit** o, si la paridad está deshabilitada, directamente a **StopBit**.
- **ParityBit**: compara el valor muestreado con la paridad esperada. Si coinciden pasa a **StopBit**; si no, activa **PAR_ERROR** y vuelve a **Idle** descartando la trama, sin

escribirla en la FIFO.

- **StopBit**: muestrea el centro del primer bit de parada. Si la línea está en alto, emite **Store_out** y la palabra se escribe en la FIFO; si está en bajo, la trama está desalineada y se activa **FRAME_ERROR**. En ambos casos se regresa a **Idle**.

Solo se comprueba el primer bit de parada: la entrada **stop_bit** no interviene en la FSM del receptor. El resto del campo de parada transcurre con la FSM ya en **Idle**, donde la línea en reposo está en alto y no se dispara ninguna detección; el receptor queda así preparado para el siguiente flanco de inicio sin necesidad de contar la duración exacta del campo.

[Figura pendiente: Diagrama FSM del receptor serie]

Figura 3.5: Diagrama FSM del receptor serie.

Lógica de apoyo a la máquina de estados

Contador de bits y decodificador de anchura. Idénticos a los del transmisor: el contador avanza con cada *tick* en **RcvData** y se compara con la anchura de palabra decodificada a partir de **data_bits**.

Cálculo de la paridad esperada. La red de puertas XOR opera sobre **shiftRegister_Q**, es decir, sobre la palabra que el registro de desplazamiento ya ha normalizado. Gracias a esa normalización el cálculo es el mismo en LSB y en MSB *first*, y reutiliza sin cambios el esquema del transmisor: paridad par, su complemento para la impar y las constantes de los modos *mark* y *space*.

Registros de error. **PAR_ERROR** y **FRAME_ERROR** son biestables que retienen el error hasta que el PS lo reconoce con **ERROR_OK**. El borrado es síncrono, dentro de la rama de reloj del proceso: en una versión anterior estaba escrito como una rama alternativa *previa* al flanco de reloj, lo que impedía ejecutar la rama síncrona mientras **ERROR_OK** estuviese activo. Como el PS mantiene ese bit permanentemente a uno, el efecto era que el contador de bits nunca avanzaba y el receptor no llegaba a completar ninguna palabra. Conviene tener presente que, con **ERROR_OK** fijo a uno, las banderas se borran en cada ciclo de reloj; para poder leerlas, el software debe desactivar ese bit.

Verificación del bit de inicio a mitad de período

El muestreo a mitad del bit de inicio descrito en **StartBit** cumple una segunda función, además de fijar la fase de muestreo: filtra los falsos inicios de trama. Pulsos de ruido breves en la línea generan flancos descendentes que un receptor sin esta comprobación aceptaría como bits de inicio válidos, capturando a continuación una trama de basura y entregando un carácter corrupto al *buffer*. Con la verificación, un pulso más corto que medio bit deja la línea de nuevo en alto antes del *tick* y la FSM regresa a **Idle** sin capturar nada; solo se prosigue con la recepción si la línea sigue en bajo al llegar a la mitad del bit.

Esta técnica es una práctica estándar de robustez en receptores UART, y es la que resolvió los caracteres corruptos observados en las pruebas sobre la placa CDHS: con ella no volvió a aparecer ninguno en las pruebas posteriores.

3.2.4. Registro de desplazamiento

El registro de desplazamiento convierte a paralelo los bits que va entregando el receptor. Sus entradas son el bit muestreado (**D**), un pulso de habilitación por bit recibido (**Enable**, conectado al **Valid_out** del receptor) y las dos señales de configuración que determinan el formato de la palabra (**data_bits** y **bit_order**); su salida **Q** es la palabra de 9 bits. El registro solo evoluciona en los ciclos en que **Enable** está activo, de modo que la FSM del receptor controla exactamente qué muestras entran en la palabra.

Está diseñado para poder almacenar datos tanto si la configuración es *LSB-first* como *MSB-first*, de modo que una vez recibido el mensaje la palabra en la salida siempre es la correcta independientemente del modo configurado. Para ello dispone de dos caminos de desplazamiento combinatoriales, seleccionados por **bit_order**: en *LSB first* el bit entrante se inserta en la posición más significativa de la palabra activa y el contenido se desplaza hacia la derecha; en *MSB first* se inserta en la posición cero y el contenido se desplaza hacia la izquierda. Cada camino tiene a su vez cinco variantes, una por anchura de palabra, que fijan dónde se inserta el bit y rellenan a cero las posiciones no utilizadas.

El efecto de esta **normalización** es que, completada la trama, el primer bit de datos recibido ocupa siempre **Q(0)** y la palabra está alineada al LSB con independencia de la configuración. Los dos consumidores de la salida —la FIFO y el cálculo de paridad esperada del receptor— pueden así ignorar por completo el orden de bits configurado, y el software del PS recibe la palabra en el mismo formato en los cinco anchos y en los dos órdenes posibles.

3.2.5. FIFO

La FIFO es un bloque IP con 9 bits de datos (para poder almacenar mensajes de 9 bits) y 512 de profundidad, que permite almacenar temporalmente varios mensajes y habilitarlos para su lectura posterior por el PS. Está configurada en modo *first word fall through*, de modo que la palabra más antigua está disponible en la salida sin necesidad de una lectura previa, y su reset síncrono se deriva del reset global del transceptor. La escritura la ordena directamente el pulso **Store_out** del receptor, que solo se emite cuando la trama ha superado las comprobaciones de paridad y bit de parada.

La habilitación de lectura se construye como la conjunción de FIFO no vacía y la señal **Data_read** procedente del PS. Esta última se toma a **nivel**: en la versión inicial se empleaba el flanco detectado de **Data_read**, lo que resulta adecuado cuando el PS lee byte a byte, pero deja de serlo con el transporte por DMA descrito en la sección 3.7, donde **Data_read** lo conduce la señal *tready* del canal AXI-Stream. La latencia del detector de flanco —dos etapas de registro— hacía que el receptor de DMA capturase varias veces la misma palabra de la FIFO. Con la lectura a nivel, la FIFO avanza exactamente en el ciclo en que el consumidor acepta el dato.

3.2.6. Bloque de canal

CONFIGURABLE_SERIAL.vhd instancia el transmisor, el receptor, el registro de desplazamiento y la FIFO, y añade la lógica de acoplamiento entre ellos y con el PS:

- **Registro de la línea de recepción.** La entrada **RD** se registra un ciclo antes de entrar al receptor, para no muestrear directamente una señal externa asíncrona

respecto del reloj del sistema.

- **Detección de flanco de los mandos del PS.** TX_Send y Data_read llegan del PS como señales de nivel a través del AXI GPIO. Sendos detectores de flanco de dos etapas de registro los convierten en pulsos de un ciclo, que es lo que esperan los bloques internos.
- **Arranque de la transmisión.** El pulso de Start hacia el transmisor solo se emite si además el transmisor está libre (EOT activo), y en ese mismo ciclo la palabra presente en Data_in se captura en un registro propio. De este modo el dato permanece estable durante toda la trama aunque el PS modifique el registro de escritura, y las peticiones que llegan con una transmisión en curso se ignoran en lugar de corromperla.
- **Adaptación del campo de parada.** El PS codifica los bits de parada en dos bits; transmisor y receptor los esperan como una cuenta de semiperiodos de tres bits. Una pequeña lógica combinacional traduce entre ambas representaciones.
- **Vector de depuración.** Una salida de cuatro bits agrupa las señales internas del camino de recepción (FIFO vacía, escritura en FIFO, bit válido y línea registrada), empleada únicamente para el diagnóstico en simulación.

3.2.7. Bloque *top* del transceptor

CONFIGURABLE_SERIAL_TOP.vhd no añade funcionalidad: empaqueta el canal completo en dos vectores, uno de entrada y otro de salida, para que pueda conectarse a los dos canales de un AXI GPIO sin lógica adicional. El reparto de bits, que es el contrato entre la PL y el driver de RTEMS descrito en la sección siguiente, se recoge en la tabla 3.4.

Bits	Campo	Función
<i>Vector de configuración y escritura (PS → PL), 28 bits</i>		
8:0	Data_in	Palabra a transmitir.
9	TX_Send	Petición de transmisión.
10	ERROR_OK	Reconocimiento de las banderas de error.
11	Data_read	Lectura de la FIFO de recepción.
17:12	baud_sel	Índice de velocidad.
19:18	stop_bit	Bits de parada.
22:20	parity	Modo de paridad.
25:23	data_bits	Número de bits de datos.
26	bit_order	Orden de bits.
27	SLO	Control del limitador de <i>slew rate</i> del transceptor físico.
<i>Vector de estado y lectura (PL → PS), 14 bits</i>		
8:0	Data_out	Palabra recibida.
9	Empty	FIFO de recepción vacía.
10	Full	FIFO de recepción llena.
11	PAR_ERROR	Error de paridad.
12	FRAME_ERROR	Error de trama.
13	TX_RDY	Transmisor libre.

Tabla 3.4: Reparto de bits de los vectores de interfaz del bloque CONFIGURABLE_SERIAL_TOP.

Junto a esos dos vectores, el bloque expone las líneas físicas del canal (TD, RD, DE y SLO) y una salida auxiliar de dos bits que duplica TX_RDY y Empty. Esta última es la que se lleva al controlador de interrupciones, de modo que el PS pueda ser avisado tanto de la llegada de datos como de la disponibilidad del transmisor sin tener que sondear el vector de estado. La salida SLO es una conexión directa desde el bit 27 del vector de configuración hasta el pin del transceptor físico: no interviene en la lógica del transceptor en VHDL.

Envoltorios de nivel superior

El canal así empaquetado es la unidad que reutilizan los tres envoltorios de nivel superior presentes en el repositorio, cada uno asociado a una de las arquitecturas de transporte entre el PS y la PL:

- **MULTI_SERIAL_CORE.vhd** agrupa hasta catorce instancias del bloque *top* en un único IP, con las interfaces de todos los canales aplanadas en vectores de ancho fijo y un par de genéricos por canal que ocultan los pines DE y SLO que no se cablean; es el bloque de catorce canales de la arquitectura basada en AXI GPIO.
- **SERIAL_CHANNEL_IP.vhd** envuelve un solo canal como IP independiente, con esos mismos genéricos, de modo que los canales puedan instanciarse y conectarse uno a

uno en el *Block Design* en lugar de como un bloque monolítico.

- `UART_AXIS.TOP.vhd` sustituye la interfaz paralela por un registro de configuración AXI-Lite y dos canales AXI-Stream —uno de transmisión y otro de recepción, este último con *TDEST* fijado al índice de canal y *TLAST* por byte—, que es lo que permite conectar el transceptor directamente a un controlador de DMA.

Ninguno de los tres modifica el comportamiento del transceptor descrito en esta sección: cambian únicamente la forma en que sus dos vectores se presentan al PS. Su papel dentro de cada arquitectura se detalla en la sección 3.7.

3.2.8. Bloque Zynq UltraScale+ MPSoC

Para que el proyecto en Vivado funcione correctamente, es importante aplicar el *preset* al bloque IP Zynq UltraScale+ antes de añadir el resto de elementos, ya que se aplican configuraciones de relojes y alimentación cruciales para el correcto funcionamiento de la PL.

3.3. Herramientas para facilitar el desarrollo del transceptor (PL)

3.3.1. Generación de transceivers con TCL

Se desarrolló un script de TCL que permite generar y conectar automáticamente un número deseado de transceptores en el *Block Design* de Vivado. Para que funcione correctamente, es necesario tener las fuentes .vhd del transceptor en el proyecto, el *Block Design* creado, y el bloque IP Zynq UltraScale+ MPSoC importado y configurado previamente; a partir de ahí, el script se encarga de añadir y conectar el resto de los bloques necesarios.

3.3.2. Generación de proyecto con TCL

También se desarrolló otro script de TCL (`regenerate_all.tcl`) que regenera el proyecto entero de Vivado con un número determinado de transceptores, partiendo desde cero. Esto permite reproducir exactamente cualquier configuración sin necesidad de guardar el proyecto de Vivado completo en el repositorio.

3.4. Diseño del driver serie en RTEMS (PS)

El subsistema de comunicaciones del OBC requiere gestionar simultáneamente múltiples interfaces serie físicas (RS422 y RS485) desde una aplicación de tiempo real ejecutada sobre RTEMS. El bloque hardware `CONFIGURABLE_SERIAL_TOP`, implementado en VHDL en la lógica programable (PL), expone cada transceptor a través de un periférico AXI GPIO de doble canal, cuya interfaz de bajo nivel es demasiado específica para ser usada directamente desde la aplicación. Para abstraer esta complejidad y ofrecer una API uniforme e independiente de la dirección física de cada instancia, se desarrollaron los archivos `transceiver.c` y `transceiver.h`.

El diseño cubre tres requisitos fundamentales: (1) descubrimiento dinámico del hardware en tiempo de arranque, de modo que el mismo binario funcione con cualquier número de

transceptores instanciados en el *bitstream*; (2) comunicación orientada a interrupciones para no bloquear el procesador durante la espera de datos; y (3) una API sencilla que permita a la aplicación enviar y recibir datos sin conocer los detalles del mapa de registros.

El driver que se describe a continuación es el de la **arquitectura de partida**, basada en AXI GPIO. Funciona, y es la referencia contra la que se mide todo lo demás, pero durante la validación con los catorce canales activos reveló un límite estructural —el procesador tiene que tocar cada byte— que motivó el rediseño completo del transporte entre el PS y la PL. Ese trabajo, con las tres arquitecturas implementadas y su comparativa, se aborda en la sección 3.7.

3.4.1. Interfaz hardware: mapa de registros

Cada instancia del transceptor ocupa una región de 4 KB del espacio de memoria del procesador, estructurada como un AXI GPIO de doble canal:

- **Canal 1 (escritura, PS → PL):** contiene el dato de transmisión (bits 8:0), los bits de control de protocolo (SEND, ERROR_OK, DATA_READ) y los campos de configuración: tasa de baudios (6 bits), bits de parada (2 bits), paridad (3 bits), bits de datos (3 bits), orden de bits y modo SLO.
- **Canal 2 (lectura, PL → PS):** contiene el byte recibido (bits 8:0) y las banderas de estado: RX_EMPTY, RX_FULL, PARITY_ERROR, FRAME_ERROR y TX_RDY.

Adicionalmente, un bloque de información del sistema situado en la dirección fija 0xA0020000 expone en tiempo de ejecución el número de transceptores instanciados, el *stride* de memoria entre instancias y la dirección base de la primera de ellas. Más allá del último transceptor se aloja el controlador de interrupciones AXI INTC, compartido por todas las instancias.

3.4.2. Arquitectura software

La librería se organiza en torno a dos estructuras de datos centrales:

`Transceiver_Config_t` agrupa los parámetros de configuración del protocolo: tasa de baudios, número de bits de datos (5 a 9), paridad (par, impar, *mark*, *space* o ninguna), bits de parada (1, 1,5 o 2), orden de bits (LSB o MSB *first*) y modo de emisión lenta (SLO, que reduce las emisiones electromagnéticas). Todos los valores se expresan mediante macros simbólicas definidas en `transceiver.h`, que corresponden directamente a los valores codificados en la LUT del NCO hardware.

`Transceiver` es el objeto *handle* que representa una instancia concreta del transceptor en ejecución. Contiene el mapa de direcciones calculado dinámicamente (`base_addr`, `intc_base` y los *offsets* derivados), las máscaras de interrupción individuales de RX y TX, y el estado software: un *buffer* circular de recepción y uno de transmisión (ambos de 4096 bytes por defecto, asignados dinámicamente), los identificadores de las primitivas RTEMS asociadas (tarea *worker*, *mutex* de RX y *mutex* de TX) y el *callback* de usuario para notificación de datos recibidos.

3.4.3. Descubrimiento dinámico del hardware

La función `Transceiver_Hardware_Discover()` lee el bloque de información del sistema al arranque y extrae el número de instancias presentes, el *stride* de memoria y la dirección base. Con estos tres valores se calcula automáticamente la dirección del INTC compartido:

$$g_intc_addr = g_hw_base + (g_hw_count \times g_hw_stride)$$

Este mecanismo permite que el mismo binario RTEMS funcione sin modificación con cualquier número de transceptores (hasta el máximo soportado de 14), cubriendo configuraciones que van desde un único canal hasta la instalación completa. La alternativa de *hardcodear* las direcciones habría requerido recompilar la aplicación para cada variante del *bitstream*.

3.4.4. Modelo de interrupciones: ISR maestra y tareas worker

El diseño de la gestión de interrupciones adopta el patrón *deferred interrupt processing* recomendado para RTEMS. Se instala una única ISR en el vector 121 (**Master_ISR**) que atiende a todos los transceptores a través del AXI INTC compartido. La ISR funciona del siguiente modo:

1. Lee el registro de interrupciones pendientes (INTC_ISR).
2. Para cada transceptor con interrupción RX activa: reconoce la interrupción (INTC_IAR), deshabilita temporalmente la línea (INTC_CIE) y envía el evento `RTEMS_EVENT_0` a la tarea *worker* correspondiente.
3. Para cada transceptor con interrupción TX activa: extrae el siguiente byte del *buffer* circular de transmisión, lo escribe en el registro de control pulsando `BIT_TX_SEND`, y limpia la interrupción. Si el *buffer* ha quedado vacío, marca `tx.busy = false`.

Cada instancia **Transceiver** tiene asociada una tarea RTEMS dedicada (**Rx_Worker_Task**) que se bloquea esperando el evento. Al recibirlo, vacía el FIFO hardware byte a byte hacia el *buffer* circular de recepción, pulsando `BIT_DATA_READ` para confirmar la lectura de cada byte al hardware. El acceso al *buffer* se protege con un semáforo binario de prioridad para evitar condiciones de carrera. Tras drenar el FIFO, la *worker* re-habilita la interrupción (INTC_SIE) e invoca el *callback* de usuario si está registrado.

La separación entre la ISR (que solo despierta la tarea) y la *worker* (que realiza el trabajo real) garantiza que el tiempo de latencia en contexto de interrupción sea mínimo y que el procesamiento se ejecute con las prioridades del planificador RTEMS.

3.4.5. Inicialización y configuración del hardware

`Transceiver_Init()` realiza la puesta en marcha completa de una instancia: calcula la dirección base y la dirección del INTC, asigna las máscaras de interrupción individuales (bit $2 \cdot id$ para RX y bit $2 \cdot id + 1$ para TX dentro del registro de 32 bits del INTC), asigna dinámicamente los *buffers* circulares, crea el semáforo y la tarea *worker* con `rtems_semaphore_create` y `rtems_task_create`, escribe la configuración de protocolo en el hardware y lanza la tarea *worker*.

La función `Transceiver_Global_INIT()` orquesta la inicialización completa del sistema: primero llama a `Transceiver_Hardware_Discover()` para poblar las variables globales,

y a continuación a `Transceiver_Global_INTC_Init()`, que habilita el modo maestro del INTC (registro MER) e instala la `Master_ISR`.

3.4.6. API pública

La interfaz expuesta al código de aplicación se reduce a cinco funciones:

Función	Descripción
<code>Transceiver_Global_INIT()</code>	Descubrimiento hardware e inicialización del INTC.
<code>Transceiver_Init(dev, id, cfg)</code>	Inicializa la instancia <code>dev</code> con el id y configuración.
<code>Transceiver_SetRxCallback(dev, cb, arg)</code>	Registra <i>callback</i> de recepción.
<code>Transceiver_Read(dev, buf, maxlen)</code>	Extrae hasta <code>maxlen</code> bytes del buffer RX.
<code>Transceiver_SendString(dev, s)</code>	Encola la cadena <code>s</code> y arranca la transmisión.

Tabla 3.5: API pública del driver serie.

3.4.7. Decisiones de diseño destacadas

Buffer circular frente a copia directa en ISR. Dado que RTEMS no permite operaciones de copia de longitud arbitraria en contexto de interrupción sin afectar la latencia del sistema, se optó por *buffers* circulares gestionados desde la tarea *worker*. El tamaño de 4096 bytes proporciona margen suficiente para ráfagas de datos a las tasas más altas soportadas (hasta 4 Mbaudios).

Transmisión dirigida por interrupción. El primer byte de cada cadena a transmitir se escribe directamente desde `Transceiver_SendString()` para arrancar el hardware; los bytes siguientes son enviados sucesivamente por la `Master_ISR` al recibir la interrupción TX_RDY, sin necesidad de que la tarea de aplicación permanezca bloqueada durante la transmisión.

Compatibilidad multi-instancia mediante tabla global. El array `g_instances[]` permite a la `Master_ISR` localizar en $O(1)$ el objeto `Transceiver` asociado a cada par de bits de interrupción, evitando búsquedas costosas en contexto de ISR.

Modo SLO. El bit BIT_SLO del registro de control **no actúa sobre la lógica en VHDL**: se limita a propagarse hasta el pin SLO del chip transceptor de la PCB, que es quien realmente limita la velocidad de flanco de sus salidas diferenciales. Un flanco más lento reduce el contenido espectral de alta frecuencia y mejora la compatibilidad electromagnética, a costa de limitar la velocidad máxima del enlace. Conviene advertir desde ya de una trampa que se destapó al caracterizar la placa (sección 4.3.3): en el transceptor empleado el pin es un habilitador de modo lento **activo a nivel bajo**, de modo que el nombre de la macro en `transceiver.h` induce a error y el valor por defecto deja el limitador *activado*.

3.5. Diseño hardware

Una vez completado el firmware del transceptor serie, se desarrolló el hardware de prueba necesario para validar el sistema en condiciones reales. El plan inicial era diseñar una única placa de comunicación serie para ejercitar múltiples líneas a la vez. Posteriormente,

el proyecto LINCE requirió dos tarjetas adicionales con especificaciones impuestas por Indra —la placa CDHS y la placa AOCS— para que Sener pudiera validar su firmware sobre la ZCU102. Los esquemáticos (PDF) y las BOM de las tres placas están disponibles en el repositorio del proyecto [3] bajo `tfm/00_docs/pcbs/`.

3.5.1. Selección de componentes: la restricción de 1,8 V

Antes de describir cada placa conviene exponer la restricción que condicionó la selección de componentes de las tres, porque no es una decisión de conveniencia sino un límite impuesto por la plataforma.

Los bancos de entrada/salida del conector FMC HPC empleados por estas tarjetas operan a **1,8 V**: es el nivel del carril VADJ con el que la ZCU102 alimenta la interfaz. Todo componente que se conecte directamente a esos pines debe, por tanto, admitir una tensión de alimentación de E/S (*VIO*) de 1,8 V. La alternativa —interponer adaptadores de nivel entre el MPSoC y cada transceptor— añade componentes activos en el camino de señal, introduce retardo de propagación en líneas que conmutan a varios megabaudios y multiplica los puntos de fallo. Se descartó salvo donde fue inevitable, y se buscaron deliberadamente componentes con *VIO* nativo a 1,8 V.

El primer criterio de búsqueda fue la **herencia de vuelo**: se exploró la posibilidad de emplear componentes ya utilizados en misiones espaciales, cualificados o con historial de vuelo demostrado, por ser la opción preferible desde el punto de vista de la fiabilidad. La búsqueda resultó infructuosa. Los transceptores de bus con herencia de vuelo disponibles en catálogo operan con niveles lógicos de 3,3 V o 5 V, herederos de una generación tecnológica anterior, y no se encontró ninguno cuya interfaz digital fuera compatible con 1,8 V. La restricción de la plataforma y el catálogo de componentes cualificados para espacio resultaron, sencillamente, incompatibles.

La consecuencia es una decisión de diseño coherente con la filosofía NewSpace descrita en la introducción: se recurrió a **componentes comerciales**, seleccionando dentro de ellos el grado de calidad más alto disponible que cumpliera la restricción eléctrica. En la práctica, eso significó priorizar piezas de **grado automotriz** (cualificación AEC-Q100, identificable por el sufijo Q1 en la referencia), que imponen rangos de temperatura extendidos y niveles de robustez muy superiores a los de grado comercial, y que son el sustituto habitual del componente cualificado para espacio en misiones de bajo coste. Los transceptores TCAN1044AVDRQ1 del bus CAN y el conversor analógico-digital ADS7950QDBTRQ1 de la placa CDHS responden a este criterio, igual que el transceptor THVD1424RGTR de los canales serie de las placas CDHS y AOCS, elegido por soportar *VIO* de 1,8 V de forma nativa.

Esta elección tiene un coste que conviene declarar: las tarjetas diseñadas son **hardware de validación funcional, no hardware de vuelo**. Permiten verificar el subsistema de comunicaciones en condiciones eléctricas realistas, que es su propósito, pero un modelo de vuelo exigiría rehacer la selección de componentes atendiendo a tolerancia a radiación, *outgassing* y cualificación térmica, y muy probablemente replantear el nivel lógico de la interfaz para poder acceder al catálogo cualificado.

3.5.2. Reglas de diseño y proceso de fabricación

El diseño de las tres placas se realizó en **Altium Designer**, y el primer paso en cualquiera de los tres proyectos —antes de colocar un solo componente— fue **establecer las reglas de diseño**. Es un paso que conviene no posponer: las reglas son el contrato que el comprobador de Altium verifica de forma continua durante el enrutado, de modo que fijarlas al principio convierte las restricciones de fabricación en un error detectado en el momento de cometerlo, en lugar de en un rechazo del fabricante cuando el diseño ya está cerrado.

El criterio adoptado fue ceñirse a las **capacidades del proceso de fabricación del fabricante seleccionado**, PCBWay [19], en lugar de definir un conjunto de reglas propio. Los límites de su proceso estándar —anchuras y separaciones mínimas de pista, diámetro mínimo de taladro, anillo de vía y distancia mínima al borde de placa— son los que acotan lo que es fabricable a coste ordinario; apurarlos por debajo obliga a recurrir a procesos de precisión, que encarecen la tirada, y superarlos con margen no aporta nada. Diseñar directamente contra esas capacidades garantiza que las tres placas fueran fabricables en la primera tirada, como en efecto ocurrió.

Sobre esas reglas generales se impusieron dos restricciones adicionales propias de las señales que transportan estas placas: **enrutado en par diferencial** con impedancia controlada para las líneas RS-422/RS-485 y CAN, cuyo desequilibrio degradaría el rechazo a modo común del que depende la inmunidad al ruido de un bus diferencial; y **longitud acotada** en las líneas de conmutación rápida procedentes del FMC.

3.5.3. Diseño de la placa de comunicación serie

El objetivo de esta placa es poder ejercitar simultáneamente los 14 transceptores del IP core, replicando las topologías de bus reales que se encontrarán en el satélite: RS485 multipunto con múltiples nodos en el mismo cable, y RS422 en configuración *master/slave* con cruce de TX y RX. A diferencia de las placas CDHS y AOCS, cuyas especificaciones vinieron dictadas por Indra, esta placa fue diseñada con total libertad para maximizar la flexibilidad de los ensayos en laboratorio.

El transceptor seleccionado para los catorce canales es el **LTC2865** [20] de Analog Devices, un transceptor RS-485/RS-422 de hasta 20 Mbps cuyo pin de alimentación lógica independiente (VL) le permite operar la interfaz digital directamente a los 1,8 V del FMC —la restricción de la sección 3.5.1—, y que expone además un pin SLO de limitación de *slew* controlable desde el MPSoC, que tendrá un papel protagonista en la caracterización de la placa (sección 4.3.3). Todas las líneas diferenciales incorporan diodos TVS **SM712-02HTG** contra transitorios, y cada driver dispone de una resistencia de terminación de 120 Ω conmutable mediante *jumper*, lo que permite terminar el bus en los nodos extremos y dejar abiertos los intermedios.

La arquitectura general de la placa divide los 14 canales en dos grupos conectados al FMC:

- **7 drivers RS485 (RS0–RS6)** conectados a un bus diferencial común mediante *jumpers*, formando una topología multipunto configurable.
- **7 drivers RS422 (RS7–RS13)** organizados en dos buses *master/slave* independientes (BUS 1: 1 *master* + 3 *slaves*; BUS 2: 1 *master* + 2 *slaves*), con la línea TX

cruzada con RX para emular la conexión real punto a punto RS422.

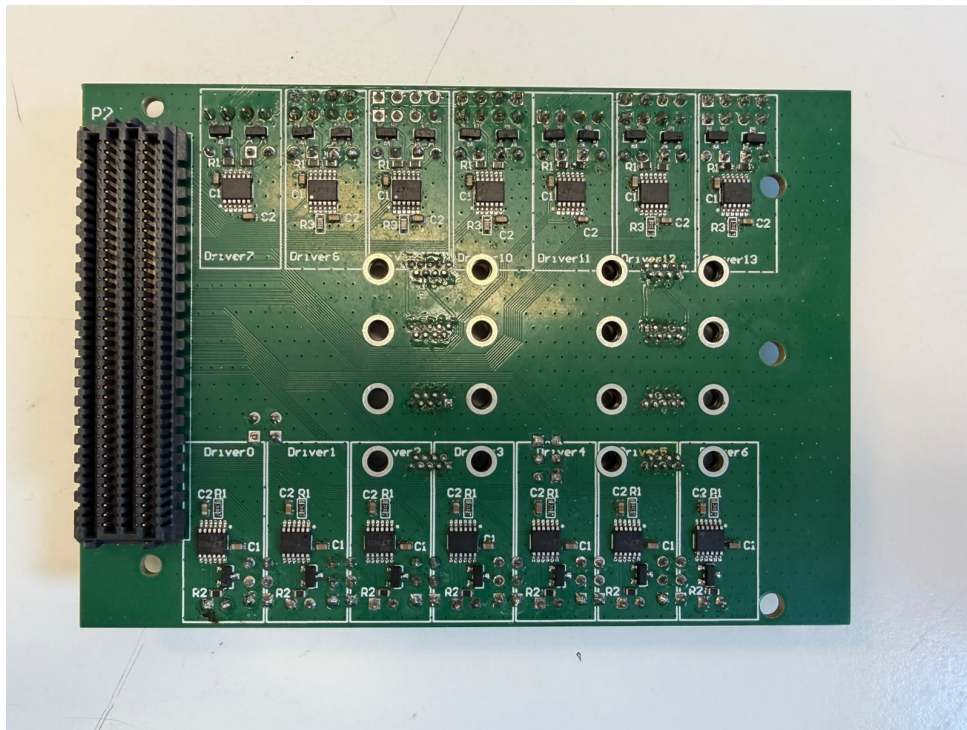


Figura 3.6: Arquitectura general de la placa de comunicación serie: los siete drivers RS422 (Driver7–Driver13, arriba), el conector FMC (P2, izquierda) y los siete drivers RS485 (Driver0–Driver6, abajo).

Topología RS485: bus compartido configurable por jumper

La decisión de diseño más relevante de esta placa es el mecanismo de bus compartido RS485 sin necesidad de recablear. Cada driver RS485 tiene un conector Dupont de 3 pines (A+, B–, GND) que actúa a la vez como punto de conexión hacia el exterior y como punto de interconexión entre drivers adyacentes. Colocando un *jumper* entre dos conectores consecutivos, las líneas A y B de ambos drivers quedan físicamente unidas, formando un segmento de bus RS485 compartido. Retirando el *jumper*, el driver queda independiente y puede conectarse a un periférico externo de forma individual.

Este mecanismo permite configurar por hardware cualquier partición de los 7 drivers RS485 en uno o varios buses, sin modificar el firmware ni el cableado exterior.

Selector de conector Micro-D para RS485

Los conectores de mayor densidad (Micro-D) de la placa son compartidos entre el Driver 0 y el Driver 6 mediante un *jumper* de selección de hardware. Dependiendo de la posición del *jumper*, los conectores Micro-D quedan asignados a uno u otro driver, lo que permite usar un único tipo de conector con cableado de satélite sin duplicar el número de conectores en la placa.

Topología RS422: master/slave configurable por jumper

Para RS422, cada driver *slave* dispone de un conector con las líneas TX cruzadas a RX (TX-Y+/TX-Z– conectadas a RX del bus), replicando la conexión estándar punto a punto

RS422 donde el receptor del esclavo escucha las transmisiones del *master*. Al igual que en RS485, un *jumper* determina si el driver se une al bus común o permanece independiente.

Alimentación

La placa se alimenta íntegramente del conector FMC, que suministra los carriles de 12 V, 3,3 V y VADJ a 1,8 V. Los transceptores LTC2865 operan con la etapa de bus a 3,3 V (VCC) y la interfaz lógica a 1,8 V (VL), de modo que —a diferencia de las placas CDHS y AOCS, que generan con un convertidor DC/DC los 5 V de su etapa de potencia— esta placa no necesita ninguna etapa de conversión: todos los carriles llegan directamente del FMC. Sendos conectores de dos pines dan acceso externo a los carriles de 12 V y 3,3 V.

El mapa completo de señales entre esta placa y el conector FMC se encuentra en el Anexo 1 de este documento. El esquemático completo (12 hojas, PDF) y la BOM están disponibles en `tfm/00_docs/pcbs/serial/` del repositorio del proyecto.

3.5.4. Diseño de la placa CDHS

La placa LINCE3 CDHS BreakoutBox es una tarjeta de interconexión entre la ZCU102 (conectada por FMC) y los periféricos del subsistema de gestión de datos y calentadores del satélite. Sus especificaciones fueron definidas por Indra: 6 conectores D-Sub-9 (J1–J6), interfaces CAN redundante, tres canales RS422/RS485, cuatro líneas PWM para calentadores y un ADC para termistores.

[Figura pendiente: diagrama jerárquico CDHS — bloque CAN (J1), RS (J2–J4), PWM (J5), ADC/THERM (J6)]

Figura 3.7: Diagrama de bloques de la placa CDHS.

[Figura pendiente: render 3D de la PCB CDHS]

Figura 3.8: Render 3D de la placa CDHS.

Nivel de tensión y selección de componentes

Como en las otras dos tarjetas, todos los bancos de pines del conector FMC HPC utilizados por esta placa operan a 1,8 V, y los componentes se seleccionaron con *VIO* nativo a ese nivel para evitar etapas de adaptación en la cadena de señal. Las implicaciones de esta restricción —y el motivo por el que obligó a descartar los componentes con herencia de vuelo— se discuten en la sección 3.5.1. Los subsistemas que siguen aplican ese criterio, y las excepciones —las señales de PWM, que sí requieren adaptación de nivel— se señalan donde corresponde.

Subsistema CAN

El bus CAN implementa topología redundante con dos instancias independientes: CAN Nominal (CAN_NOM) y CAN Redundante (CAN_RED). Se seleccionó el transceptor

TCAN1044AVDRQ1 de Texas Instruments por su compatibilidad nativa con VIO de 1,8 V, grado automotriz y baja corriente en modo *standby*. Ambos canales convergen en el conector J1.

Las resistencias de terminación siguen el esquema *split termination* recomendado en las notas de aplicación del estándar CAN (ISO 11898-2). En lugar de una única resistencia de 120 Ω entre CANH y CANL, se emplean **dos resistencias de 60 Ω en serie** con un **condensador de 4,7 nF** conectado desde el punto medio al plano de masa. Cada una de las dos resistencias de 60 Ω está controlada por un *jumper* independiente, lo que permite habilitar o deshabilitar la terminación sin modificar el circuito. La resistencia equivalente sigue siendo 120 Ω cuando ambos *jumpers* están colocados, y el condensador de derivación crea un camino de baja impedancia para las interferencias de modo común de alta frecuencia hacia masa, mejorando el comportamiento EMC del bus.

Las líneas CANH y CANL de cada canal incorporan diodos de protección ESD **ESDCAN24-2BLY**, específicamente diseñados para buses CAN, que clampean transitorios por encima de su tensión de ruptura sin introducir una capacitancia parásita significativa.

[Figura pendiente: esquemático CAN.SchDoc — transceptores TCAN1044 con terminación split y diodos ESD]

Figura 3.9: Subsistema CAN de la placa CDHS.

Subsistema RS422/RS485

Se disponen tres canales serie idénticos e independientes (RS1, RS2, RS3), cada uno con el transceptor **THVD1424RGTR**. Este integrado llegó como sugerencia de un compañero del proyecto LINCE en una reunión de seguimiento, posterior al diseño de la placa de comunicación serie propia, y resultó ser una opción mejor que el LTC2865 empleado en aquella: además de soportar VIO de 1,8 V, incorpora resistencias de terminación internas conmutables y permite seleccionar entre modo *Half-Duplex* y *Full-Duplex* sin cambiar el hardware, por lo que se adoptó para las dos tarjetas oficiales. La configuración se expone mediante *jumpers* físicos de 2 pines: **H/F** (conmuta entre *Half-Duplex* y *Full-Duplex*), **SLR** (habilita control de *slew rate*), **TERM_TX / TERM_RX** (conectan las resistencias de terminación internas). Los tres canales salen por los conectores J2, J3 y J4.

Tanto las líneas RX como las TX de cada canal llevan un par de diodos TVS **SM712-02HTG** conectados entre las líneas diferenciales y el plano de masa.

Decisión de diseño: cortocircuito DE–RE. El THVD1424 expone dos pines de control independientes: **DE** (*Driver Enable*, activo en alto) habilita el transmisor, y **RE** (*Receiver Enable*, activo en bajo) habilita el receptor. En el esquemático, ambos pines están conectados a la misma señal de control procedente del MPSoC, de modo que una única línea digital controla simultáneamente transmisor y receptor de forma complementaria (tabla 3.6).

Señal DE/RE	Transmisor (DE)	Receptor (RE activo bajo)
1 (alto)	Habilitado	Deshabilitado
0 (bajo)	Deshabilitado	Habilitado

Tabla 3.6: Control complementario DE/RE en el THVD1424.

Esta decisión responde a dos motivos. Por un lado, el equipo de Indra comunicó que en la arquitectura del satélite los esclavos solo transmiten bajo demanda del OBC, por lo que no existe un escenario real en el que un nodo deba transmitir y recibir simultáneamente. Por otro lado, en modo *Half-Duplex* las líneas TX y RX comparten físicamente el mismo par diferencial A/B; si el receptor permaneciera activo durante la transmisión, el nodo escucharía su propio eco.

[Figura pendiente: RS.SchDoc — circuito completo de un canal RS con THVD1424RGTR y diodos TVS]

Figura 3.10: Canal RS de la placa CDHS con THVD1424RGTR.

Subsistema PWM (calentadores)

Cuatro señales PWM de 1,8 V procedentes del FMC se elevan a 3,3 V mediante el adaptador de niveles **TXU0104PWR** para excitar los circuitos de control de calentadores externos. Las cuatro líneas adaptadas salen por J5.

Para la validación de esta interfaz se diseñó un bloque VHDL autónomo, **PWMx4_auto_test**, sintetizado en la PL junto al resto del diseño de Vivado. El bloque genera cuatro señales PWM independientes con *duty cycle* del 50 % a partir del reloj de 100 MHz de la FPGA: canal 0 a 10 kHz, canal 1 a 5 kHz, canal 2 a 1 kHz y canal 3 a 100 Hz. Al ser completamente autónomo no requiere ningún driver ni intervención del PS, lo que permitió verificar el subsistema PWM de la placa con el mismo BOOT.BIN utilizado para las pruebas serie.

Subsistema ADC (termistores)

Cuatro entradas analógicas de termistores (CH0–CH3) se digitalizan con el ADC SAR de 12 bits **ADS7950QDBTRQ1**, comunicado al procesador por SPI a 1,8 V. La circuitería analógica opera a 3,3 V; ambos carriles (+VBD a 1,8 V y +VA a 3,3 V) son independientes para evitar acoplos entre el dominio digital y el analógico [21]. Los termistores se conectan por J6.

Alimentación

El conector FMC suministra 12 V, 3,3 V y el carril VADJ a 1,8 V. El convertidor conmutado **R-78E5.0-1.0** genera los 5 V necesarios para la etapa de potencia de los transceptores RS y CAN a partir de los 12 V del FMC. Se eligió este módulo DC/DC por su integración en un *footprint* reducido de 3 pines (equivalente a un regulador lineal) y su eficiencia $\geq 96\%$, evitando la disipación térmica que tendría un LDO con ese diferencial de tensión.

Conectores externos

Los seis puertos D-Sub-9 (J1–J6) son de tipo macho o hembra según la convención de uso. Los escudos metálicos de todos los conectores están conectados al plano de GND para mantener la continuidad de apantallamiento EMI con los cables blindados.

El mapa completo de señales entre la placa CDHS y el conector FMC se encuentra en el Anexo 1 de este documento. El esquemático (PDF) y la BOM están disponibles en [tfm/00_docs/pcbs/cdhs/](#).

3.5.5. Diseño de la placa AOCS

La placa LINCE3 AOCS BreakoutBox es la tarjeta de interconexión para el subsistema de control de actitud y órbita del satélite. Al igual que la CDHS, conecta la ZCU102 mediante FMC y expone las interfaces al exterior a través de 8 conectores D-Sub-9 (J1–J8).

[Figura pendiente: diagrama jerárquico AOCS — bloques RS (J1, J3–J5, J8), MOT-PWM (J2), SpaceWire (J6, J7)]

Figura 3.11: Diagrama de bloques de la placa AOCS.

[Figura pendiente: render 3D de la PCB AOCS]

Figura 3.12: Render 3D de la placa AOCS.

Subsistema RS422/RS485

La placa incorpora 5 canales serie (RS1, RS3, RS4, RS5, RS8) con exactamente el mismo diseño de transceptor THVD1424RGTR descrito en la sección de la placa CDHS, incluyendo el cortocircuito DE–RE y los *jumpers* de configuración H/F, SLR y terminación.

Subsistema SpaceWire

Se implementan dos enlaces SpaceWire *full-duplex* independientes (SPW1 y SPW2) mediante señalización diferencial LVDS, sin transceptor activo adicional, ya que el estándar SpaceWire opera directamente a niveles LVDS compatibles con los bancos del FMC. Los pares diferenciales —cuatro por enlace: DIN, SIN, SOUT y DOUT— se han trazado en la PCB con técnicas de igualación de longitud (*length matching*) para minimizar el *skew* entre el par de datos y el par de *strobe*, y con impedancia característica controlada de 100 Ω diferencial.

[Figura pendiente: esquemático SpaceWire — cuatro pares diferenciales y su conexión al FMC]

Figura 3.13: Subsistema SpaceWire de la placa AOCS.

Subsistema de control de motores (MOT-PWM)

Seis señales PWM de 1,8 V procedentes del FMC se elevan al nivel de tensión requerido por los puentes en H externos mediante un adaptador de niveles. Las señales se agrupan en tres pares (X, Y, Z para los tres ejes del satélite), con dos líneas por eje para controlar la dirección de giro del motor. Salen por J2. La alimentación de potencia de los motores se conecta mediante dos bornes banana independientes de la alimentación de la placa.

Alimentación

Misma arquitectura que la placa CDHS: 12 V del FMC, convertidor DC/DC para los 5 V de potencia, y VADJ 1,8 V para los transceptores. La única diferencia es la etapa de potencia de los motores, que no se alimenta del FMC: admite alimentación exterior a través de los dos bornes banana, independiente del resto de la placa.

El mapa completo de señales entre la placa AOCS y el conector FMC se encuentra en el Anexo 1. El esquemático (PDF) y la BOM están disponibles en [tfm/00_docs/pcbs/aocs/](#).

3.5.6. Fabricación

Una vez validados los esquemáticos y completado el rutado de las PCBs, se encargaron las placas con stencil a **PCBWay**, y los componentes a **Mouser** y **DigiKey** según disponibilidad. El proceso de montaje en el laboratorio siguió los pasos habituales de soldadura por reflujo SMD:

1. **Aplicación de pasta de soldadura.** Se fijó la placa a la mesa con placas de idéntico grosor alrededor y cinta de pintor. Se colocó el stencil alineado y se extendió la pasta con espátula.
2. **Colocación de componentes.** Con pinzas de punta fina, se colocaron los componentes SMD sobre la pasta. Se montaron dos placas en paralelo para optimizar el tiempo de proceso.
3. **Reflujo.** Las placas se introdujeron en el horno de reflujo del laboratorio seleccionando el perfil de temperatura adecuado para la pasta de soldadura utilizada.

[Figura pendiente: foto del proceso de fabricación — aplicación de pasta con stencil]

Figura 3.14: Aplicación de pasta de soldadura con stencil.

[Figura pendiente: foto de la placa CDHS soldada (cara superior)]

Figura 3.15: Placa CDHS tras el proceso de soldadura.

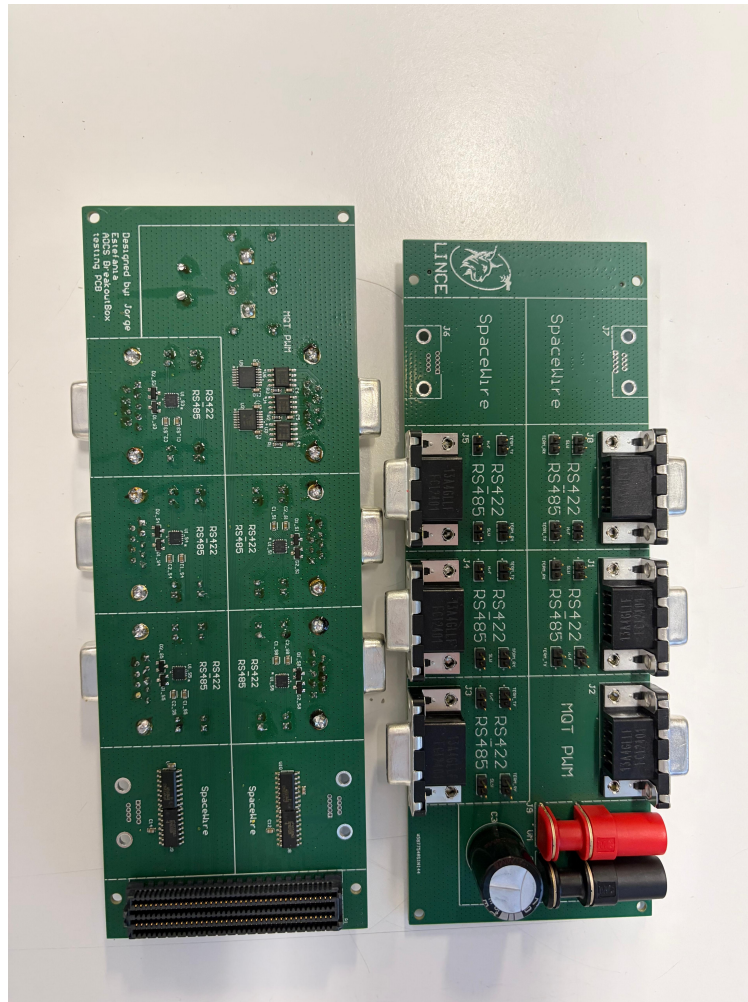


Figura 3.16: Placa AOCS tras el proceso de soldadura.

3.6. Desarrollo de herramientas de testing

3.6.1. Aplicación de testing del driver serie en RTEMS

La aplicación de testing del driver serie está estructurada en tres archivos:

- `init.c`: configuración del sistema RTEMS mediante macros de `confdefs.h`.
- `transceiver.c` + `transceiver.h`: driver del transceptor serie sobre AXI GPIO.
- `main.c`: aplicación de testing que usa la API del driver.

Configuración del sistema RTEMS — `init.c`

`init.c` es el fichero de configuración estática de RTEMS. No contiene lógica de aplicación; define el entorno de ejecución mediante macros que `<rtems/confdefs.h>` convierte en estructuras de datos internas del kernel:

Programación 3.3: Configuración estática de RTEMS en `init.c`

```
1  CONFIGURE_APPLICATION_NEEDS_CLOCK_DRIVER    //
    rtems_task_wake_after()
```

```

2  CONFIGURE_APPLICATION_NEEDS_CONSOLE_DRIVER // stdin/stdout (
    printf, fgets)
3  CONFIGURE_MICROSECONDS_PER_TICK 10000      // Tick = 10 ms
    (100 Hz)
4  CONFIGURE_UNLIMITED_OBJECTS                // Sin limite de
    tareas/semaforos
5  CONFIGURE_UNIFIED_WORK_AREAS              // Un unico pool de
    memoria
6  CONFIGURE_INIT_TASK_STACK_SIZE (64 KB)     // Stack grande para
    Init

```

Aplicación de testing — main.c

La aplicación tiene tres secciones claramente separadas:

Callback de recepción on_rx_data(). Se registra en el driver y se invoca desde la tarea *worker* cuando llegan datos. Implementa un ensamblador de líneas por canal: por cada byte recibido, acumula hasta encontrar un carácter de fin de línea (`\n` o `\r`), momento en el que imprime la línea completa con el formato `[RX UART 02]: mensaje`. Hay un `AppLineBuffer` (1024 bytes) por cada uno de los 14 canales, declarados estáticamente en el array `rx_lines[MAX_TRANSCEIVERS]`, para evitar mezclar caracteres de UARTs distintas en la misma línea de consola.

Tarea de consola TX Tx_Console_Task(). Es una tarea RTEMS independiente (prioridad 100, baja) que bloquea en `fgets(stdin)` esperando comandos del usuario por la consola USB. El protocolo de comandos es:

Programación 3.4: Protocolo de comandos de la aplicación de testing

```

1  <ID> <MENSAJE>      -> Envia MENSAJE por la UART numero ID
2  ALL <MENSAJE>       -> Envia MENSAJE por TODAS las UARTs
3  <ID> SLO ON         -> Reconfigura UART ID con Slew Rate
    limitado
4  <ID> SLO OFF        -> Reconfigura UART ID con Slew Rate normal
5  ALL SLO ON/OFF     -> Aplica configuracion SLO a todas las
    UARTs

```

Los comandos SLO re-ejecutan `Transceiver_Init()` con una configuración distinta, lo que permite reconfigurar el hardware en caliente sin reiniciar el sistema.

Función Init() — punto de entrada RTEMS. RTEMS la llama al arrancar en lugar de `main()`. La secuencia de inicialización es:

1. `mmu_map_pl_axi_early()`: mapea en la MMU la región del PL para acceso *bare-metal*.
2. `Transceiver_Global_INIT()`: descubre el hardware e instala la ISR maestra.
3. Bucle de inicialización: `Transceiver_Init()`, `Transceiver_SetRxCallback()` y `Transceiver_SerN Lista.\r\n"` para cada UART.
4. Creación y lanzamiento de `Tx_Console_Task`.
5. `rtems_task_delete(RTEMS_SELF)`: la tarea Init se elimina a sí misma; el kernel queda con las *worker tasks* + la tarea de consola.

3.6.2. Aplicación de testing de las placas CDHS y AOCS

Ampliación en PL para soportar interfaces adicionales CDHS

Para poder probar la funcionalidad de la placa CDHS, fue necesario añadir soporte firmware que controlara las líneas adicionales a RS422 y RS485. La configuración del entorno fue la siguiente: se preparó el hardware en la PL para probar las líneas de RS422/RS485, SPI, CAN y PWM creando un proyecto en Vivado con el script TCL `regenerate_all` con 3 transceptores serie, y añadiendo el bloque de control PWM. Además se configuró el PS para conectar SPI y CAN con el exterior activando las líneas de SPI 0, CAN 0 y CAN 1 como interfaces externas hacia el conector FMC.

Para controlar las líneas de PWM se instanció el bloque `PWMx4_auto_test`, que genera desde la PL cuatro señales PWM a 10 kHz, 5 kHz, 1 kHz y 100 Hz con *duty cycle* del 50 %, sin necesidad de ningún driver en el PS. Se utilizó el mismo `BOOT.bin` para todas las pruebas de CDHS.

Para probar el ADC ADS7950 de la placa CDHS, se desarrolló una pequeña aplicación de lectura SPI sobre RTEMS. Dado que el BSP de RTEMS 7 para ZCU102 no incluía en ese momento un driver SPI de alto nivel, se accedió directamente al controlador SPI Cadence integrado en el PS mediante mapeado de registros en memoria (MMIO), siguiendo el mapa de registros descrito en el Manual de Referencia Técnico del Zynq UltraScale+ [22]. El protocolo de comunicación con el ADC —formato de trama de 16 bits, selección de canal en *Manual Mode* y gestión de la latencia de conversión— se implementó conforme al *datasheet* del ADS7950 [21].

Para probar el CAN, se utilizó una aplicación desarrollada por Diego Ramos, compañero en el laboratorio B105 y en el proyecto LINCE. En su aplicación, se configuran las líneas de CAN y se establecen tests que comprueban que la conexión por CAN funciona correctamente.

Ampliación para soportar interfaces adicionales AOCS

Para probar la placa AOCS, se generó un proyecto de Vivado con `regenerate_all` con 5 transceptores, añadiendo un bloque VHDL para controlar los puentes en H por PWM (`Motor_H_bridge_test.vhd`). Este bloque genera 3 PWMs y los enruta por la línea 1 o línea 2 de cada motor en función de la dirección deseada. Para estas pruebas, se fijó un tiempo para cada dirección de manera que durante unos segundos el PWM fuera en un sentido (línea 1 activa, línea 2 a 0) y durante los siguientes segundos en el sentido contrario.

La comunicación por SpaceWire quedó fuera del alcance de la validación por su alta complejidad técnica: el estándar no se agota en la capa física LVDS que expone la placa, sino que exige implementar en la lógica programable la codificación *data-strobe*, la máquina de estados de establecimiento y recuperación del enlace y los niveles de paquete y red, un desarrollo que por sí solo excede el alcance de este trabajo.

3.6.3. Validación de hardware: arneses y equipamiento

Arnés para verificar comunicaciones RS422

Arnés a 4 hilos, cruzando las líneas TX-P/N de un driver con las RX-P/N del opuesto (figura 3.17).



Figura 3.17: Arnés de *loopback* RS422 a 4 hilos entre dos conectores D-Sub-9.

Arnés para verificar comunicaciones RS485

Arnés a 2 hilos, en el que se conectan A+ y B— de un driver con los del otro (figura 3.18).

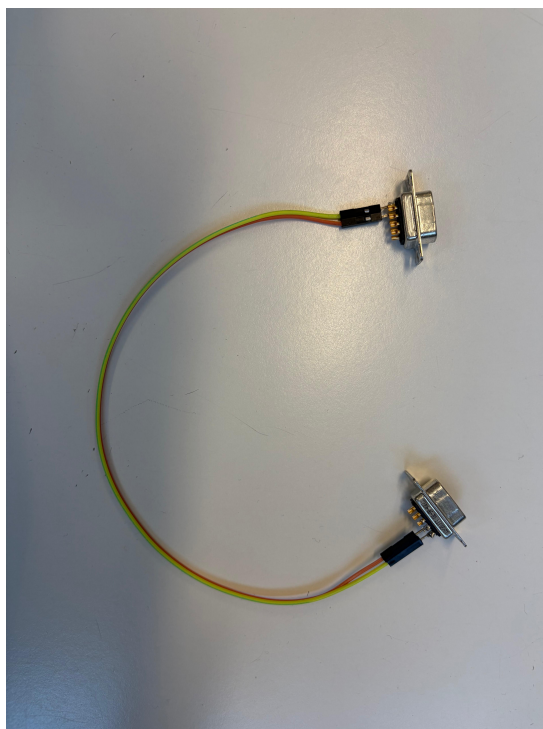


Figura 3.18: Arnés de *loopback* RS485 a 2 hilos entre dos conectores D-Sub-9.

Arnés para verificar comunicaciones CAN

Se conectan las líneas CAN_H y CAN_L del canal nominal con las del canal redundante. Como ambos canales convergen en el mismo conector J1, el arnés emplea un único D-Sub-9 con los hilos cruzados entre ambos pares, dejando puntas accesibles para la sonda del osciloscopio (figura 3.19).

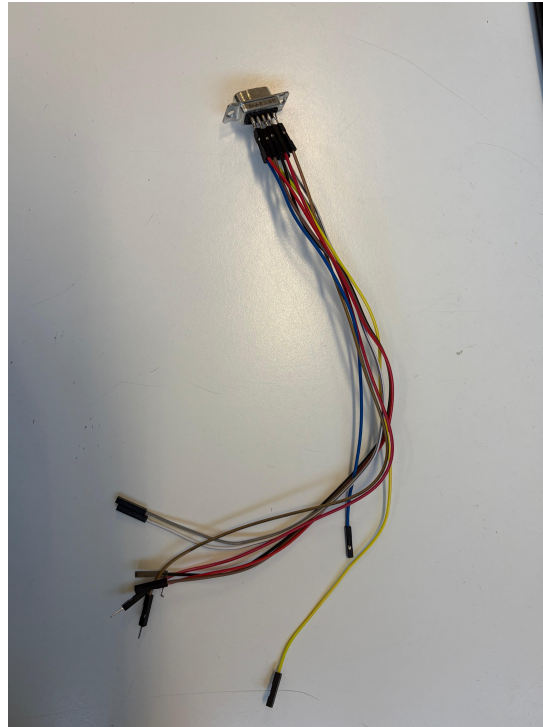


Figura 3.19: Arnés de interconexión CAN nominal–redundante sobre el conector J1.

Equipo de laboratorio utilizado

Se utilizó un osciloscopio y una fuente de alimentación de hasta 32 V.

3.7. Arquitecturas de transporte PS–PL

Las secciones anteriores describen el transceptor serie en la PL y el driver que lo gobierna desde RTEMS. Ambos funcionan, pero durante la validación con los catorce canales activos apareció un límite que no era del transceptor ni del software, sino **del camino por el que los bytes viajan entre la DDR y la lógica programable**: cada byte recibido genera una interrupción, y con catorce canales el procesador no da abasto.

Esa observación reorientó la última fase del trabajo hacia una pregunta concreta: *¿cuál es la arquitectura de transporte PS–PL adecuada para catorce líneas serie sobre este MPSoC?* Para responderla con datos y no con estimaciones se implementaron **tres arquitecturas completas** y se midieron las tres con el mismo programa de pruebas. El transceptor serie de la sección anterior es idéntico en las tres; lo único que cambia es el transporte y el driver `transceiver.c/h` que lo maneja.

Variante	Transporte PS↔PL	Idea
A — GPIO	AXI-Lite / AXI GPIO	Un registro por canal; el PS mueve byte a byte.
B — DMA14	14 × AXI DMA (PG021)	Un motor DMA completo por canal.
C — MCD-MA	1 × AXI MCDMA (PG288) + puente VHDL	Un motor multicanal compartido.

Tabla 3.7: Las tres arquitecturas de transporte implementadas y comparadas.

Las tres son funcionales y arrancan sobre la misma placa con el mismo código de aplicación. El capítulo de resultados (sección 4.3.2) recoge la comparativa cuantitativa; esta sección describe cómo está construida cada una y qué se aprendió al construirla.

3.7.1. Variante A — AXI GPIO (referencia)

Es la arquitectura de partida, la descrita en las secciones anteriores de este capítulo, y la referencia contra la que se mide todo lo demás. El IP `MULTI_SERIAL_CORE` agrupa los catorce canales bajo una única interfaz AXI-Lite; cada canal expone un registro de escritura (dato de transmisión, bits de control y configuración de protocolo) y un registro de lectura (dato recibido y banderas de estado). Un AXI INTC concentra las veintiocho fuentes de interrupción —RX y TX de cada canal— en una sola línea hacia el PS.

Su virtud es la simplicidad: es la variante con menos lógica, no necesita ningún mapeo especial de la MMU y no tiene que preocuparse por la coherencia de caché, porque todos los accesos son del propio procesador. Su defecto es estructural y no tiene arreglo dentro de esta arquitectura: **el procesador tiene que tocar cada byte**. En recepción, cada carácter completo genera una interrupción; la ISR maestra despierta a la tarea *worker* del canal, que drena el FIFO hardware byte a byte. En transmisión, la ISR escribe el siguiente byte cada vez que el transmisor queda libre.

El coste medido es de **1250 interrupciones por kilobyte** transferido. Con eso, el *throughput* de un canal se estanca en torno al 30 % del techo físico de la línea, y —lo que es más revelador— **no mejora al subir la velocidad**: de 115 200 a 4 Mbaudios el rendimiento apenas pasa de 3471 a 4907 B/s, porque el cuello de botella ya no es la línea serie sino el manejo de interrupciones. El sistema está en el régimen de *interrupt livelock* descrito en el marco teórico.

3.7.2. Variante B — catorce AXI DMA

La primera respuesta al problema es la más directa: si el coste está en que la CPU toque cada byte, se pone un DMA que los mueva por ella. Y si un `axi_dma` sirve a un solo *stream*, se instancian catorce.

Adaptación del UART a AXI-Stream

El transceptor original habla con registros, no con *streams*. El módulo `UART_AXIS_TOP.vhd` hace de adaptador: envuelve un `CONFIGURABLE_SERIAL` y le añade una interfaz AXI4-Stream esclava en el lado de transmisión (TDATA, TVALID, TREADY) y una maestra en el

de recepción (con **TLAST** para delimitar el paquete). Cada uno de los catorce UART se convierte así en un periférico AXI-Stream que cuelga de su propio DMA.

Operación

Los DMA se configuran en modo directo, sin *scatter-gather*, que es todo lo que necesita un canal con un único *buffer* activo:

- **Transmisión.** El driver escribe la dirección del *buffer* y la longitud en los registros del canal MM2S; el motor lee la DDR y alimenta el *stream*. La ISR de MM2S libera un semáforo al completar la transferencia.
- **Recepción.** El canal S2MM se arma con longitud 1. El **TLAST** que el adaptador emite con cada byte completa el paquete, la ISR de S2MM copia el dato al *ring* de software y rearma el descriptor.

Lo que se aprendió: el precio de replicar

La variante se hizo funcionar en placa, y en transmisión consigue exactamente lo que se buscaba: **3 interrupciones por kilobyte**, frente a las 1250 de la variante GPIO. Pero el enfoque no escala, por dos razones que solo se ven al implementarlo:

El coste en lógica. Catorce motores DMA completos ocupan **40 217 LUT**, un 14,7 % del dispositivo: 2,8 veces más lógica que la solución multicanal, para el mismo número de canales. La cifra por canal —2873 LUT/canal frente a los 1014 del MCDMA— expresa directamente lo que cuesta replicar en vez de compartir. Y ese coste no es solo de hoy: hipoteca el endurecimiento futuro del diseño, porque una triplicación modular (TMR, *Triple Modular Redundancy*) de la lógica —la técnica habitual para tolerar los efectos de la radiación en órbita— multiplicaría por tres esa huella y la llevaría a cerca de la mitad del dispositivo, mientras que sobre la variante multicanal sigue siendo asumible.

El límite de las interrupciones. Cada *axi_dma* expone dos líneas de interrupción, MM2S y S2MM. Catorce DMA son **veintiocho líneas**, y el PS solo ofrece ocho en *pl_ps_irq0*. Hubo que reducirlas por OR en la propia PL hasta dejarlas en dos, con lo que la ISR pierde la información de *qué* canal ha interrumpido y debe escanear los catorce registros de estado en cada evento. El ahorro de interrupciones que aportaba el DMA se paga en parte con un coste de escaneo dentro de la ISR.

Ese límite de ocho líneas es, literalmente, lo que empuja a la tercera variante: no es una limitación del diseño propio, sino del silicio, y no se puede resolver replicando más.

La recepción de esta variante, sin embargo, **no llegó a medirse**. El banco de pruebas de la sección 3.7.5 exige insertar el módulo de *loopback* en el diseño de cada variante, y sobre esta no se consiguió hacer funcionar el lazo de recepción: el autodiagnóstico del banco lo detectó abierto y todos los tests que dependen de recibir salieron a cero. Llegados a ese punto había que decidir dónde invertir el tiempo restante, y los datos ya disponibles hacían la decisión sencilla: con el coste en lógica y el límite de las líneas de interrupción sobre la mesa, esta arquitectura estaba descartada como solución final con independencia de lo que midiera su recepción. Se optó por no seguir depurándola y dedicar el esfuerzo al diseño de la variante MCDMA. Sus cifras de recepción se hacen constar como no disponibles en los resultados; lo que sí es válido de su corrida —y es lo que sostiene el argumento— es el coste de hardware, la huella de memoria y las interrupciones de transmisión.

3.7.3. Variante C — AXI MCDMA con puente VHDL

La tercera arquitectura, y la que se adopta como solución final, sustituye los catorce motores por **uno solo multicanal** (`axi_mcdma`, PG288) y añade un **puente en VHDL** que reparte y recoge de los catorce UART. Es la variante con más diseño propio y la única en la que el problema deja de resolverse instanciando IP de terceros.

La topología es deliberadamente **asimétrica**, porque los dos sentidos tienen exigencias distintas:

- En **transmisión** el PS decide cuándo y a quién escribe. Basta con un canal DMA si el destino viaja *dentro* del dato.
- En **recepción** los catorce canales pueden hablar a la vez y sin avisar. Cada uno necesita su propio *buffer* en la DDR para que los flujos no se mezclen, lo que exige catorce canales S2MM.

[Figura pendiente: diagrama de bloques de la variante MCDMA — MM2S (1 canal) → conversor 32→8 → BRIDGE_TX_TOP → 14 UART; 14 UART → BRIDGE_RX_TOP → conversor 8→32 → S2MM (14 canales)]

Figura 3.20: Arquitectura asimétrica de la variante MCDMA: un canal de transmisión y catorce de recepción.

Camino de transmisión: un canal para catorce UART

El canal MM2S del MCDMA emite un único *stream* de 32 bits, que un conversor de anchura reduce a 8. La pregunta es cómo saber a cuál de los catorce UART va cada byte con un solo canal de DMA. La respuesta es que **el destino viaja en la cabecera del propio paquete**:

```
[ {CH_ID[3:0], LEN[11:8]} | {LEN[7:0]} | payload...]
```

Dos bytes de cabecera con el identificador de canal y la longitud del *payload*. El bloque BRIDGE_TX_TOP los interpreta y encamina el resto del paquete:

- TX_DMA_ROUTER parsea la cabecera y extrae el canal y la longitud.
- MUX_2x16 demultiplexa la escritura hacia la FIFO del canal indicado.
- Catorce TX_WORKER, uno por canal, con una FIFO de 8 bits en modo *first-word-fall-through* y una máquina de estados (S_IDLE → S_START → S_BUSY) que alimenta el núcleo UART byte a byte, respetando su *handshake*: presenta el byte, espera a que el transmisor lo acepte (Send_ok), lo extrae de la FIFO y espera a que termine de serializarlo (TX_RDY) antes de presentar el siguiente.

Un único canal DMA sirve así a los catorce transceptores. El precio es que las transmisiones se serializan: el driver protege el acceso al canal compartido con un *mutex*, y por eso esta variante es la única en la que `Transceiver_Send` es bloqueante.

Notificación de fin de transmisión: el bloque TX_ISR_EOF_HANDLER

El DMA sabe cuándo ha terminado de *entregar* el paquete al puente, pero no cuándo el UART ha terminado de *emitirlo* por el cobre. Para una línea RS-485 con control de dirección esa diferencia importa: no se puede soltar el bus antes de que el último bit haya salido.

Sacar los catorce pulsos de fin de trama a catorce líneas de interrupción no es una opción —las líneas son ocho—. Se diseñó por tanto un bloque propio, TX_ISR_EOF_HANDLER.vhd, que aplica el patrón de **registro *sticky* más una única interrupción**: engancha el pulso de fin de trama de cada *worker* en un bit de un registro, lo expone por AXI-Lite y genera una sola interrupción de nivel. Su mapa de registros es:

Offset	Registro	Acceso	Función
0x00	TX_DONE	RO	Un bit por canal; se pone a 1 al fin de trama.
0x04	TX_DONE_CLEAR	W1C	Escribir 1 borra el bit correspondiente.
0x08	IRQ_ENABLE	RW	Máscara de canales que generan interrupción.
0x0C	IRQ_STATUS	RO	TX_DONE AND IRQ_ENABLE.

Tabla 3.8: Mapa de registros del bloque TX_ISR_EOF_HANDLER (base 0xA0001000).

La interrupción es el OR de los bits habilitados, y la puesta a 1 por fin de trama tiene prioridad sobre el borrado por software, para que no se pierda un evento que llegue justo durante la escritura de *clear*. El driver espera en un semáforo que esta ISR libera, de modo que Transceiver_Send retorna cuando el byte ha salido físicamente, no cuando el DMA lo ha copiado.

Camino de recepción: catorce canales y el TDEST

En recepción cada UART entrega bytes de forma autónoma. El bloque BRIDGE_RX_TOP recoge los catorce flujos y los presenta al MCDMA como un único *stream* etiquetado:

- Una **FIFO por canal** absorbe la ráfaga de bytes que llega mientras el árbitro atiende a otro.
- Un **DEMUX** recorre las catorce FIFO.
- El bloque **DRAIN** vacía la FIFO seleccionada hacia el *stream* de salida, cierra el paquete con TLAST y —esto es lo esencial— fija el TDEST con el número de canal de procedencia.

El MCDMA demultiplexa por TDEST: el paquete etiquetado con el canal N acaba en el descriptor y el *buffer* de DDR del canal N . Sin esa etiqueta el hardware no tiene forma de saber de quién es cada byte, y todo el tráfico se deposita en el canal cero. Es exactamente lo que ocurrió, y se relata en la sección 3.7.4.

El paquete se cierra por dos condiciones: cuando la FIFO del canal se vacía (fin natural de la ráfaga) o cuando se alcanza el tope duro de longitud del generico MAX_PKT, fijado en 256 bytes. Ese segundo criterio no es una optimización: es una **invariante de seguridad** impuesta por el IP, y su razón se explica más abajo.

Mapa de memoria y de interrupciones

Dirección	Bloque	Tamaño
0xA000_0000	AXI_UART_CONFIG — un registro de 32 bits por canal	4 KB
0xA000_1000	TX_ISR_EOF_HANDLER	4 KB
0xA001_0000	Registros de control del <code>axi_mcdma</code>	64 KB

Tabla 3.9: Mapa de memoria de la variante MCDMA.

Las tres líneas de interrupción hacia el PS son `mm2s_introut` (SPI 89, vector 121 en RTEMS), `s2mm_introut` (SPI 90, vector 122) y la del bloque propio de fin de trama (SPI 91, vector 123). Las catorce salidas de interrupción de los canales S2MM se reducen por OR a una sola línea, y la ISR escanea los registros de estado para saber qué canales han completado.

El driver: descriptores scatter-gather

El MCDMA opera siempre en modo *scatter-gather*, de modo que el driver de esta variante no escribe direcciones en registros, sino que **construye descriptores en memoria**. Cada BD ocupa 64 bytes alineados a 64, y el driver mantiene un anillo de un solo descriptor por canal, cuyo puntero al siguiente apunta a sí mismo:

Offset	Campo	Contenido
+0x00	NDESC	Dirección del siguiente descriptor (aquí, él mismo).
+0x08	BUFA	Dirección del <i>buffer</i> de datos.
+0x14	CTRL	SOF (bit 31), EOF (bit 30), longitud (bits 25:0).
+0x18	STS	Recepción: COMPLETE (bit 31) y bytes realmente escritos.
+0x1C	SIDEBAND	Transmisión: COMPLETE (bit 31).

Tabla 3.10: Estructura del descriptor de buffer (BD) del MCDMA.

Rearmar un canal de recepción consiste en borrar el campo de estado, volcar la línea de caché del descriptor y volver a escribir el puntero de cola: el motor lo relee y reanuda. La recepción entrega el paquete cuando el puente cierra el **TLAST**, no byte a byte, que es justamente el origen de la reducción de interrupciones.

3.7.4. Verificación y depuración iterativa del driver

La variante MCDMA no funcionó a la primera, y el driver descrito en las páginas anteriores no es un diseño escrito de una sentada, sino el resultado de **varias iteraciones** de una metodología de verificación que se aplicó de forma sistemática a lo largo de todo el trabajo:

1. **Verificación aislada en simulación.** Ningún bloque de la PL se integraba en el diseño sin pasar antes su propio *testbench* en Vivado. El puente y el módulo de *loopback* tienen bancos de prueba dedicados que ejercitan el protocolo AXI-Stream y la topología de buses, respectivamente.

2. **Integración incremental.** Cada bloque validado se incorporaba al diseño de bloques y se comprobaba de nuevo en placa antes de añadir el siguiente, de modo que cualquier regresión quedara acotada a la última pieza introducida.
3. **Batería de pruebas automáticas en placa.** La aplicación de *benchmark* (sección 3.7.5) actúa como prueba de regresión del sistema completo: una campaña de treinta y seis comprobaciones cuyo resultado —cuántas pasan y cuáles fallan— dirigía la siguiente iteración.

Las primeras iteraciones del driver resolvieron cuestiones de integración relativamente directas: una versión inicial por *polling*, que se sustituyó por un modelo íntegramente dirigido por interrupción una vez el IP se regeneró con la interrupción por canal habilitada (`c.enable_multi_intr`); y la propagación de la señal TDEST a lo largo de toda la jerarquía VHDL, sin la cual el MCDMA no puede demultiplexar y todo el tráfico de recepción acaba atribuido al canal cero.

El verdadero cuello de botella del trabajo fue, sin embargo, la **depuración del bus AXI-Stream contra el motor de la DMA**. Los fallos de este bus no se manifiestan como un error: no hay bit de estado, ni interrupción, ni descriptor marcado; el motor simplemente deja de aceptar datos y el canal enmudece a mitad de una transferencia. Esa ausencia total de observabilidad desde el software hace que depurar sobre la placa sea inviable, y obligó a **reproducir el escenario en simulación**. Fueron necesarias numerosas iteraciones de *testbench* —inyectando contrapresión, paquetes de distinto tamaño y ráfagas solapadas— hasta poder observar el *handshake* congelándose ciclo a ciclo e identificar las dos condiciones que lo provocaban.

De esas simulaciones salieron las dos **invariantes estructurales** que la lógica del puente garantiza hoy por construcción:

- **Tamaño de paquete acotado.** El motor de recepción está configurado en modo *store-and-forward*, de modo que acumula el paquete AXI-Stream completo en un *buffer* interno antes de volcarlo a la DDR; un paquete mayor que ese *buffer* lo bloquea indefinidamente, a la espera de un TLAST que ya no puede llegar. El bloque DRAIN impone por ello un tope duro `MAX_PKT = 256`, además de cerrar el paquete cuando la FIFO se vacía. El tope por longitud no es redundante: hace que la invariante sea estructural y no dependa de que el drenado gane siempre la carrera a la línea serie.
- **TLAST registrado.** La señal de fin de paquete no puede ser combinacional sobre el estado de «FIFO vacía» mientras hay un dato ofrecido. Si un byte nuevo llegara durante una espera por contrapresión, tumbaría el TLAST ya presentado, reabriendo el paquete y devolviendo el sistema al mismo bloqueo. La señal se congela en cuanto el byte se ofrece.

Un último aprendizaje, de flujo de trabajo más que de diseño, merece mención porque invalidó durante días la interpretación de los experimentos: el bloque que engloba el puente y los transeptores está integrado como *module reference*, y Vivado **cachea su *checkpoint*** de síntesis fuera de contexto. Editar el VHDL y relanzar la implementación no cambia el *bitstream*: el flujo termina sin aviso, cierra *timing* y escribe un `.bit` con el RTL antiguo. La comprobación quedó incorporada al procedimiento —verificar que el *hash* del `BOOT.bin` cambia antes de dar por bueno un *bitstream* nuevo—, y es un buen recordatorio de que una metodología de pruebas solo es fiable si se valida también que lo que se está probando

es realmente lo que se ha escrito.

Con estas correcciones el driver quedó estable: la campaña completa pasó de **0 de 36 a 36 de 36** comprobaciones, la prueba de *throughput* entrega los 5120 bytes de las cinco repeticiones sin pérdidas y el barrido de velocidad supera los siete puntos, de 115 200 a 4 Mbaudios.

3.7.5. Banco de pruebas: loopback en la PL

Comparar tres arquitecturas exige medirlas en igualdad de condiciones. El problema práctico es que una medida sobre la PCB real depende del cableado, de los jumpers de configuración de bus y de la integridad física de las líneas, factores que varían entre montajes y contaminan la comparación del transporte, que es lo que se quiere aislar.

La solución fue **cerrar los buses dentro de la propia FPGA**. Un módulo de *loopback* insertado en la PL reproduce la topología de buses que la PCB implementa en cobre:

Bus	Maestro	Esclavos
A (multipunto)	CH0	CH1–CH6
B	CH7	CH8–CH10
C	CH11	CH12–CH13

Tabla 3.11: Topología de buses reproducida por el módulo de *loopback* de la PL.

La emulación es fiel al comportamiento eléctrico de un bus RS-485: en reposo la línea está a nivel alto y los transmisores hacen *wired-AND*, de modo que la señal de recepción de un canal es el AND de las transmisiones de **los demás** canales de su bus. En particular, **un canal no se oye a sí mismo**, igual que en la PCB, donde el receptor se inhibe mientras el control de dirección está activo. Este detalle no es cosmético: si se realimentara el eco, cada byte transmitido se contaría dos veces en los contadores de recepción y la prueba de desbordamiento mediría el doble de pérdidas. Los tres buses están además aislados entre sí, para que la prueba de transmisión concurrente mida solapamiento real y no diafonía.

El módulo se verifica con su propio *testbench* (dieciséis comprobaciones) y se inserta en cada una de las tres variantes mediante un *script* TCL que desconecta la red que unía cada pin físico de recepción con su transceptor y la alimenta desde el *loopback*. Las líneas de transmisión siguen llegando a los pines, de modo que los *constraints* de la placa siguen siendo válidos y no hubo que modificar ni una asignación de pines.

Sobre este banco se ejecuta `bench_main.c`, el mismo programa en las tres variantes, con ocho pruebas: autodiagnóstico del lazo (T0), latencia de un frame (T1), *throughput* de un canal (T2), transmisión concurrente de tres maestros (T3), recepción multipunto de seis esclavos (T4), desbordamiento del *buffer* de software (T5), barrido de velocidad de 115 200 a 4 Mbaudios (T6) y recuperación ante frames corruptos (T7). El cronómetro para siempre **en el receptor**, nunca al retornar de la función de envío: hacerlo al retornar daría cifras falsas, porque el envío es bloqueante en la variante MCDMA y no bloqueante en las otras dos.

Conviene delimitar qué mide este banco y qué no. Con el lazo dentro de la FPGA, la señal no atraviesa los transceptores RS-485 ni el cableado, de modo que lo que se mide

es exactamente el **coste del transporte PS–PL**, que es el objeto de la comparativa. Lo que **no** se mide es la integridad física del bus —reflexiones, terminación, tiempos de vuelta del control de dirección, ruido—; para eso hace falta la PCB. En particular, el barrido de velocidad da aquí mejores cifras de las que dará en cobre.

4. Resultados

En este capítulo se recogen los resultados de la validación hardware del sistema completo, organizados en tres bloques según la placa validada: la sección 4.1 cubre las pruebas realizadas con la placa CDHS, la sección 4.2 las realizadas con la placa AOCS, y la sección 4.3 recoge la validación de la placa de comunicación serie —la comparativa de las tres arquitecturas de transporte PS-PL y la caracterización eléctrica de la placa ya fabricada, sobre sus catorce transceptores—. Las tres placas se conectan a la ZCU102 mediante FMC.

4.1. Validación CDHS

4.1.1. Montaje

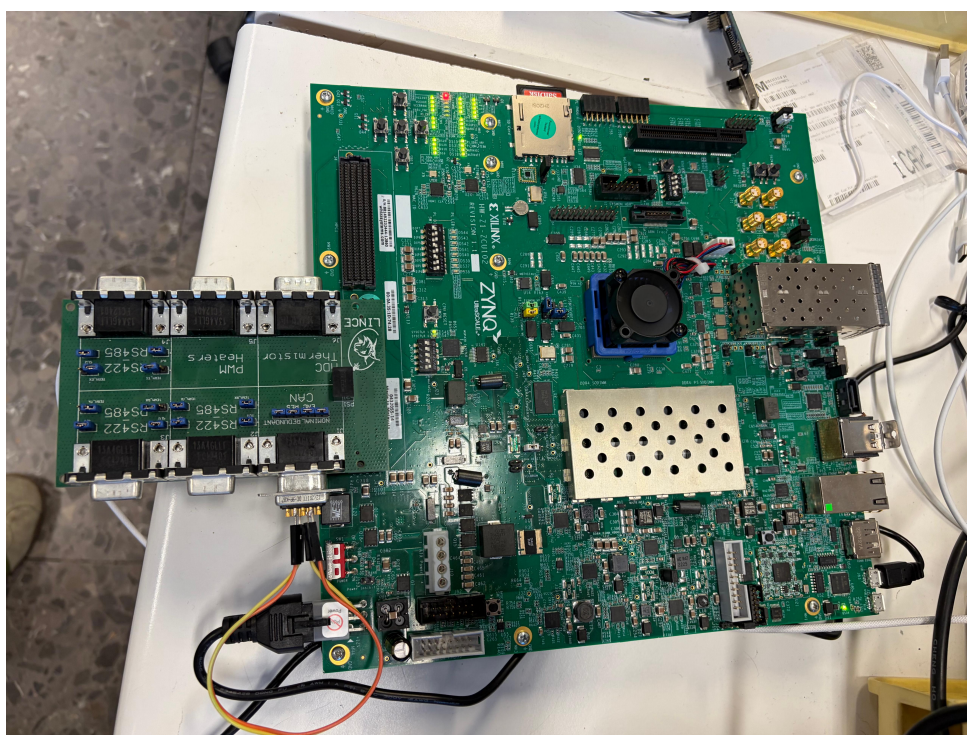


Figura 4.1: Sistema completo montado: ZCU102 con la placa CDHS conectada al FMC.

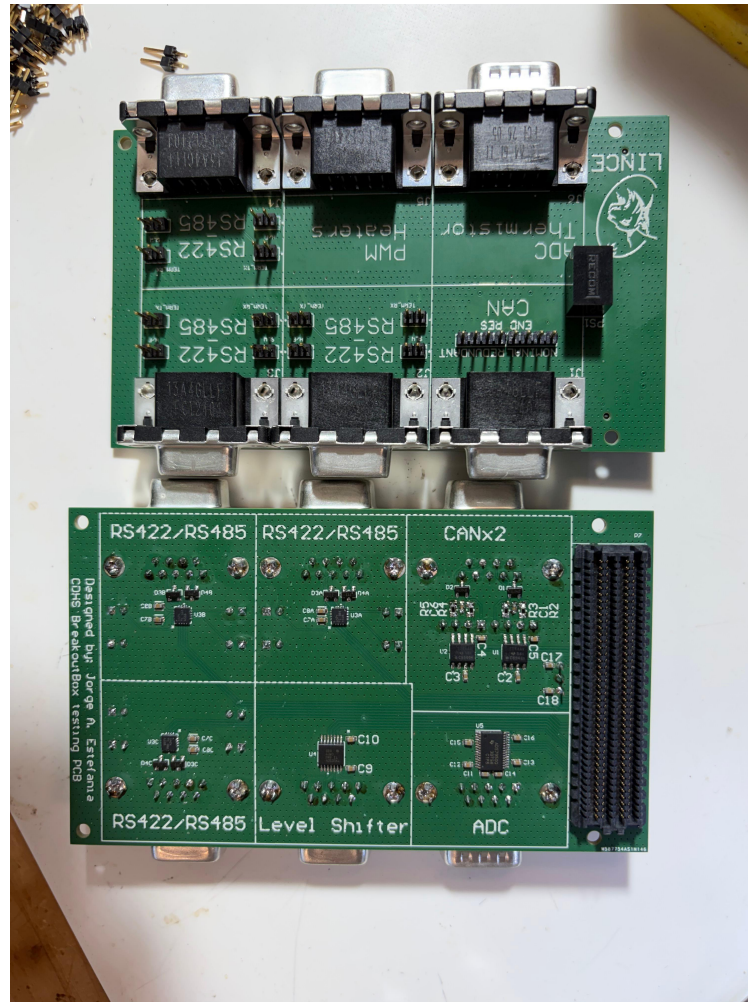


Figura 4.2: Placa CDHS con componentes montados (ambas caras).

4.1.2. Comunicaciones serie RS422/RS485

Las pruebas de comunicaciones serie se realizaron con la aplicación de testing en RTEMS, que expone una consola interactiva por USB donde el usuario puede enviar mensajes a cualquier transceptor con el formato <ID><MENSAJE> o a todos simultáneamente con ALL <MENSAJE>. Se construyeron arneses de *loopback* para interconectar los canales:

- **RS422:** arnés a 4 hilos cruzando TX_P/N de un canal con RX_P/N del opuesto.
- **RS485:** arnés a 2 hilos conectando A+ y B- de un canal con A+ y B- del otro.

Al arrancar, el driver descubrió automáticamente los 3 transceptores instanciados en el *bitstream* e imprimió sus direcciones base (0xA0000000, 0xA0001000, 0xA0002000), confirmando el mecanismo de descubrimiento dinámico de hardware. A continuación, la consola reportó UART 00 [OK], UART 01 [OK], UART 02 [OK].

Las pruebas de *loopback* mostraron que los mensajes enviados por UART 0 llegaban a UART 2 y viceversa, y los enviados por UART 1 llegaban correctamente. En las primeras recepciones de esta captura aparecen caracteres corruptos (?), correspondientes a una iteración previa del diseño antes de aplicar la corrección de robustez del receptor descrita en el Capítulo 3 (verificación de *start-bit* a mitad de período). Una vez implementada esa

mejora en la FSM del receptor, los caracteres corruptos desaparecieron por completo en las pruebas posteriores.

Programación 4.1: Salida de terminal — prueba RS422/RS485 con la placa CDHS (3 transceptores)

```

1 [TRANSCIEVER DEBUG] Detectados: 3 | Base: 0xA0000000 | INT: 0
  xA0003000
2 UART 00 [OK]  UART 01 [OK]  UART 02 [OK]
3 CMD> 0 HOLA MUNDO
4 Tx -> UART 0: OK
5 [RX UART 02]: HOLA MUNDO
6 CMD> 2 HOLA MUNDO
7 Tx -> UART 2: OK
8 [RX UART 00]: HOLA MUNDO

```

[Figura pendiente: captura completa del terminal `cdhs_testing_rs` disponible en `tfm/00_docs/logs_terminal/`]

Figura 4.3: Captura completa del terminal de pruebas RS en la placa CDHS.

4.1.3. Bus CAN

Las pruebas del bus CAN se realizaron con la aplicación de test desarrollada por Diego Ramos (compañero del laboratorio B105 en el proyecto LINCE), que configura los canales CAN y ejecuta una batería de tests de enlace.

La suite de tests ejecutó 26 pruebas cubriendo inicialización, *loopback* interno, transferencia física CAN0↔CAN1, filtrado de identificadores estándar y extendido, manejo de tramas RTR y pruebas de interrupción por hardware. El resultado fue **26/26 tests PASS, 0 FAILED**:

Programación 4.2: Resumen de resultados del test CAN (placa CDHS)

```

1 [PASS] CAN0 initialization (Fast Mode)
2 [PASS] CAN1 initialization (Slow Mode)
3 [PASS] Loopback RX ID matches TX ID
4 [PASS] Loopback RX Data matches TX Data
5 [PASS] CAN0 received correct ID from CAN1      (fisico)
6 [PASS] CAN0 received correct Data from CAN1    (fisico)
7 [PASS] Standard ID: Exact Match Accepted (0x1A4)
8 [PASS] Extended ID: Exact Match Accepted (0x12345678)
9 [PASS] RTR Filter: Accepted matching Remote Request
10 [PASS] Tx0k fired successfully
11 [PASS] Rx0k fired and FIFO was drained successfully (No Storm)
12 ... (26/26 PASS, 0 FAILED)

```

Un resultado relevante adicional fue la verificación del efecto de las resistencias de terminación sobre la integridad de la señal CAN. Sin terminación, la señal presentaba reflexiones visibles que degradaban los flancos; con ambas resistencias de 60 Ω conectadas, la forma de onda resultó limpia y bien definida.

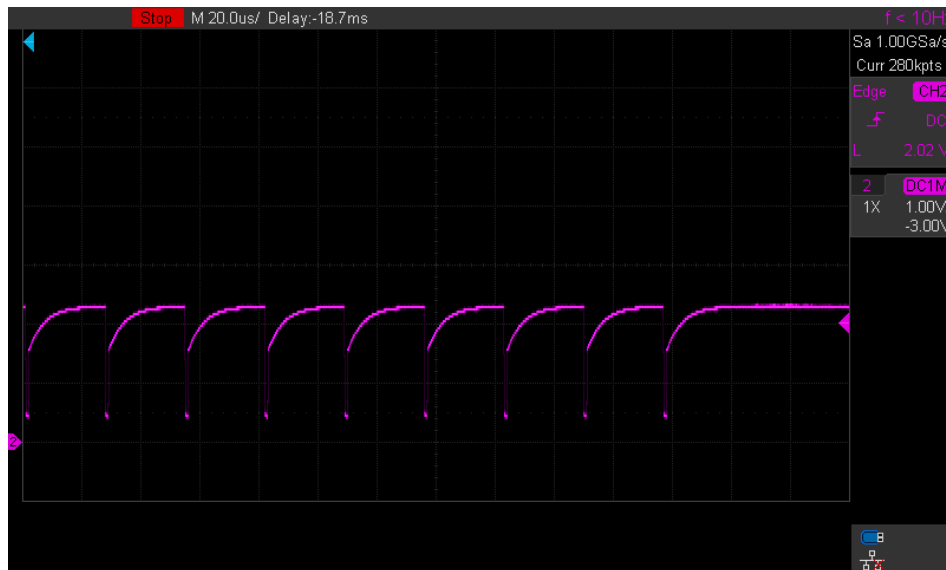


Figura 4.4: Señal diferencial CAN sin resistencias de terminación.

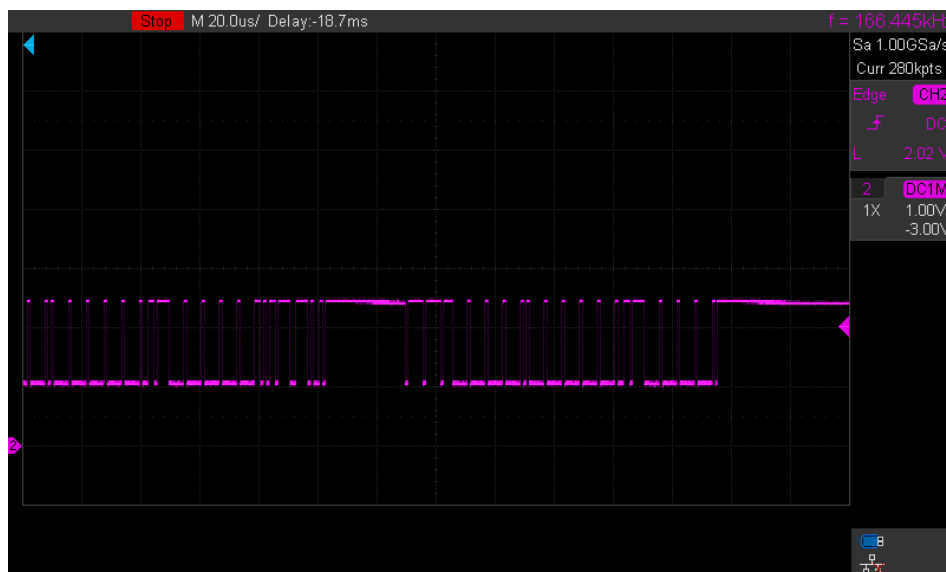


Figura 4.5: Señal diferencial CAN con resistencias de terminación ($60\ \Omega + 60\ \Omega$).

4.1.4. ADC SPI (termistores)

La aplicación de lectura del ADC ADS7950 muestreó los cuatro canales a 500 ms de intervalo e imprimió los valores digitales por el terminal. Para verificar la linealidad de la cadena de adquisición se aplicaron tensiones de referencia conocidas a las entradas analógicas:

- Tensión de entrada 0 V $\rightarrow$ valor digital próximo a 0 (fondo de escala inferior).
- Tensión de entrada 3,3 V $\rightarrow$ valor digital próximo a 4095 (fondo de escala superior, 12 bits).

Los resultados confirmaron el rango de conversión esperado, validando el driver SPI de bajo nivel y la cadena analógica de la placa CDHS. La captura registra el canal CH2 siendo sometido a un barrido de tensión con la fuente de laboratorio:

Programación 4.3: Lectura del ADC ADS7950 durante barrido de tensión en CH2

1	ADS7950: CH0= 244	CH1= 43	CH2= 0	CH3= 19	<- tension en
	CH2 = 0 V				
2	ADS7950: CH0= 245	CH1= 43	CH2=1577	CH3= 19	<- subida
3	ADS7950: CH0= 245	CH1= 43	CH2=3419	CH3= 21	<- cerca del
	maximo				
4	ADS7950: CH0= 245	CH1= 43	CH2=3565	CH3= 21	<- ~2.9 V
	aplicados				
5	ADS7950: CH0= 245	CH1= 43	CH2=2436	CH3= 22	<- bajada
6	ADS7950: CH0= 246	CH1= 43	CH2= 102	CH3= 22	<- de vuelta
	a 0 V				

Los canales CH0 (≈ 245), CH1 (≈ 43) y CH3 (≈ 19) mostraron valores estables bajos durante todo el ensayo, correspondientes a las entradas no excitadas.

[Figura pendiente: captura completa del terminal `cdhs_testing_adc.txt` disponible en `tfm/00_docs/logs_terminal/`]

Figura 4.6: Captura completa del terminal de lectura ADC (placa CDHS).

4.1.5. PWM (calentadores)

Las señales PWM generadas por el bloque `PWMx4_auto_test` se midieron con el osciloscopio sobre el conector J5 de la placa CDHS, verificando los cuatro canales: 10 kHz, 5 kHz, 1 kHz y 100 Hz, todos con *duty cycle* del 50 %.

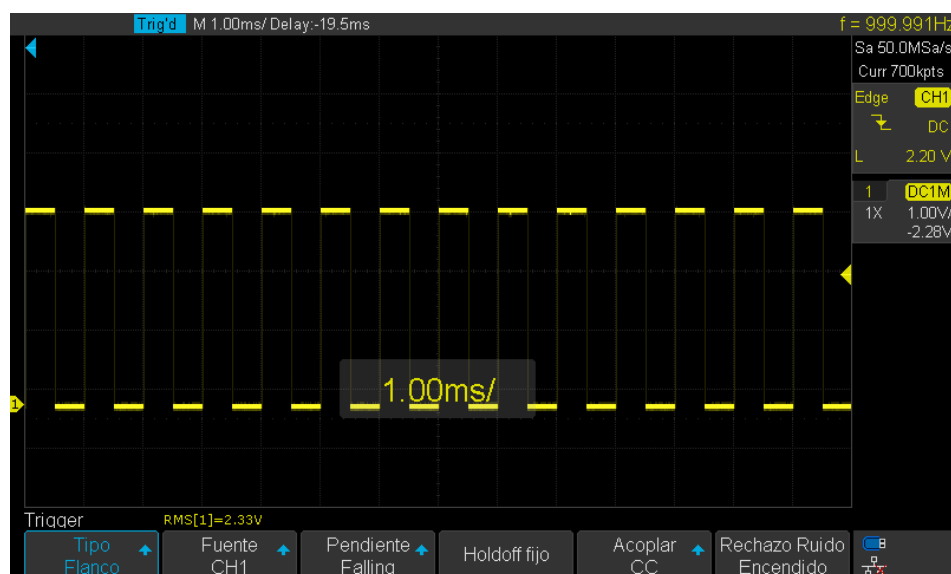


Figura 4.7: Señal PWM medida en el conector J5 de la placa CDHS: canal de 1 kHz con *duty cycle* del 50 %.

4.2. Validación AOCS

4.2.1. Montaje

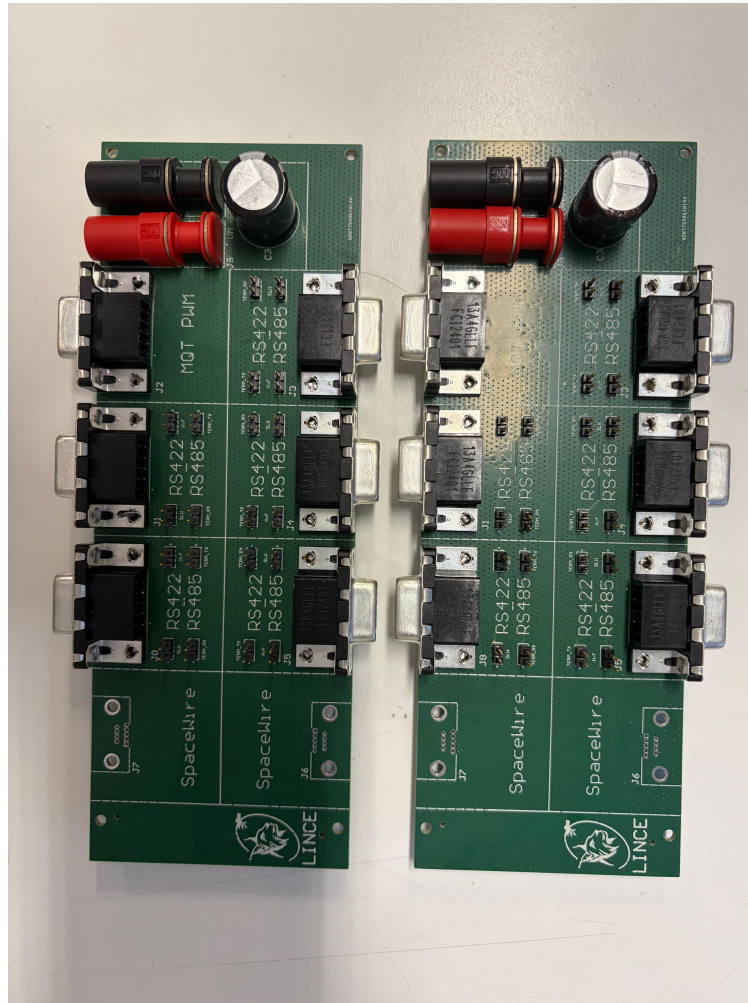


Figura 4.8: Placas AOCS con componentes montados (vista superior de las dos unidades fabricadas).

4.2.2. Comunicaciones serie RS422/RS485

El procedimiento de prueba —consola interactiva por USB y arneses de *loopback* RS422/RS485— es el mismo descrito para la placa CDHS (sección 4.1.2).

Con el *bitstream* de 5 transceptores para la placa AOCS, el descubrimiento detectó correctamente 5 instancias (0xA0000000–0xA0004000, INTC en 0xA0005000). Las pruebas de *loopback* entre distintos pares de canales —UART 0↔4, UART 0↔3, UART 0↔2, UART 0↔1— resultaron todas correctas sin errores de *framing*.

Programación 4.4: Salida de terminal — prueba RS422/RS485 con la placa AOCS (5 transceptores)

```

1 [TRANSCIEVER DEBUG] Detectados: 5 | Base: 0xA0000000 | INT: 0
  xA0005000
2 CMD> 0 HOLA -> [RX UART 04]: HOLA
3 CMD> 4 HOLA -> [RX UART 00]: HOLA

```

```
4 CMD> 0 HOLA -> [RX UART 03]: HOLA
5 CMD> 0 HOLA -> [RX UART 01]: HOLA
```

[Figura pendiente: captura completa del terminal `aocs_testing_rs` disponible en `tfm/00_docs/logs_terminal/`]

Figura 4.9: Captura completa del terminal de pruebas RS en la placa AOCS.

4.2.3. PWM (puentes en H)

Para la placa AOCS, el bloque VHDL de control de motores generó señales PWM alternando la dirección de giro cada pocos segundos (línea 1 activa con línea 2 a 0, y a continuación línea 1 a 0 con línea 2 activa), permitiendo verificar el control de puentes en H con una fuente de alimentación externa conectada a los bornes banana.

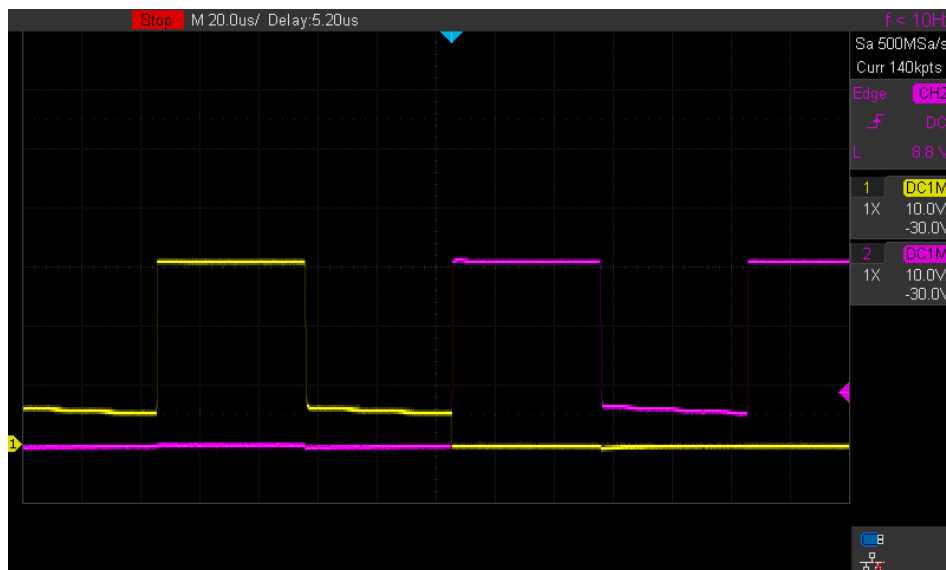


Figura 4.10: Señales PWM de control de motor en la placa AOCS (par complementario para puente en H).

4.3. Validación de la placa de comunicación serie

4.3.1. Prueba previa: 14 transceptores simultáneos

Para verificar el correcto funcionamiento del driver con un número elevado de instancias, se cargó un *bitstream* con 14 transceptores instanciados (canales 0 a 13), conectando los canales adicionales a los pines externos del conector J3 de la ZCU102. El sistema arrancó y operó correctamente con todos los canales, confirmando que la arquitectura de ISR maestra con tareas *worker* individuales escala sin problemas hasta el número máximo de instancias soportado. La prueba con los 14 transceptores sobre la placa de comunicación serie ya fabricada se aborda en la sección 4.3.3.

4.3.2. Comparativa de las arquitecturas de transporte PS–PL

Esta sección recoge la medida de las tres arquitecturas descritas en la sección 3.7. Las tres se ejecutaron sobre la misma ZCU102, con el mismo transceptor serie de catorce canales, el mismo programa de pruebas (`bench_main.c`) y los buses cerrados por el módulo de *loopback* de la PL descrito en la sección 3.7.5. Lo único que difiere entre las tres corridas es el transporte y su driver, que es precisamente lo que se quiere comparar.

Una salvedad previa. En la variante DMA14 **no se consiguió hacer funcionar el módulo de *loopback***: su autodiagnóstico detectó que el lazo de recepción estaba abierto y, en consecuencia, no llegó ningún byte. Como se explica en la sección 3.7, se decidió no invertir más tiempo en depurarlo, porque el coste en lógica y el límite de líneas de interrupción ya descartaban esta arquitectura como solución final. Todo lo que depende de recibir —latencia, *throughput*, multipunto, desbordamiento, recuperación— sale a cero en esa variante, y ese cero **no mide el transporte**: mide la ausencia de lazo. Se hace constar con un guion en las tablas. Lo que sí es plenamente válido de esa corrida, y lo que sostiene el argumento sobre esta variante, es el coste de hardware, la huella de memoria y el coste en interrupciones de la transmisión.

Coste de hardware

Las tres arquitecturas están implementadas sobre el mismo dispositivo (XCZU9EG, 274 080 LUT, 548 160 registros y 912 bloques de RAM), de modo que las cifras salen directamente de los informes de utilización de Vivado y son comparables sin normalización alguna.

Variante	LUT	%	Registros	BRAM	Potencia	WNS	LUT/canal
GPIO	6 434	2,35 %	7 726	7	3,52 W	4,69 ns	460
DMA14	40 217	14,67 %	53 943	35	3,86 W	4,05 ns	2 873
MCDMA	14 191	5,18 %	16 438	18	3,62 W	5,14 ns	1 014

Tabla 4.1: Ocupación de la lógica programable de las tres variantes sobre el XCZU9EG.

La figura 4.11 representa las tres magnitudes normalizadas como porcentaje de los recursos disponibles en el dispositivo, que es lo que permite dibujarlas sobre un mismo eje y compararlas de un vistazo.

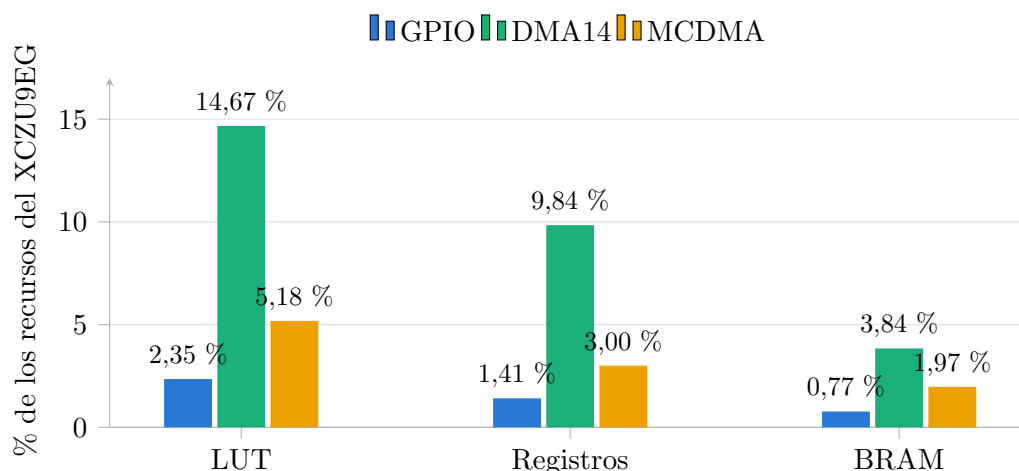


Figura 4.11: Ocupación de la FPGA por variante, en porcentaje de los recursos del dispositivo. Menos es mejor. El banco de catorce DMA triplica largamente el consumo de la solución multicanal para el mismo número de canales.

La cifra que resume la comparación es la última columna de la tabla. El banco de catorce AXI DMA cuesta **2,8 veces más lógica que el MCDMA** para exactamente el mismo número de canales, porque replica catorce motores completos en lugar de compartir uno multicanal. La variante GPIO es, con diferencia, la más pequeña —y así debe ser: apenas contiene los transceptores y un puñado de registros—, pero ese ahorro de lógica no es gratuito: lo paga la CPU, como se ve más abajo.

El margen de *timing* (WNS) es positivo y holgado en las tres, de modo que ninguna cierra al límite y las diferencias de rendimiento no son atribuibles a la implementación física.

Coste de software

Variante	Código	RAM del driver	Líneas IRQ	Tareas	Semáforos
GPIO	3 099 B	114 688 B	1	14	28
DMA14	3 036 B	115 584 B	2	1	15
MCDMA	3 884 B	122 434 B	3	1	17

Tabla 4.2: Huella de software del driver de cada variante (tamaño de `.text` y recursos RTEMS reservados en ejecución).

La memoria total reservada es prácticamente la misma en las tres —en torno a 112–120 KB, dominada por los *buffers* circulares de 4 KB por canal—, aunque su origen difiere: la variante GPIO los pide con `malloc` y las de DMA los reservan estáticamente, porque el motor necesita direcciones fijas y alineadas.

La diferencia significativa está en las **primitivas de RTEMS**. La variante GPIO necesita **una tarea de recepción por canal** —catorce tareas y veintiocho semáforos— porque cada canal drena su propio FIFO hardware de forma independiente. Las dos variantes con DMA se apañan con **una sola tarea de despacho**, ya que el motor entrega paquetes completos y la ISR solo tiene que señalar quién ha terminado. Trece tareas menos es trece cambios de contexto menos, y trece pilas menos.

Coste en interrupciones

Esta es la métrica que explica todas las demás. Cuenta cuántas veces se expropia la CPU por cada kilobyte movido, acumulado sobre la sesión completa de pruebas.

Variante	IRQ/KB	IRQ TX	IRQ RX	Bytes TX	Bytes RX
GPIO	1 250	22 161	129 382	22 161	101 890
DMA14	3 *	85	—	23 132	—
MCDMA	3	194	192	23 132	101 472

* Solo transmisión: sin lazo de recepción, la cifra no es comparable.

Tabla 4.3: Interrupciones por kilobyte transferido.

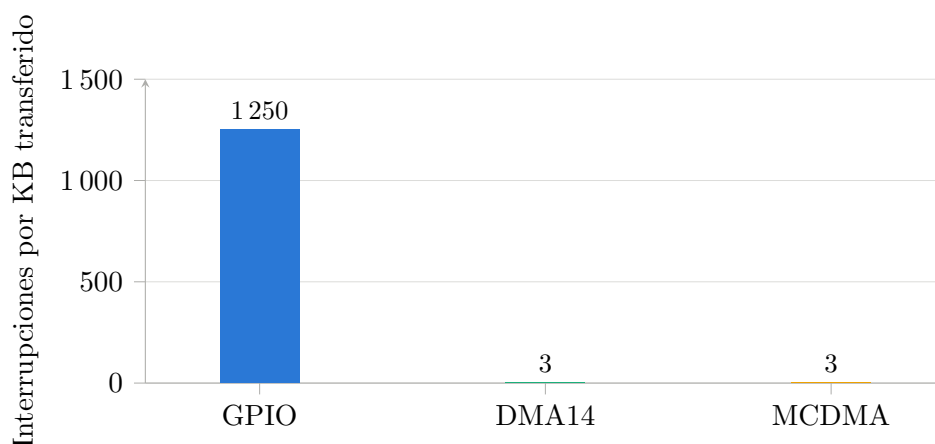


Figura 4.12: Interrupciones por kilobyte transferido. Menos es mejor: cada interrupción es una expropiación de la CPU. Las barras de DMA14 y MCDMA son invisibles a esta escala, y eso es precisamente el resultado. La cifra de DMA14 recoge solo la transmisión.

Los números hablan por sí solos. En la variante GPIO, el número de interrupciones de transmisión **coincide exactamente con el número de bytes transmitidos** (22 161 y 22 161): es la confirmación empírica de que el procesador toca cada byte. En recepción ocurre lo mismo, con un sobrecoste añadido por los eventos de estado. El resultado son **1250 interrupciones por kilobyte**.

El MCDMA mueve la misma cantidad de datos con **194 interrupciones de transmisión y 192 de recepción**: tres por kilobyte, unas **cuatrocientas veces menos**. La CPU pasa de estar atendiendo el bus a estar disponible para la aplicación.

Latencia

Tiempo desde justo antes de la llamada de envío hasta que el receptor tiene el frame de 11 bytes completo, sobre 50 muestras.

Variante	Media (μs)	p50	p95	Máx.	Timeouts
GPIO	4 606	4 608	4 611	4 611	0
DMA14	—	—	—	—	50
MCDMA	1 530	1 529	1 535	1 535	0

Tabla 4.4: Latencia de entrega de un frame de 11 bytes, del envío en el maestro a la recepción completa en el esclavo (115 200 8N1).

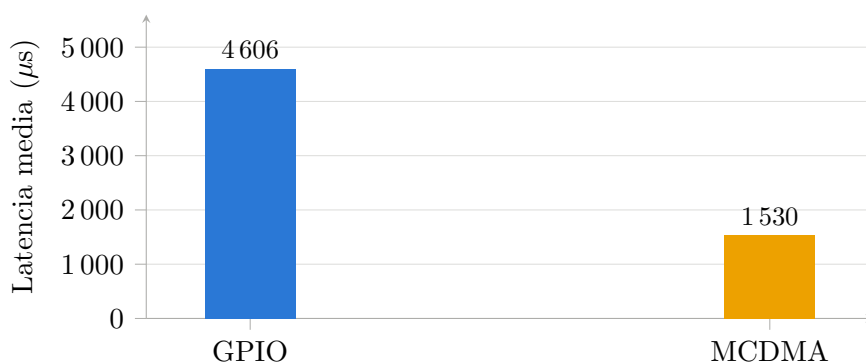


Figura 4.13: Latencia media de entrega de un frame de 11 bytes. Menos es mejor. Se excluye DMA14 por carecer de lazo de recepción.

El MCDMA es **tres veces más rápido** que el GPIO, y con una dispersión muy baja: la diferencia entre la mediana y el máximo es de 6 μs sobre 50 muestras, lo que indica un comportamiento notablemente determinista —una propiedad especialmente valiosa en un sistema de tiempo real embarcado.

Throughput

A 115 200 baudios en formato 8N1, cada byte ocupa 10 bits de línea, de modo que el techo físico de un canal es de **11 520 B/s**. Esa es la referencia contra la que hay que leer la primera columna.

Variante	1 canal	3 maestros	6 receptores	Reps. sin pérdidas
GPIO	3 471 (30 %)	10 411	20 838	5 / 5
DMA14	—	—	—	—
MCDMA	11 461 (99 %)	34 303	68 504	5 / 5

Tabla 4.5: *Throughput* en B/s. El porcentaje es sobre el techo físico de la línea (11 520 B/s).

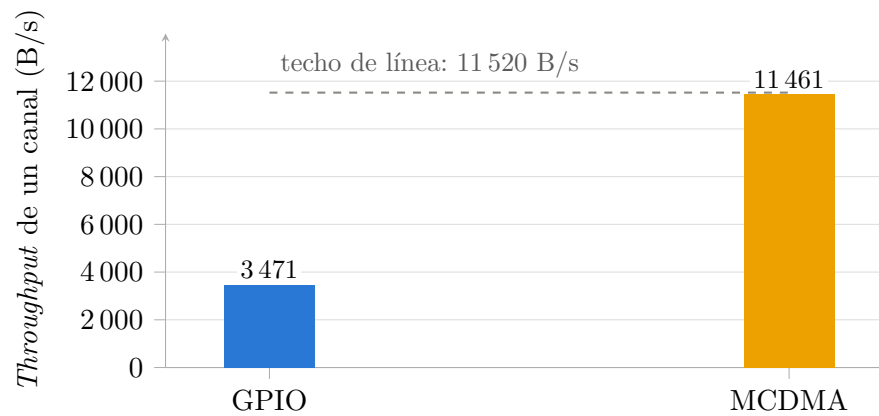


Figura 4.14: *Throughput* de un canal frente al techo físico de la línea a 115 200 8N1. Más es mejor. El MCDMA lo alcanza al 99 %; la variante GPIO se queda en el 30 %.

El MCDMA alcanza el **99 % del techo físico de la línea**: el transporte deja de ser el cuello de botella y el único límite pasa a ser la propia línea serie, que es exactamente el objetivo de diseño. La variante GPIO se queda en el 30 %: dos de cada tres bytes que la línea podría llevar se pierden atendiendo interrupciones.

Con carga concurrente la diferencia se amplía en lugar de reducirse: con seis receptores simultáneos en el bus multipunto, el MCDMA sostiene **68 504 B/s** frente a los 20 838 del GPIO, y lo hace sin perder un solo byte.

Barrido de velocidad

La prueba más reveladora de todas. Se reprograman los catorce transceptores a cada velocidad y se mide un bloque de 1 KB en cada punto.

Variante	115200	230400	460800	921600	1M	2M	4M
GPIO	3 471	4 087	4 485	4 714	4 733	4 848	4 907
MCDMA	11 458	22 789	45 104	88 298	96 186	183 053	344 781

Tabla 4.6: *Throughput* (B/s) frente a la velocidad de línea (baudios). La variante DMA14 se omite por carecer de lazo de recepción.

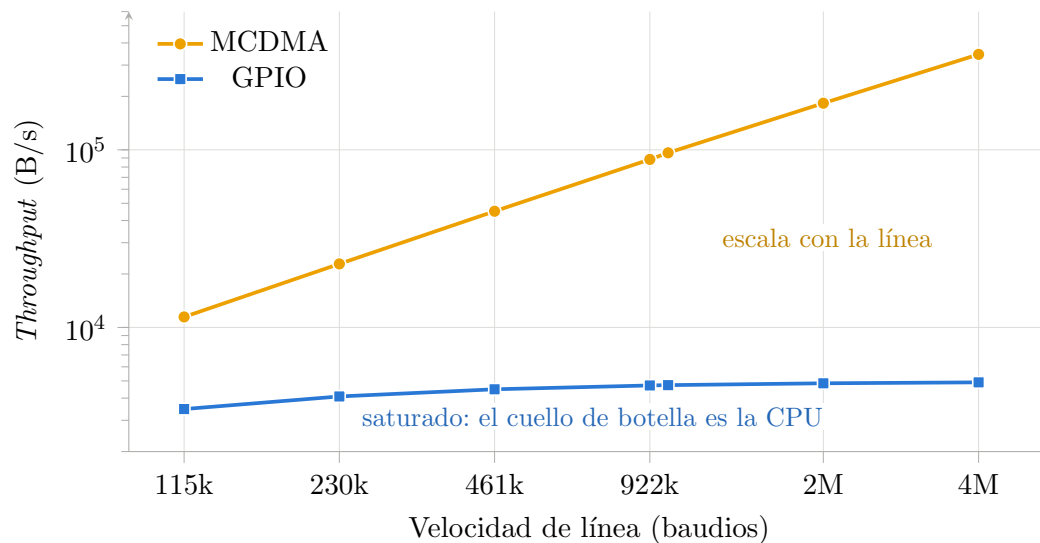


Figura 4.15: *Throughput* frente a la velocidad de línea, en escala logarítmica en ambos ejes. El MCDMA crece de forma proporcional a la velocidad del enlace; la variante GPIO es una recta plana, porque su límite no está en la línea sino en el coste de una interrupción por byte. En el punto de 4 Mbaudios la diferencia es de 70 veces.

Aquí se ve, sin necesidad de más argumentos, la naturaleza del límite de cada arquitectura:

- El **GPIO se satura**. Multiplicar la velocidad de línea por 35 (de 115 200 a 4 Mbaudios) mejora el *throughput* en un mísero 41 %, de 3471 a 4907 B/s. La curva es plana porque **el cuello de botella no es la línea, es la CPU**: el coste de una interrupción por byte no depende de lo rápido que vaya el bus. Por rápido que se haga el hardware serie, el sistema no aprovechará esa velocidad.
- El **MCDMA escala linealmente**. El *throughput* crece de forma prácticamente proporcional a la velocidad de línea, hasta **344 781 B/s** a 4 Mbaudios: **70 veces** lo que consigue el GPIO en el mismo punto. La curva sigue a la línea porque el transporte ya no interviene.

En los siete puntos, y en ambas variantes, el número de errores de transporte es **cero**: la saturación del GPIO no produce corrupción, solo pérdida de rendimiento.

Robustez

Dos pruebas adicionales verifican que la ganancia de rendimiento no se paga con fragilidad: el envío de tres frames corruptos seguidos de uno válido (T7) y el envío de 8 KB sin que la aplicación vacíe un *ring* de 4 KB (T5), diseñado explícitamente para desbordar.

Variante	Frames corruptos	Se recupera	Bytes aceptados	Bytes descartados
GPIO	3 / 3	sí	24 576	16 246
DMA14	3 / 3	—	—	—
MCDMA	3 / 3	sí	24 576	24 576

Tabla 4.7: Rechazo de frames corruptos (T7) y comportamiento en desbordamiento del *ring* de software (T5).

Ambas variantes rechazan los tres frames corruptos y siguen aceptando el frame válido posterior: el error no deja el driver en un estado inconsistente.

La columna de bytes descartados merece una aclaración, porque se presta a leerse al revés. **No son errores de transporte** —esos son cero en las tres variantes—: cuentan un byte cada vez que un dato llega correctamente pero **no cabe en el *ring* de software porque nadie lo ha leído**. La mayor parte procede de la prueba T5, cuyo propósito es justamente ese, y el resto de los canales que nadie lee en las pruebas de un solo maestro: al ser un bus multipunto, los seis receptores reciben cada bloque.

Por eso el número del MCDMA es *mayor* que el del GPIO, y eso es una buena noticia y no una mala: en la variante GPIO el propio cuello de botella de interrupciones frena la entrega —el *ring* se llena más despacio y desborda menos—, mientras que el MCDMA entrega a la velocidad de la línea todo lo que ésta produce y, en consecuencia, sobra más en los *buffers* que la aplicación de prueba deliberadamente no vacía. **Un contador de descartes alto es aquí la firma de un transporte sano, no de uno defectuoso**. Conviene recordar que estas cifras corresponden a la campaña final, con las treinta y seis comprobaciones en verde: las corridas anteriores a las correcciones de contrapresión y de tamaño de paquete descritas en la sección 3.7.4 entregaban menos datos a los *rings* y no son comparables.

Discusión: qué arquitectura y por qué

	GPIO	DMA14	MCDMA	
Lógica (LUT)	6 434	40 217	14 191	
IRQ/KB	1 250	3 (solo TX)	3	
Throughput 1 canal	3 471 B/s (30 %)	—	11 461 (99 %)	B/s
Latencia	4 606 μ s	—	1 530 μs	
Escala con el baudrate	no	—	sí	
Líneas IRQ necesarias	1	28 (reducidas a 2)	3	
Complejidad de diseño	baja	media	alta	

Tabla 4.8: Síntesis de la comparativa. En negrita, el mejor valor de cada fila.

La conclusión que sostienen los datos es que **el MCDMA con puente VHDL es la arquitectura adecuada para este sistema**, y que su ventaja no procede de ser «más rápido» en un sentido vago, sino de una causa concreta y medible: **mueve paquetes en lugar de bytes**. Las 1250 interrupciones por kilobyte de la variante GPIO se convierten en 3, y todo lo demás —la latencia tres veces menor, el 99 % del techo de línea, la escalabilidad con la velocidad— es consecuencia de eso.

La variante DMA14 es instructiva precisamente por lo que demuestra que **no** hay que hacer. Fue el primer intento de mejorar la arquitectura frente al AXI GPIO, y resuelve el problema correcto —quitar a la CPU de en medio— pero por el camino equivocado:

replicar catorce motores cuesta 2,8 veces más lógica y, sobre todo, exige veintiocho líneas de interrupción cuando el silicio solo ofrece ocho. La arquitectura no escala, y el límite no es del diseño sino del dispositivo. A ello se suma un argumento de futuro: un 14,7 % del dispositivo es una hipoteca incompatible con la triplicación TMR que exigiría endurecer el diseño frente a la radiación, mientras que el 5,2 % del MCDMA deja ese margen abierto. Es el conjunto de razones que justificó dejar de invertir en esta variante cuando su lazo de recepción se resistió, y el que sostiene el esfuerzo de diseñar el puente VHDL de la variante C: no era una optimización, era la única vía.

En cuanto al GPIO, sigue siendo una opción perfectamente razonable **para pocos canales a baja velocidad**: es cuatro veces más pequeño en lógica, no necesita mapeo de MMU ni gestión de coherencia de caché, y su driver es sensiblemente más simple. Lo que los datos demuestran es que **no sirve para catorce canales**, y que su limitación no se cura subiendo la velocidad de la línea, porque el cuello de botella está en el otro extremo.

4.3.3. Caracterización eléctrica de la placa de comunicación serie

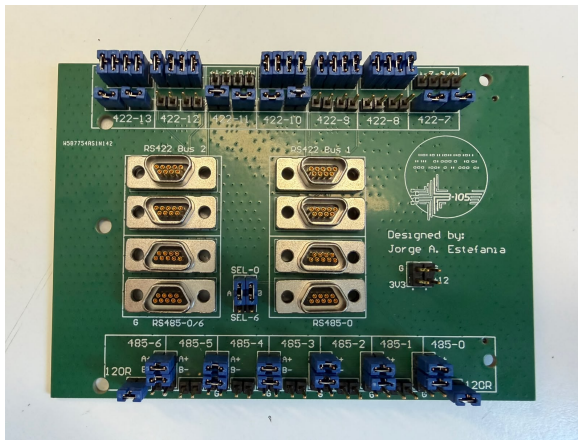
Esta sección caracteriza la **placa de comunicación serie**. Las señales medidas salen por los pines del MPSoC, atraviesan los catorce transceptores RS-485/RS-422, recorren el cobre y regresan.

La campaña se ejecuta sobre el transporte MCDMA, que constituye la solución final, con un *bitstream* **sin el módulo de *loopback***. Esta condición es necesaria: con el *bitstream* del banco de pruebas los pines de recepción quedan desconectados de los transceptores, por lo que la PCB no intervendría en la medida.

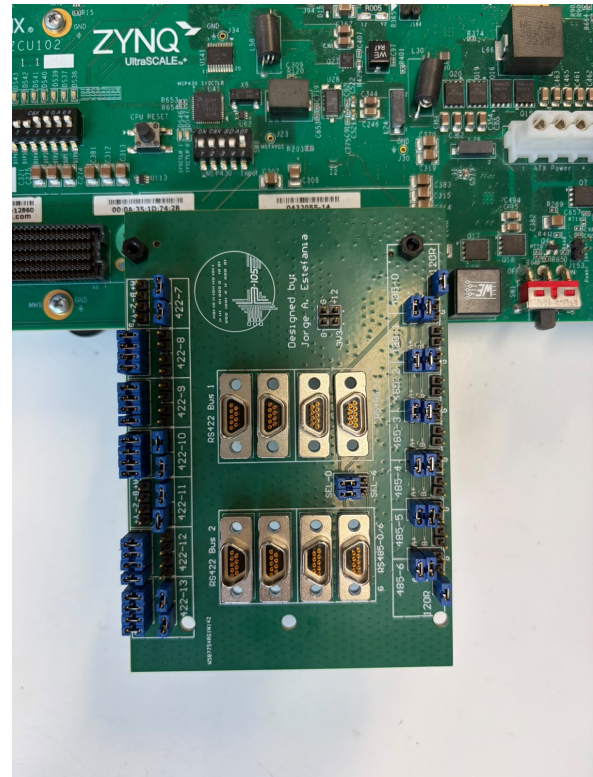
La topología física de la placa, fijada por los *jumpers*, es la siguiente:

Bus	Tipo	Maestro	Esclavos
A	RS-485 multipunto (7 nodos en un bus compartido)	CH0	CH1–CH6
B	RS-422 punto a punto (estrella desde el maestro)	CH7	CH8–CH10
C	RS-422 punto a punto (estrella desde el maestro)	CH11	CH12–CH13

Tabla 4.9: Topología de buses de la PCB de comunicación serie.



(a) Vista superior: conectores D-Sub-9 y *jumppers* de topología.



(b) Placa montada en el conector FMC HPC0 de la ZCU102.

Figura 4.16: PCB de comunicación serie configurada para la campaña de caracterización, con los *jumppers* fijando la topología de la tabla 4.9.

Conviene señalar que **ningún test presupone esta topología**: el test T2 la mide de forma independiente y el resto de pruebas se apoya en lo medido. La diferencia entre el bus A y los buses B y C condiciona la lectura de todos los resultados: en el bus multipunto los seis esclavos se oyen entre sí, mientras que en los buses punto a punto los esclavos solo oyen al maestro.

Batería de pruebas de caracterización

Test	Resultado
T1 Reposo	0 bytes espurios en los catorce canales: la polarización de <i>fail-safe</i> de la placa funciona y las líneas no flotan.
T2 Topología	Los tres buses están puenteados como indican los <i>jumpers</i> y no hay ni un cruce entre buses (0 enlaces no esperados).
T3 Multipunto	6/6 esclavos del bus A oyen íntegro al maestro, hasta 460 kbps.
T4 Velocidad máx.	Bus A: 460 kbps . Buses B y C: 1 Mbps . Por encima, la tasa de error se dispara.
T5 Limitador SLO	Con el bit SLO a 1, los tres buses llegan limpios a 4 Mbps .
T6 Vuelta del bus	<i>Guard time</i> mínimo de 0 μs : el control de dirección del transceptor suelta la línea a tiempo y no hace falta guarda en software.
T7 Diafonía	0 bytes de fuga del bus A hacia los buses B y C.
T8 Colisión	La trama sale corrupta —como debe— y el bus se recupera : el intercambio siguiente es limpio.
T9 Estabilidad	10 s de tráfico continuo: bus A a 460 kbps, BER de 147 ppm; buses B y C a 1 Mbps, BER de 0 .

Tabla 4.10: Resultado de los nueve tests de caracterización de la PCB.

El contador de fallos de transmisión de la campaña completa es **cero**: ningún envío quedó sin transmitir, de modo que todas las cifras corresponden a bytes que efectivamente viajaron por el bus. Esta distinción es relevante para la validez de los datos: distinguir «el transporte no llegó a transmitir» de «el bus corrompió los bytes» es esencial, ya que una primera tanda de medidas que no separaba ambos casos daba los buses B y C como defectuosos de forma incorrecta.

Efecto del limitador de slew rate sobre la velocidad máxima

El pin SLO del transceptor LTC2865 es un habilitador de modo lento **activo a nivel bajo**: con el valor por defecto (0) el limitador de *slew rate* está activo y limita el driver a 250 kbps nominales; con el bit a 1 el limitador se desactiva y el driver alcanza hasta 20 Mbps. Las medidas de esta sección se obtuvieron con el limitador desactivado (SLO = 1). En esa configuración los tres buses operan de forma limpia a **4 Mbps**, cuatro veces la velocidad del modo por defecto. La comparación, repetida en dos tandas independientes, se recoge en la figura 4.17.

Este comportamiento lo fija la hoja de características del transceptor **LTC2865** [20], según la cual el pin SLO es activo a nivel bajo:

«*SLO (Slow Mode Enable): a low input switches the transmitter to the slew rate limited 250 kbps max data rate mode. A high input supports 20 Mbps.*»

El bit de configuración viaja del registro directamente al pin del chip, sin lógica intermedia. Con el valor por defecto (0) el driver queda limitado a 250 kbps nominales, y con el bit a 1 el limitador se desactiva y el driver alcanza hasta 20 Mbps.

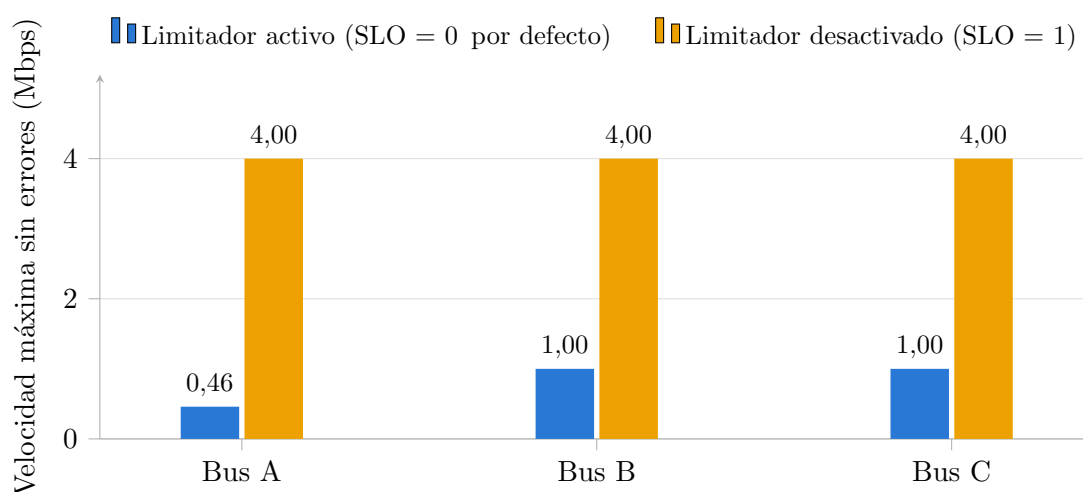


Figura 4.17: Velocidad máxima con tasa de error nula en cada bus de la PCB, según el valor del bit SLO. El pin SLO del LTC2865 es un habilitador de modo lento activo a nivel bajo: con el valor por defecto (0) el limitador de *slew rate* está activo y limita el driver a 250 kbps nominales, con lo que el bus multipunto alcanza 460 kbps. Desactivándolo (1), los tres buses operan de forma limpia a 4 Mbps.

El efecto se observa en las pruebas. La figura 4.18 muestra la tasa de error del bus multipunto, el más exigente de los tres, para ambos valores del bit.

Comportamiento del bus multipunto

El bus A alcanza su límite antes que los otros dos (460 kbps frente a 1 Mbps con el mismo ajuste de SLO). Este comportamiento es coherente con su naturaleza y no constituye un defecto de la placa: siete nodos conectados a un bus compartido cargan más la línea y generan más reflexiones que un enlace punto a punto. La diferencia se mantiene en la parte alta del barrido incluso con el bit SLO a 1, por lo que es independiente del efecto descrito en la sección anterior.

El test T3 lo cuantifica: registra cuántos de los seis esclavos reciben la trama del maestro **íntegra** a cada velocidad.

Velocidad	9 600	115 200	460 800	921 600	1 M
Esclavos íntegros (de 6)	6	6	6	5	0
Esclavos parciales	0	0	0	1	6

Tabla 4.11: Integridad del bus multipunto (T3) frente a la velocidad, con SLO = 0. El escalón entre 460 kbps y 1 Mbps es nítido.

La transición no es gradual sino abrupta: a 460 kbps los seis esclavos reciben la trama completa; a 921 kbps uno la recibe truncada; a 1 Mbps ninguno la recibe entera. Este punto de ruptura define la velocidad de operación del bus, y es el que el bit SLO desplaza hasta 4 Mbps.

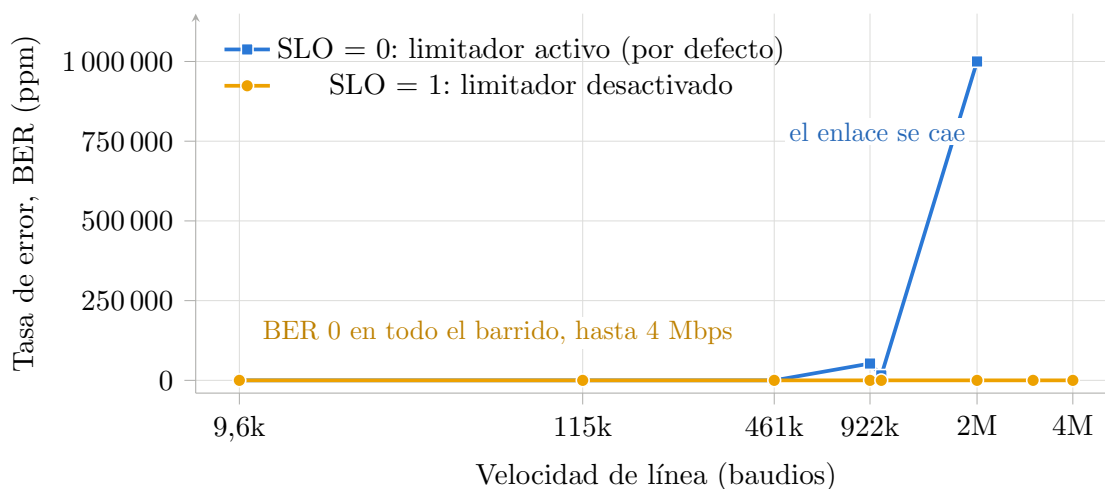


Figura 4.18: Tasa de error del bus A (RS-485 multipunto, 7 nodos) frente a la velocidad de línea, con y sin el limitador de *slew*. Con el limitador activo —el valor por defecto— el enlace se degrada a partir de 460 kbps y colapsa por completo a 2 Mbps, coherente con el máximo nominal de 250 kbps del chip en ese modo; desactivándolo la tasa de error es nula en todo el barrido. Los puntos de 3 y 4 Mbps no existen en la curva de SLO = 0 porque el barrido se interrumpe en cuanto un punto no deja pasar un solo byte correcto.

Integridad, aislamiento y estabilidad

El resto de la campaña confirma que la placa se comporta correctamente en los aspectos que no dependen de la velocidad:

Polarización de reposo (T1). Con los catorce transceptores en silencio y a la escucha durante 300 ms, no aparece **ningún byte espurio** en ningún canal. Es la comprobación de que la red de polarización de *fail-safe* está presente y cumple su función: si la línea flotara, el receptor amplificaría el ruido y lo entregaría como datos.

Aislamiento entre buses (T7). Saturando el bus A con una ráfaga a su velocidad máxima, no se filtra **ningún byte** a los buses B y C. No existe acoplamiento apreciable entre pistas ni cruces de cableado, lo que la matriz de conectividad de T2 confirma de forma independiente: cero enlaces no esperados.

Vuelta del bus (T6). En *half-duplex*, el esclavo no puede responder hasta que el maestro haya liberado la línea. Barriendo el tiempo de guarda de 0 a 1000 μ s, las ocho respuestas llegan íntegras **ya con guarda nula**: el control de dirección que genera el propio transceptor libera la línea a tiempo, por lo que un protocolo construido sobre este nivel no necesita añadir guarda por software.

Colisión (T8). Cuando dos esclavos emiten simultáneamente, la trama resulta corrupta —por el AND cableado de los drivers—, pero el bus **se recupera** y el intercambio siguiente es correcto. Ningún driver permanece conduciendo de forma indefinida.

Estabilidad sostenida (T9). Diez segundos de tráfico continuo por bus, a la velocidad máxima limpia de cada uno:

Bus	Velocidad	Tramas	Bytes	BER
A (RS-485 multipunto)	460 800	6 781	433 984	147 ppm
B (RS-422)	1 000 000	14 408	922 112	0
C (RS-422)	1 000 000	14 466	925 824	0

Tabla 4.12: Tasa de error en régimen permanente (T9), 10 s de tráfico continuo por bus.

Los dos buses RS-422 no cometen **ningún error** en casi un millón de bytes cada uno. El bus multipunto se queda en 147 ppm, una cifra baja pero no nula, coherente con la mayor exigencia de una línea compartida por siete nodos cerca de su límite de velocidad.

Limitaciones del método de medida

Deben tenerse en cuenta tres precisiones sobre el alcance de estos resultados:

Los conteos parciales de la matriz de conectividad no miden la calidad del enlace. En la matriz de T2 aparecen enlaces con 25 de 32 bytes correctos, mientras que T3 y T4 a esa misma velocidad dan seis de seis esclavos íntegros y BER nulo sobre 1024 bytes. Se trata de un artefacto del método de T2, que espera un tiempo fijo y lee una sola vez: si un byte llega tarde se pierde y descuadra la comparación. T2 es válido para determinar el **mapa** de conectividad, que es su propósito, pero no para cuantificar la calidad.

Los ocho enlaces ausentes en la matriz no constituyen un fallo. De los sesenta enlaces esperados se miden cincuenta y dos. Los ocho ausentes corresponden a los esclavos de los buses B y C, que no se oyen entre sí porque esos buses son RS-422 punto a punto en estrella desde el maestro, y no multipunto. La topología medida coincide exactamente con la física de la placa.

El contador de errores de hardware de T9 no es fiable, por lo que no se ha empleado en ninguna conclusión: marca millones de errores en tandas cuyo BER medido es nulo, lo que indica que contabiliza eventos que no son errores de trama. Las cifras de BER de la tabla 4.12 se calculan comparando byte a byte lo recibido con lo enviado.

5. Conclusiones y líneas futuras

5.1. Conclusiones

El objetivo principal de este trabajo era desarrollar una plataforma funcional de comunicación con periféricos serie sobre el MPSoC Zynq UltraScale+ ZCU102, adaptada a los requisitos del proyecto LINCE. A lo largo del desarrollo se han obtenido los siguientes resultados:

El transceptor serie configurable diseñado en VHDL funciona correctamente sobre la lógica programable del ZCU102. Soporta los estándares RS422 y RS485 con configuración completa de protocolo en tiempo de ejecución, y el NCO de 32 bits que genera su temporización, validado velocidad a velocidad con un *testbench* dedicado, presenta un error de baudrate nulo o inferior a 1 ppm en la mayoría de las velocidades estándar y de 120 ppm en el peor punto del rango, dos órdenes de magnitud por debajo de la tolerancia habitual de un receptor UART.

El driver desarrollado para RTEMS abstrae el hardware de forma efectiva y permite gestionar hasta 14 instancias simultáneas del transceptor mediante una arquitectura orientada a interrupciones. La separación entre la ISR maestra y las tareas *worker* garantiza tiempos de latencia mínimos en la atención de interrupciones, manteniendo el determinismo propio de un sistema de tiempo real.

El estudio del transporte de datos entre el PS y la PL constituye la aportación de mayor recorrido del trabajo. Se implementaron y midieron **tres arquitecturas completas** — AXI GPIO, catorce AXI DMA y un AXI MCDMA con puente VHDL propio— sobre el mismo transceptor y con el mismo programa de pruebas, y la comparación es concluyente: la arquitectura basada en AXI GPIO, que era la de partida, **interrumpe al procesador una vez por cada byte** (1250 interrupciones por kilobyte), lo que la deja en el 30 % del techo físico de la línea y, sobre todo, la vuelve insensible a la velocidad del enlace: de 115 200 a 4 Mbaudios su rendimiento apenas mejora un 41 %, porque el cuello de botella no está en la línea sino en la CPU.

La solución final —un MCDMA multicanal con un puente en VHDL que multiplexa los catorce canales— baja a **3 interrupciones por kilobyte**, alcanza el **99 % del techo físico** de la línea, reduce la latencia a un tercio y, a diferencia de la anterior, escala linealmente con la velocidad hasta los 344 KB/s a 4 Mbaudios. El camino intermedio —replicar catorce controladores DMA— se hizo funcionar en placa, pero resultó inviable no por rendimiento sino por recursos: cuesta 2,8 veces más lógica y exige veintiocho líneas de interrupción cuando el MPSoC solo ofrece ocho, un consumo de área incompatible, además, con una futura triplicación TMR de la lógica para tolerancia a la radiación. Ese límite del silicio es lo que justificó el diseño del puente propio.

Se diseñaron y fabricaron **tres** tarjetas de circuito impreso, todas ellas en el propio laboratorio mediante proceso de soldadura por reflujo con *stencil*. Las dos que responden a especificación externa —la placa CDHS y la placa AOCS— cumplen los requisitos impuestos por Indra a través de Sener y han permitido validar el sistema de comunicaciones en un entorno hardware real.

La placa de comunicación serie, con sus catorce transceptores RS-485/RS-422, se caracterizó eléctricamente con una batería de nueve tests que recorren la polarización de reposo, el mapa de conectividad real, la integridad del bus multipunto, la velocidad máxima de cada bus, la diafonía, la colisión y la estabilidad sostenida. La placa supera todos: cero bytes espurios en reposo, cero fugas entre buses, recuperación limpia tras una colisión y tasa de error nula en los dos buses RS-422 tras diez segundos de tráfico continuo. De esa campaña salió además un hallazgo con consecuencias prácticas inmediatas: la placa venía funcionando con el **limitador de slew de los transceptores activado** sin que nadie lo supiera. El pin SL0 del LTC2865 es un habilitador de modo lento **activo a nivel bajo** —con él a 0 el driver queda capado a 250 kbps nominales—, y el valor por defecto del software era precisamente 0, pese a que la macro correspondiente se llamaba «SLO desactivado». El hardware no estaba mal: era el nombre de la macro el que inducía a error. Desactivando de verdad el limitador, los tres buses pasan de 460 kbps–1 Mbps a **4 Mbps limpios**, con tasa de error nula: cuatro veces más velocidad sin tocar una sola pista de cobre ni resintetizar nada.

La validación experimental ha confirmado el correcto funcionamiento de las interfaces RS422, RS485 y CAN de la placa CDHS, así como de las interfaces RS422 y RS485 de la placa AOCS. En el caso del bus CAN, se comprobó experimentalmente la importancia de las resistencias de terminación para la integridad de la señal diferencial. El driver SPI del ADC ADS7950 produjo lecturas coherentes con la tensión de entrada aplicada, validando la cadena de adquisición analógica de la placa CDHS.

En términos de volumen, el trabajo ha producido **20.188 líneas de código** de autoría propia —excluyendo todo lo generado automáticamente por las herramientas y el código de terceros—, repartidas entre el RTL del transceptor y el puente, sus *testbenches*, el driver de RTEMS, las aplicaciones de prueba y las herramientas de automatización del flujo. El desglose completo se recoge en el anexo (tabla 5.5), y su distribución refleja las dos decisiones metodológicas que vertebraron el desarrollo: el código de verificación pesa prácticamente lo mismo que el hardware que verifica, y la automatización del flujo, con un 40 % del total, es la mayor partida individual del trabajo.

En conjunto, el trabajo ha servido como contribución directa al proyecto LINCE, proporcionando al laboratorio B105 una base funcional y documentada para el subsistema de comunicaciones del ordenador de a bordo del satélite.

5.2. Líneas futuras

A partir del trabajo realizado, se identifican las siguientes líneas de continuación:

Validación SpaceWire. La interfaz SpaceWire de la placa AOCS quedó fuera del alcance por su alta complejidad técnica: la placa expone la capa física LVDS, pero ejercitarla exige implementar en la lógica programable la codificación *data-strobe*, la máquina de estados de establecimiento del enlace y los niveles superiores del estándar. Completar esta validación

es el paso inmediato más relevante, ya que SpaceWire es el protocolo de alta velocidad previsto para los enlaces de mayor ancho de banda del satélite.

Corregir el valor por defecto del bit SLO y operar la plataforma a 4 Mbps.

La caracterización de la PCB (sección 4.3.3) demuestra que el limitador de *slew* de los transceptores está activo con la configuración por defecto, y que desactivándolo los tres buses pasan de 460 kbps–1 Mbps a 4 Mbps limpios. El cambio es de una línea, pero arrastra tres tareas: hay que **renombrar la macro** en `transceiver.h`, que es la que indujo el error; **invertir el valor por defecto** del driver, hoy todavía en el modo lento; y **repetir la campaña de estabilidad** a 4 Mbps, donde el bus multipunto —el más cargado— tendrá menos margen que los dos enlaces punto a punto.

Aprovechar el limitador de slew para lo que sirve. Una vez entendido que el modo lento es el que reduce las emisiones electromagnéticas a costa de ancho de banda, el bit deja de ser una fuente de confusión y pasa a ser una herramienta: permite elegir, canal a canal y en caliente, entre un enlace rápido y uno de baja emisión. En un satélite, donde la compatibilidad electromagnética es un requisito duro, es una palanca que merece la pena caracterizar de forma explícita, midiendo la emisión radiada en ambos modos.

Reducir la tasa de error del bus multipunto. El bus A sostiene 147 ppm a 460 kbps, frente al cero absoluto de los dos buses RS-422. La cifra es baja y coherente con siete nodos colgados de una línea compartida, pero merece la pena investigar si se debe a la terminación, a los *stubs* de derivación o simplemente a estar operando cerca del límite de velocidad del bus en la configuración por defecto.

Completar la medida de la variante DMA14. El módulo de *loopback* no llegó a funcionar sobre la arquitectura de catorce controladores DMA, y se optó por no seguir depurándolo y dedicar el tiempo a la variante MCDMA, de modo que sus cifras de recepción no son comparables y en las tablas del capítulo de resultados aparecen como no disponibles. Retomar esa depuración y repetir la captura con el lazo cerrado cerraría la comparativa por completo. No se espera que cambie la conclusión —el argumento contra esa arquitectura es de recursos y de líneas de interrupción, no de rendimiento— pero sí completaría el dato.

Endurecimiento frente a radiación: replicar la variante MCDMA con TMR.

La lógica programable comercial es susceptible a los efectos de la radiación en órbita —en particular a los *single event upsets* (SEU), que alteran el contenido de biestables y de la memoria de configuración—, y la técnica habitual de mitigación es la triplicación modular redundante (TMR): tres copias de la lógica y módulos votadores que enmascaran el fallo de cualquiera de ellas. La línea de trabajo concreta es replicar el proyecto de la variante MCDMA triplicando su lógica propia —el puente VHDL y los transceptores serie—, incorporar los votadores necesarios y demostrar que el sistema sigue superando la misma campaña de treinta y seis comprobaciones. El presupuesto de área lo permite: triplicar el 5,2 % de ocupación del MCDMA mantiene el diseño en torno al 16 % del dispositivo, mientras que la misma operación sobre la variante DMA14 rondaría el 44 % y la haría inviable —es el argumento de futuro que ya pesó en su descarte (sección 4.3.2).

Eliminar el *store-and-forward* del canal de recepción del MCDMA. El límite de 512 bytes por paquete que impone la configuración actual del IP obliga al bloque de drenado de la PL a trocear los bloques a 256 bytes. Regenerar el `axi_mcdma` con `c_include_s2mm_sf = 0` eliminaría esa restricción y permitiría paquetes de longitud arbitraria, con una reducción adicional del número de interrupciones en transferencias largas.

Explotar el margen de CPU liberado. La consecuencia práctica más interesante del cambio de transporte es que el procesador ha quedado libre: se ha pasado de 1250 a 3 interrupciones por kilobyte. Ese margen es precisamente lo que necesita el software de vuelo para ejecutar las tareas de control y gestión de datos del OBC mientras los catorce enlaces serie funcionan a plena velocidad. Cuantificar la carga de CPU real del sistema completo, con la aplicación de vuelo corriendo en paralelo al tráfico serie, es el siguiente paso natural de la caracterización.

Integración con el stack de software de vuelo de Sener. El driver y las PCBs han sido diseñados con los requisitos de Indra/Sener como guía. El siguiente paso natural es la integración del driver en el stack de software de vuelo de Sener y la realización de pruebas de validación conjuntas en entorno *Hardware-in-the-Loop*.

Soporte de protocolos adicionales. La arquitectura modular del transceptor VHDL facilita la incorporación de nuevos modos de operación. Una extensión interesante sería añadir soporte nativo del protocolo MIL-STD-1553, ampliamente utilizado en sistemas espaciales de mayor herencia.

Bibliografía

- [1] AMD/Xilinx. *ZCU102 Evaluation Board User Guide*. AMD/Xilinx, 2023. Copia disponible en `tfm/00_docs/ug1182-zcu102-eval-bd.pdf` del repositorio del proyecto.
- [2] Indra. Proyecto LINCE: Línea de industrialización de cargas de pago y plataformas espaciales. Indra Sistemas S.A., <https://www.ptelince.es/>, 2026. Recuperado el 12 de junio de 2026.
- [3] Jorge Antonio Estefanía Hidalgo. Repositorio de código del TFM: transceptores serie configurables y transporte de datos sobre Zynq UltraScale+. <https://github.com/jaestefaniah27/tfm>, 2026. Recuperado el 6 de agosto de 2026.
- [4] VITA Standards Organization. VITA 57.1: FPGA Mezzanine Card (FMC) standard. VITA Standards Organization, 2010.
- [5] Texas Instruments. The RS-485 design guide. Application Report SLLA272C, Texas Instruments, 2014.
- [6] Electronic Industries Alliance (EIA). Electrical characteristics of generators and receivers for use in balanced digital multipoint systems. TIA/EIA-485-A, 1998.
- [7] Texas Instruments. RS-422 and RS-485 standards overview and system configurations. Application Report SLLA070D, Texas Instruments, 2018.
- [8] Telecommunications Industry Association. Electrical characteristics of balanced voltage digital interface circuits. TIA/EIA-422-B, 1994.
- [9] European Cooperation for Space Standardization (ECSS). Space engineering – SpaceWire – links, nodes, routers and networks, 2008.
- [10] S. M. Parkes. *SpaceWire User's Guide*. STAR-Dundee Ltd., 2012.
- [11] International Organization for Standardization (ISO). Road vehicles — controller area network (CAN) — part 1: Data link layer and physical signalling. ISO 11898-1:2015, 2015.
- [12] Agencia Espacial Europea (ESA). CAN in space: Implementation guide and application notes. Technical report, TEC-ED & TEC-SW, ESA, 2015.
- [13] ARM Ltd. *AMBA AXI and ACE Protocol Specification (AXI3, AXI4, AXI4-Lite, ACE, ACE-Lite)*. ARM Ltd., 2011.
- [14] AMD/Xilinx. *AXI DMA v7.1 — LogiCORE IP Product Guide*. AMD/Xilinx, 2022.
- [15] AMD/Xilinx. *AXI Multichannel DMA (MCDMA) v1.1 — LogiCORE IP Product Guide*. AMD/Xilinx, 2022.
- [16] RTEMS Project. *RTEMS User Manual (RTEMS 7) — Cap. 2: Quick Start*. RTEMS Project, 2026. Guía paso a paso de instalación del *toolchain* y compilación de los BSP mediante el RTEMS Source Builder. Consultado en 2026.

- [17] RTEMS Project. *RTEMS User Manual (RTEMS 7) — Cap. 15: Source Builder (RSB)*. RTEMS Project, 2026. Consultado en 2026.
- [18] Reinhold P. Weicker. Dhrystone: A synthetic systems programming benchmark. *Communications of the ACM*, 27(10):1013–1030, 1984.
- [19] PCBWay. PCB capabilities: Manufacturing tolerances and design limits. <https://www.pcbway.com/capabilities.html>, 2025. Consultado en 2025.
- [20] Analog Devices. *LTC2862/LTC2863/LTC2864/LTC2865 — $\pm 60V$ Fault Protected 3V to 5.5V RS485/RS422 Transceivers*. Analog Devices (Linear Technology), 2013. Documento 2862345fc.
- [21] Texas Instruments. *ADS7950/51/52/53 — 12/10/8-bit, 1-MSPS, 4-/8-channel, serial interface, MicroPower sampling analog-to-digital converter*. Technical Report SLAS605C, Texas Instruments, 2018.
- [22] AMD/Xilinx. *Zynq UltraScale+ MPSoC Technical Reference Manual*. AMD/Xilinx, 2023.

Anexo 1: Mapas de señales y tabla NCO

Este anexo recoge el mapa de señales completo entre los conectores FMC de la ZCU102 y las placas LINCE Comunicación Serial, CDHS y AOCS, así como la tabla de frecuencias soportadas por el NCO del transceptor serie junto con su error medido.

A1.1 Mapa de señales LINCE Comunicación Serial

La siguiente tabla recoge el mapeado de los 14 drivers RS-485/RS-422 de la placa LINCE Comunicación Serial con el conector FMC HPC0 (P2) de la ZCU102. Cada driver expone las señales TX, RX, SLO y DE (esta última ausente en los drivers 7 y 11, que sólo disponen de control de dirección hardware).

Driver	Señal	Red Interna	Red (FMC HPC0)	Pin FMC (P2)	Pin Zynq
Driver0	TX	TX0	FMC_HPC0_LA32_N	H38	T11
Driver0	RX	RX0	FMC_HPC0_LA26_N	D27	K15
Driver0	SLO	SLO0	FMC_HPC0_LA27_N	C27	L10
Driver0	DE	DE0	FMC_HPC0_LA33_N	G37	V11
Driver1	TX	TX1	FMC_HPC0_LA33_P	G36	V12
Driver1	RX	RX1	FMC_HPC0_LA30_N	H35	U6
Driver1	SLO	SLO1	FMC_HPC0_LA32_P	H37	U11
Driver1	DE	DE1	FMC_HPC0_LA27_P	C26	M10
Driver2	TX	TX2	FMC_HPC0_LA30_P	H34	V6
Driver2	RX	RX2	FMC_HPC0_LA26_P	D26	L15
Driver2	SLO	SLO2	FMC_HPC0_LA31_N	G34	V7
Driver2	DE	DE2	FMC_HPC0_LA31_P	G33	V8
Driver3	TX	TX3	FMC_HPC0_LA29_N	G31	U8
Driver3	RX	RX3	FMC_HPC0_LA29_P	G30	U9
Driver3	SLO	SLO3	FMC_HPC0_LA28_N	H32	T6
Driver3	DE	DE3	FMC_HPC0_LA28_P	H31	T7
Driver4	TX	TX4	FMC_HPC0_LA14_P	C18	AC7
Driver4	RX	RX4	FMC_HPC0_LA13_N	D18	AC8
Driver4	SLO	SLO4	FMC_HPC0_LA22_P	G24	M15

Driver	Señal	Red Interna	Red (FMC HPC0)	Pin FMC (P2)	Pin Zynq
Driver4	DE	DE4	FMC_HPC0_LA19_N	H23	K13
Driver5	TX	TX5	FMC_HPC0_LA20_N	G22	M13
Driver5	RX	RX5	FMC_HPC0_LA15_N	H20	Y9
Driver5	SLO	SLO5	FMC_HPC0_LA19_P	H22	L13
Driver5	DE	DE5	FMC_HPC0_LA20_P	G21	N13
Driver6	TX	TX6	FMC_HPC0_LA15_P	H19	Y10
Driver6	RX	RX6	FMC_HPC0_LA13_P	D17	AB8
Driver6	SLO	SLO6	FMC_HPC0_LA16_N	G19	AA12
Driver6	DE	DE6	FMC_HPC0_LA16_P	G18	Y12
Driver7	TX	TX7	FMC_HPC0_LA06_P	C10	AC2
Driver7	RX	RX7	FMC_HPC0_LA01_CC_N	D9	AC4
Driver7	SLO	SLO7	FMC_HPC0_LA01_CC_P	D8	AB4
Driver8	TX	TX8	FMC_HPC0_LA05_P	D11	AB3
Driver8	RX	RX8	FMC_HPC0_LA05_N	D12	AC3
Driver8	SLO	SLO8	FMC_HPC0_LA06_N	C11	AC1
Driver8	DE	DE8	FMC_HPC0_LA10_P	C14	W5
Driver9	TX	TX9	FMC_HPC0_LA02_P	H7	V2
Driver9	RX	RX9	FMC_HPC0_LA00_CC_N	G7	Y3
Driver9	SLO	SLO9	FMC_HPC0_LA00_CC_P	G6	Y4
Driver9	DE	DE9	FMC_HPC0_LA02_N	H8	V1
Driver10	TX	TX10	FMC_HPC0_LA09_P	D14	W2
Driver10	RX	RX10	FMC_HPC0_LA03_N	G10	Y1
Driver10	SLO	SLO10	FMC_HPC0_LA03_P	G9	Y2
Driver10	DE	DE10	FMC_HPC0_LA04_P	H10	AA2
Driver11	TX	TX11	FMC_HPC0_LA08_N	G13	V3
Driver11	RX	RX11	FMC_HPC0_LA08_P	G12	V4
Driver11	SLO	SLO11	FMC_HPC0_LA04_N	H11	AA1
Driver12	TX	TX12	FMC_HPC0_LA10_N	C15	W4
Driver12	RX	RX12	FMC_HPC0_LA07_N	H14	U4
Driver12	SLO	SLO12	FMC_HPC0_LA07_P	H13	U5
Driver12	DE	DE12	FMC_HPC0_LA09_N	D15	W1
Driver13	TX	TX13	FMC_HPC0_LA11_N	H17	AB5
Driver13	RX	RX13	FMC_HPC0_LA11_P	H16	AB6
Driver13	SLO	SLO13	FMC_HPC0_LA12_P	G15	W7
Driver13	DE	DE13	FMC_HPC0_LA12_N	G16	W6

Tabla 5.1: Mapa de señales entre la placa LINCE Comunicación Serial y el conector FMC HPC0 (P2) de la ZCU102.

A1.2 Mapa de señales CDHS

Subsistema	Señal	Red (FMC HPC0)	Pin FMC	Pin Zynq
SPI (ADC)	SDO	FMC_HPC0_LA02_N	H8	V1
SPI (ADC)	SDI	FMC_HPC0_LA00_CC_N	G7	Y3
SPI (ADC)	SCLK	FMC_HPC0_LA02_P	H7	V2
SPI (ADC)	CS_N	FMC_HPC0_LA00_CC_P	G6	Y4
CAN (Nominal)	TX_N	FMC_HPC0_LA21_P	H25	P12
CAN (Nominal)	RX_N	FMC_HPC0_LA22_P	G24	M15
CAN (Redundante)	TX_R	FMC_HPC0_LA21_N	H26	N12
CAN (Redundante)	RX_R	FMC_HPC0_LA22_N	G25	M14
RS1 (Serial 1)	TX	FMC_HPC0_LA15_P	H19	Y10
RS1 (Serial 1)	RX	FMC_HPC0_LA11_N	H17	AB5
RS1 (Serial 1)	DE	FMC_HPC0_LA16_P	G18	Y12
RS2 (Serial 2)	TX	FMC_HPC0_LA12_N	G16	W6
RS2 (Serial 2)	RX	FMC_HPC0_LA12_P	G15	W7
RS2 (Serial 2)	DE	FMC_HPC0_LA11_P	H16	AB6
RS3 (Serial 3)	TX	FMC_HPC0_LA07_N	H14	U4
RS3 (Serial 3)	RX	FMC_HPC0_LA07_P	H13	U5
RS3 (Serial 3)	DE	FMC_HPC0_LA08_N	G13	V3
PWM (Heaters)	IN1	FMC_HPC0_LA04_N	H11	AA1
PWM (Heaters)	IN2	FMC_HPC0_LA04_P	H10	AA2
PWM (Heaters)	IN3	FMC_HPC0_LA03_P	G9	Y2
PWM (Heaters)	IN4	FMC_HPC0_LA03_N	G10	Y1

Tabla 5.2: Mapa de señales entre la placa CDHS y el conector FMC HPC0 de la ZCU102.

A1.3 Mapa de señales AOCs

Subsistema	Señal	Red (FMC HPC0)	Pin FMC	Pin Zynq
RS1 (Serial J1)	TX	FMC_HPC0_LA20_N	G22	M13
RS1 (Serial J1)	RX	FMC_HPC0_LA20_P	G21	N13
RS1 (Serial J1)	DE	FMC_HPC0_LA19_P	H22	L13
MOT-PWM (J2)	PWM_X_1	FMC_HPC0_LA17_CC_N	D21	N11
MOT-PWM (J2)	PWM_X_2	FMC_HPC0_LA15_N	H20	Y9
MOT-PWM (J2)	PWM_Y_1	FMC_HPC0_LA16_N	G19	AA12
MOT-PWM (J2)	PWM_Y_2	FMC_HPC0_LA15_P	H19	Y10
MOT-PWM (J2)	PWM_Z_1	FMC_HPC0_LA16_P	G18	Y12
MOT-PWM (J2)	PWM_Z_2	FMC_HPC0_LA13_N	D18	AC8
RS3 (Serial J3)	TX	FMC_HPC0_LA11_N	H17	AB5
RS3 (Serial J3)	RX	FMC_HPC0_LA12_P	G15	W7
RS3 (Serial J3)	DE	FMC_HPC0_LA11_P	H16	AB6
RS4 (Serial J4)	TX	FMC_HPC0_LA09_P	D14	W2
RS4 (Serial J4)	RX	FMC_HPC0_LA08_N	G13	V3
RS4 (Serial J4)	DE	FMC_HPC0_LA07_N	H14	U4
RS5 (Serial J5)	TX	FMC_HPC0_LA07_P	H13	U5
RS5 (Serial J5)	RX	FMC_HPC0_LA05_P	D11	AB3
RS5 (Serial J5)	DE	FMC_HPC0_LA08_P	G12	V4
SPW1 (J6)	DIN_P	FMC_HPC0_LA02_P	H7	V2
SPW1 (J6)	DIN_N	FMC_HPC0_LA02_N	H8	V1
SPW1 (J6)	SIN_P	FMC_HPC0_LA04_P	H10	AA2
SPW1 (J6)	SIN_N	FMC_HPC0_LA04_N	H11	AA1
SPW1 (J6)	SOUT_P	FMC_HPC0_LA01_CC_P	D8	AB4
SPW1 (J6)	SOUT_N	FMC_HPC0_LA01_CC_N	D9	AC4
SPW1 (J6)	DOUT_P	FMC_HPC0_LA00_CC_P	G6	Y4
SPW1 (J6)	DOUT_N	FMC_HPC0_LA00_CC_N	G7	Y3
SPW2 (J7)	DIN_P	FMC_HPC0_LA21_P	H25	P12
SPW2 (J7)	DIN_N	FMC_HPC0_LA21_N	H26	N12
SPW2 (J7)	SIN_P	FMC_HPC0_LA24_P	H28	L12
SPW2 (J7)	SIN_N	FMC_HPC0_LA24_N	H29	K12
SPW2 (J7)	SOUT_P	FMC_HPC0_LA28_P	H31	T7
SPW2 (J7)	SOUT_N	FMC_HPC0_LA28_N	H32	T6
SPW2 (J7)	DOUT_P	FMC_HPC0_LA30_P	H34	V6
SPW2 (J7)	DOUT_N	FMC_HPC0_LA30_N	H35	U6
RS8 (Serial J8)	TX	FMC_HPC0_LA22_N	G25	M14
RS8 (Serial J8)	RX	FMC_HPC0_LA19_N	H23	K13
RS8 (Serial J8)	DE	FMC_HPC0_LA22_P	G24	M15

Tabla 5.3: Mapa de señales entre la placa AOCs y el conector FMC HPC0 de la ZCU102.

A1.4 Tabla de frecuencias del NCO

La siguiente tabla recoge las 54 frecuencias de baudrate soportadas por el oscilador controlado numéricamente (NCO) de 32 bits del transceptor serie, junto con los valores de incremento configurados y el error medido en ppm respecto a la frecuencia nominal.

Baud	Half	INC_ROM	INC_HALF_ROM	Medido (Hz)	Error (ppm)
9600	0	412317	824634	9600,002	0,2
14400	0	618475	1236951	14399,988	-0,8
19200	0	824634	1649267	19200,012	0,6
28800	0	1236951	2473901	28799,977	-0,8
31250	0	1342177	2684355	31250,000	0,0
38400	0	1649267	3298535	38400,025	0,6
56000	0	2405182	4810363	55999,978	-0,4
57600	0	2473901	4947802	57600,037	0,6
74400	0	3195456	6390911	74400,057	0,8
115200	0	4947802	9895605	115200,074	0,6
128000	0	5497558	10995116	128000,000	0,0
153600	0	6597070	13194140	153600,393	2,6
230400	0	9895605	19791209	230398,820	-5,1
256000	0	10995116	21990233	256000,000	0,0
312500	0	13421773	26843546	312500,000	0,0
460800	0	19791209	39582419	460808,258	17,9
500000	0	21474836	42949673	500000,000	0,0
576000	0	24739012	49478023	576003,686	6,4
614400	0	26388279	52776558	614401,573	2,6
750000	0	32212255	64424509	749990,625	-12,5
921600	0	39582419	79164837	921616,515	17,9
1000000	0	42949673	85899346	1000000,000	0,0
1152000	0	49478023	98956046	1152007,373	6,4
1500000	0	64424509	128849019	1499925,004	-50,0
1843200	0	79164837	158329674	1843148,097	-28,2
2000000	0	85899346	171798692	2000000,000	0,0
2500000	0	107374182	214748365	2500000,000	0,0
3000000	0	128849019	257698038	3000300,030	100,0
3686400	0	158329674	316659349	3685956,506	-120,3

Tabla 5.4: Tabla NCO: frecuencias de baudrate soportadas y error medido (extracto, modo *full-bit*).

A1.5 Volumen y distribución del código desarrollado

Se recoge aquí el recuento de líneas de código de autoría propia escritas en el trabajo, desglosado por lenguaje y por función. El criterio de recuento es restrictivo: se excluye todo el código generado automáticamente por las herramientas (los IPs de Xilinx, los ficheros que Vivado produce al empaquetar un IP o al sintetizar un diseño de bloques), el código de terceros incorporado al proyecto (el sistema de compilación `waf` de RTEMS, el árbol de dispositivos de Xilinx) y las copias de un mismo fichero replicadas entre las distintas variantes del proyecto, de las que se contabiliza únicamente la versión final.

Lenguaje	Función	Líneas	Peso
TCL	Automatización del flujo de Vivado	5.338	26,4 %
VHDL	RTL sintetizable (transceptor, puente)	4.710	23,3 %
VHDL	<i>Testbenches</i> de simulación	4.551	22,5 %
C	Driver RTEMS y aplicaciones de prueba	2.617	13,0 %
Python	Utilidades de instrumentación y GUI	1.525	7,6 %
Bash	Compilación y despliegue de imágenes	1.231	6,1 %
XDC	<i>Constraints</i> de la FPGA	216	1,1 %
Total		20.188	100 %

Tabla 5.5: Distribución de las líneas de código desarrolladas en el trabajo.

La distribución es en sí misma un resultado, y admite dos lecturas que respaldan los principios metodológicos enunciados en la sección de metodología.

La primera es que el **código de verificación pesa prácticamente lo mismo que el código que verifica**: 4.551 líneas de *testbench* frente a 4.710 de VHDL sintetizable, una relación de 0,97:1. Por cada línea de hardware descrita se escribió casi una línea destinada exclusivamente a comprobarla en simulación, lo que cuantifica la política de no integrar ningún bloque sin verificación previa aislada.

La segunda es que la **automatización es la mayor partida del trabajo**: TCL, Bash y Python suman 8.094 líneas, el 40,1 % del total, por encima de cualquier otra categoría y muy por encima del código de aplicación. Es la traducción cuantitativa de una decisión deliberada —invertir por adelantado en herramientas que hicieran el flujo reproducible y barato de iterar— cuyo retorno se manifestó en las campañas de depuración, donde regenerar el diseño completo desde cero pasó a ser un comando en lugar de una tarde de trabajo manual.

Anexo A: Aspectos éticos, económicos, sociales y ambientales

El apartado “Requisitos de las acreditaciones internacionales EUR-ACE y ABET” de la Normativa de TFT de la ETSIT-UPM establece que *“La memoria del TFT del GITST, GIB y MUIT, y en general la de aquellas titulaciones que hayan obtenido o para las que se desee solicitar una acreditación internacional EUR-ACE o ABET, debe mostrar conciencia de la responsabilidad de la aplicación práctica de la ingeniería, el impacto social y ambiental, el compromiso con la ética profesional, la responsabilidad y las normas de la aplicación práctica de la ingeniería, así como sobre las prácticas de gestión de proyectos, gestión, y control de riesgos, entendiendo sus limitaciones”*.

Este anexo obligatorio del TFT tendrá un carácter sintético con los siguientes apartados:

A1. INTRODUCCIÓN

Este trabajo se enmarca en el proyecto LINCE, una iniciativa del PERTE Aeroespacial para el desarrollo de microsátélites LEO liderada por Indra. El objetivo es desarrollar el subsistema de comunicaciones serie del ordenador de a bordo (OBC) del satélite, cubriendo tanto el diseño de hardware electrónico (PCBs de prueba) como el firmware embebido sobre un sistema operativo de tiempo real. Este tipo de tecnología tiene implicaciones relevantes en ámbitos sociales, económicos, ambientales y éticos que se analizan a continuación.

A2. DESCRIPCIÓN DE IMPACTOS RELEVANTES RELACIONADOS CON EL PROYECTO

[Pendiente de redacción — describir impactos en: soberanía tecnológica espacial, doble uso civil/militar de la tecnología satelital, residuos electrónicos (PCBs), consumo energético de la FPGA, implicaciones de los sistemas de observación LEO.]

A3. ANÁLISIS DETALLADO DE ALGUNO DE LOS IMPACTOS

[Pendiente de redacción — análisis en profundidad del impacto seleccionado.]

A4. CONCLUSIONES

[Pendiente de redacción — valoración del proyecto desde el punto de vista ético, social, económico y medioambiental.]

Anexo B: Presupuesto económico

El apartado “Requisitos de las acreditaciones internacionales EUR-ACE y ABET” de la Normativa de TFT de la ETSIT-UPM establece que “*La memoria del TFT del GITST, GIB y MUIT, y en general la de aquellas titulaciones que hayan obtenido o para las que se desee solicitar una acreditación internacional EUR-ACE o ABET, ... debe incluir un presupuesto económico*”. A modo de ejemplo, la tabla del presupuesto de un proyecto podría ser la siguiente:

COSTE DE MANO DE OBRA (coste directo)		Horas	Precio/hora	Total
		300	15 €	4.500 €

COSTE DE RECURSOS MATERIALES (coste directo)	Precio de compra	Uso en meses	Amortización (en años)	Total
Ordenador personal (Software incluido).....	1.500,00 €	6	5	150,00 €
Impresora láser	500,00 €	6	5	50,00 €
Otro equipamiento				

COSTE TOTAL DE RECURSOS MATERIALES				200,00 €
------------------------------------	--	--	--	----------

GASTOS GENERALES (costes indirectos)	15 %	sobre CD	705,00 €
BENEFICIO INDUSTRIAL	6 %	sobre CD + CI	324,30 €

MATERIAL FUNGIBLE			
Impresión			100,00 €
Encuadernación			300,00 €

SUBTOTAL PRESUPUESTO			6.129,30 €
IVA APLICABLE		21 %	1.287,15 €

TOTAL PRESUPUESTO			7.416,45 €
-------------------	--	--	------------

Tabla 5.6: Presupuesto económico